# Foundations of the Synthetic Control Method

Jared Greathouse

# Table of contents

# Chapter 1

# Introduction

Hey, I'm Jared, a PhD candidate studying public policy at Georgia State University. I work in econometrics for policy analysis. Welcome to *Foundations of the Synthetic Control Method* (SCM). This book is a practical guide to mastering SCM, directed at graduate students and industry scientists seeking an intuitive yet rigorous understanding of this transformative method for policy and geolift analysis (the study of location-based impacts in marketing, using holdout studies).

One idea runs through every chapter: a synthetic control is just a carefully chosen weighted average. We build it up slowly — from a plain mean, to regression, to the convex synthetic control — so that nothing ever arrives as a formula you are asked to accept on faith.

## 1.1 Why I Wrote This Book?

While excellent articles on SCM exist, they often skip critical details that cannot fit in a single paper. This book bridges that gap, offering a clear, hands-on introduction to SCM's mechanics and applications without overwhelming readers with advanced theory.

I remember reading in masters school articles/book chapters (Nguyen, 2025; Stöffelbauer, 2023) that said things like

> In many real-world cases, however, we actually find ourselves more interested in understanding a causal effect rather than correlations. Part of the reason is that causal effects are more robust because they do not suffer as much from model drift, and they are a great tool for supporting data-driven decisions. This article is about one such method, called the synthetic control method (SCM). In short, its main idea is to use a synthetic control group instead of a randomized control group, which is what is typically done in A/B testing. As a result, SCM is significantly easier to apply in various use cases.

or

> The Synthetic Control Method (SCM) is a powerful causal inference tool used

> when only one treated unit exists, and researchers must construct a synthetic comparison group from a donor pool of untreated units.

These definitions only confused me. Both of them used the word "synthetic" in the definition. Other articles would simply present what I now know to be the dot product or the convex optimization formula, say a few words about it, and move on. Almost no resource, that I could find, bothered to start from the barebones first principles and walk through what was meant as a concept by a "fake unit" or a "weighted average" or how it differed from a regular average. So, this book has a plain quest: explain the synthetic control method to students and working professionals who may use it for their own tasks.

## 1.2 Who This Book Is For

This book is for graduate students and practitioners in economics, social sciences, or related fields who want to apply SCM in practice. No prior SCM experience is needed, but readers should be comfortable with algebra, causal reasoning, and basic econometrics such as regression analysis. You do not need linear algebra or calculus going in: the book builds the specific math it uses — vectors, dot products, matrices, a derivative here and there — from scratch, at the moment each idea is first needed, rather than front-loading it in a chapter you have to survive before the payoff. If you want a full course in those subjects, the standard references are excellent (e.g., Gilbert Strang's Linear Algebra for the linear algebra, the numerous open source texts on causal inference, or the list of free Python books).

This is the master's volume. It deliberately stops short of the advanced extensions — proximal inference, matrix completion, staggered adoption, and the heavier asymptotic theory — which are the subject of a planned second, more technical volume. Where those methods matter, the closing chapter points to them; it does not develop them here.

## 1.3 Personal Reflection

### 1.3.1 Historical & Methodological Context

Given the advent of the computing advances of the 20th century, economics, public policy/political science, and related fields developed an increasing reliance on computing methods in academic and industrial research. Part of this transition involved a more careful concern with causal impact estimation methods. Sometimes called the credibility revolution, social scientists of all varieties began to ask questions such as "What was the effect of Policy $X$ on Outcome $Y$?"

In the early 1990s and 2000s, methods such as propensity score matching, difference-in-differences, instrumental variable regression models, and others became important weapons in the toolkit of empirical scientists. Across the 2000s and 2010s, the SCM — developed by Alberto Abadie and his advisor Javier Gardeazabal — was formalized in articles published in *Journal of the American Statistical Association* and *American Journal of Political Science*.

### 1.3.2 My Journey with SCM

My journey with the SCM began in 2021. I had just become a PhD student of public policy at Georgia State University, and then the Georgia Institute of Technology. The COVID-19 pandemic had begun only a year and a few months earlier. Young and aspiring economists, computer scientists, and policy scholars rushed to use SCMs to evaluate the effect of public policy interventions on COVID-19 cases, vaccination outcomes, and other outcomes of interest.

Throughout my undergraduate years, I had heard about how SCM was a kind of generalization of the difference-in-differences method (another design one could write several books on). However, the reasoning for this to me was very arcane. Furthermore, the math behind SCM intimidated me; I was a starting-level PhD student with little background in calculus or real analysis. Either way, these questions burnt at me from within, and I sought to develop a better understanding of SCM for my own work. This is the book I wish I had learning the method years ago, coming from a very non-technical background.

## 1.4 Structure of the Book

The book has one organizing idea: we build the counterfactual — the synthetic control — one rung at a time, each method motivated by the limits of the one before it. It is organized in three parts.

1. *Foundations.* The mathematical vocabulary you will actually use, introduced as it comes up, and the central insight that every estimator in the book is a structured weighted average.
2. *Estimation and inference.* The ladder itself: from the potential-outcomes framing and the plain control-group mean, through least squares and penalized regression, up to the convex synthetic control — and then how to validate an estimate and attach uncertainty to it.
3. *Applications.* How to choose a donor pool, a full end-to-end study, and a short closing survey of what lies beyond this volume — proximal methods, matrix completion, staggered designs — as signposts to the second book rather than a treatment of them.

Each section ends with problems to work, and every method is demonstrated in `mlsynth`, the companion Python library, so you can run everything as you read.

# Part I

# Foundations

# Chapter 2

# Mathematical Preliminaries

> "If you stop at general math, you're only going to make general math money."
> — Snoop Dogg

This chapter collects the minimal, concrete mathematical vocabulary used throughout the book. Worked examples illustrate each concept. These tools will appear frequently in synthetic control methods: sets, vectors, matrices, norms, derivatives, objective functions, and convexity.

## 2.1 Objects

**Definition 2.1** (Topological Space)**.** A *topological space* is a set equipped with a notion of "closeness" or limits between elements. For our purposes, the most common topological space is the set of real numbers:

$$\mathbb{R} = \text{the set of real numbers.}$$

**Worked example:** $y_1 = 2.5, y_2 = 4.0, y_3 = 3.1 \implies (y_1, y_2, y_3) \in \mathbb{R}^3$.

This is also called the real line. The study of the real numbers and the functions that live within it is called *Real Analysis*. However, you will not need any real analysis to read this book.

**Definition 2.2** (Set)**.** A *set* is a collection of distinct and well-defined objects, called *elements* or *members* of the set. If an object $x$ belongs to a set $\mathcal{A}$, we write $x \in \mathcal{A}$; otherwise, $x \notin \mathcal{A}$. Sets are unordered and contain no duplicates. Two sets are equal if and only if they contain exactly the same elements. Or more formally, by the axiom of extensionality:

$$\mathcal{A} = \mathcal{B} \iff (\forall x)(x \in \mathcal{A} \leftrightarrow x \in \mathcal{B}).$$

The set with no elements is called the empty set, denoted $\emptyset$. For example:

$$\mathcal{A} = \{1, 2, 3\}, \quad \mathcal{B} = \{x \in \mathbb{N} : x \text{ is even}\}, \quad \mathcal{C} = \emptyset.$$

The union of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of all elements that belong to either $\mathcal{A}$ or $\mathcal{B}$:

$$\mathcal{A} \cup \mathcal{B} = \{x : x \in \mathcal{A} \text{ or } x \in \mathcal{B}\}.$$

The intersection of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of all elements that belong to both $\mathcal{A}$ and $\mathcal{B}$:

$$\mathcal{A} \cap \mathcal{B} = \{x : x \in \mathcal{A} \text{ and } x \in \mathcal{B}\}.$$

The difference of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of elements in $\mathcal{A}$ but not in $\mathcal{B}$:

$$\mathcal{A} \setminus \mathcal{B} = \{x : x \in \mathcal{A} \text{ and } x \notin \mathcal{B}\}.$$

If $\mathcal{U}$ is the universal set, the complement of a set $\mathcal{A}$ is:

$$\mathcal{A}^c = \{x \in \mathcal{U} : x \notin \mathcal{A}\}.$$

The cardinality of a set $\mathcal{A}$ is the number of elements in the set, denoted $|\mathcal{A}|$.

In econometrics, sets appear frequently to organize units, time periods, and other objects of interest. For example, let $\mathcal{N}$ denote the set of all units, and $\mathcal{N}_0 \subseteq \mathcal{N}$ the donor set. Let $\mathcal{T}$ be the set of all time periods, with $\mathcal{T}_1 = \{t \in \mathbb{N} : t \leq T_0\} \subseteq \mathcal{T}$ representing the pre-intervention periods, and $\mathcal{T}_2 = \{t \in \mathbb{N} : t > T_0\} \subseteq \mathcal{T}$ representing the post-treatment periods. Sets in econometrics can contain numbers, individuals, time periods, or even other sets, depending on the application.

**Definition 2.3** (Scalar)**.** A *scalar* is a single real number, $x \in \mathbb{R}$.

**Worked example:** $c = 2.5 \cdot q$; if $q = 4$, $c = 10$.
In SCM, $y_{jt} \in \mathbb{R}$ is an observed outcome, $d_j \in \{0, 1\}$ is treatment.

We work with these all the time.

**Definition 2.4** (Summation)**.** For numbers $x_1, \ldots, x_n$:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

**Worked example:** $x_1 = 2, x_2 = 4, x_3 = 5 \implies \sum_{i=1}^{3} x_i = 11$.

This offers a convenient way to express summations of scalars.

**Definition 2.5** (Sequence)**.** A *sequence* is a function

$$a : \mathbb{N} \to S$$

where $\mathbb{N} = \{1, 2, 3, \dots\}$ is the set of natural numbers, and $S$ is some set (often $\mathbb{R}$). We usually denote a sequence by $(a_n)_{n\in\mathbb{N}}$ or simply $(a_n)$, where

$$a_n = a(n)$$

is the $n$-th term of the sequence. In words, a sequence is an *ordered list of elements* $(a_1, a_2, a_3, \dots)$ indexed by natural numbers. For example, the sequence of natural numbers: $(1, 2, 3, 4, \dots)$, where $a_n = n$ and the sequence of reciprocals: $\left(1, \frac{1}{2}, \frac{1}{3}, \dots\right)$, where $a_n = \frac{1}{n}$.

## 2.2 Matrices and Vectors

Scalars can be collected into rows and columns, which we call elements of vectors (single row/column sequences of scalars) or matrices (tables of vectors).

**Definition 2.6** (Vector)**.** An *n-dimensional vector* is an ordered collection of scalars of *length* $n \in \mathbb{N}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

**Worked example:**

$$\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} \in \mathbb{R}^3.$$

**Definition 2.7** (Transpose)**.** The *transpose* of a vector $\mathbf{x}$, denoted $\mathbf{x}^\top$, is a row vector formed by flipping the vector:

$$\mathbf{x}^\top = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}.$$

**Worked example:**
If

$$\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}, \quad \text{then } \mathbf{v}^\top = \begin{bmatrix} 3 & 5 & 7 \end{bmatrix}.$$

**Definition 2.8** (Dot Product)**.** The *dot product* of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is

$$\mathbf{a}^\top \mathbf{b} = \sum_{i=1}^{n} a_i b_i.$$

**Worked example:**
Let

$$\mathbf{a} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}.$$

Then

$$\mathbf{a}^\top \mathbf{b} = 2 \cdot 1 + 4 \cdot 0 + 6 \cdot 3 = 20.$$

**Definition 2.9** (Matrix)**.** A *matrix* is a two-dimensional array (a table) of scalars arranged in rows and columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

Here, $m$ is the number of rows and $n$ the number of columns.

**Worked example:**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

As we can see, matrices are simple tables of vectors that we use to organize data into.

**Definition 2.10** (Matrix Transpose)**.** The *transpose* of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, written $\mathbf{A}^\top$, is obtained by interchanging its rows and columns:

$$(\mathbf{A}^\top)_{ij} = a_{ji}, \quad \mathbf{A}^\top \in \mathbb{R}^{n \times m}.$$

**Worked example:**
If

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

then

$$\mathbf{A}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

**Definition 2.11** (Matrix–Vector and Vector–Matrix Products)**.** For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}\mathbf{x} \in \mathbb{R}^m, \quad (\mathbf{A}\mathbf{x})_i = \sum_{j=1}^{n} a_{ij} x_j.$$

**Worked example:**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}.$$

Then

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 1 + 3 \cdot 3 \\ 4 \cdot 2 + 5 \cdot 1 + 6 \cdot 3 \end{bmatrix} = \begin{bmatrix} 13 \\ 31 \end{bmatrix}.$$

For $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{n \times m}$,

$$\mathbf{x}^\top \mathbf{A} \in \mathbb{R}^{1 \times m}, \quad (\mathbf{x}^\top \mathbf{A})_j = \sum_{i=1}^{n} x_i a_{ij}.$$

**Worked example:**

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 4 & 0 \\ 2 & 5 & 1 \\ 3 & 6 & 2 \end{bmatrix}.$$

Then

$$\mathbf{x}^\top \mathbf{A} = \begin{bmatrix} 13 & 31 & 7 \end{bmatrix}.$$

A *norm* measures the *magnitude* (or *length*) of a vector or matrix. It generalizes the concept of distance from the origin.

**Definition 2.12** (Norm)**.** Formally, a norm $\|\cdot\|$ satisfies:

1. *Non-negativity:* $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0$ iff $\mathbf{x} = 0$.
2. *Homogeneity:* $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$.
3. *Triangle inequality:* $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. We will formally define this below.

Norms are used to measure *size*, *distance*, and *error*, and to impose *regularization* in optimization problems (another thing we will define below).

### 2.2.1 Vector Norms

**Definition 2.13** ($\ell_0$ "Norm")**.** The $\ell_0$ "norm" counts the number of nonzero entries in a vector:

$$\|\mathbf{x}\|_0 = \#\{i : x_i \neq 0\}.$$

It measures *sparsity* (how many components are active). It is *not* a true norm, since it violates the triangle inequality.

**Worked example:**
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_0 = 2.$$

**Definition 2.14** ($\ell_1$ Norm)**.** The $\ell_1$ *norm* (also called the *Manhattan* or *taxicab* norm) is the sum of absolute values:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|.$$

It measures *total magnitude*, encouraging *sparsity* when used in optimization.

**Worked example:**
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_1 = |3| + |-4| + |0| = 7.$$

I did not really understand norms until I came up with silly examples for them. For the $\ell_1$ *norm*, imagine walking on a trail. You take 10 steps forward and 2 steps back — even though your net position is 8 steps ahead, the total *distance walked* is $|10| + |-2| = 12$. That total distance is your $\ell_1$ norm.

**Definition 2.15** ($\ell_2$ Norm)**.** The $\ell_2$ *norm* (Euclidean norm) measures the *straight-line distance* from the origin:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}.$$

It corresponds to the geometric length of $\mathbf{x}$.

**Worked example:**
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_2 = \sqrt{3^2 + (-4)^2 + 0^2} = \sqrt{25} = 5.$$

In this case, it measures how far you are from the origin had you walked in a straight line. We can use the same trail analogy above. Suppose you take ten steps forward, and 2 back: we've traveled a total of 12 steps, but we are 8 steps from the starting point: $\|\mathbf{x}\|_2 = \sqrt{(10 + (-2))^2} = \sqrt{8^2} = 8$.

If **?@fig-pythag** look familiar to you, this is simply the Pythagorean Theorem in action! This is also why people jaywalk/cross the street at places other than cross-walks, because they will get to their destination quicker if they take a straight line instead of walking to the corner and crossing.

We also have the infinity norm:

**Definition 2.16** ($\ell_\infty$ Norm)**.** The $\ell_\infty$ *norm* (supremum or max norm) takes the largest absolute entry:

$$\|\mathbf{x}\|_\infty = \max_i |x_i|.$$

It measures the *maximum deviation* among components.

**Worked example:**
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_\infty = \max(|3|, |-4|, |0|) = 4.$$

Using the same steps $\mathbf{x} = [10, -2]$, the $\ell_\infty$ norm measures your *largest single stride.*

$$\|\mathbf{x}\|_\infty = \max(|10|, |-2|) = 10.$$

It answers the question: *"What was my biggest single effort along the path?"*

### 2.2.2 Matrix Norms

As it turns out, matrices can have norms too! The first is the Frobenius norm

**Definition 2.17** (Frobenius Norm)**.** The *Frobenius norm* measures the magnitude of a matrix by aggregating the squares of all its entries:

$$\|\mathbf{Y}\|_F = \sqrt{\sum_{i,j} y_{ij}^2}.$$

It generalizes the $\ell_2$ norm to matrices and corresponds to the Euclidean length of the vector formed by stacking all entries.

If each column of $\mathbf{Y}$ represents a separate person on a trail, each row entry is a step along that trail. The Frobenius norm sums up the squared steps of all people and takes the square root. It measures the *combined magnitude of movement* across all people.

**Worked example:**
Let

$$\mathbf{Y} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \end{bmatrix}.$$

Then

$$\|\mathbf{Y}\|_F = \sqrt{1^2 + 2^2 + 0^2 + 2^2 + 1^2 + 3^2} = \sqrt{19}.$$

**Definition 2.18** (Eigenvalues)**.** Let $A \in \mathbb{R}^{n \times n}$ be a square matrix. A *scalar* $\lambda \in \mathbb{R}$ (or $\mathbb{C}$) is called an *eigenvalue* of $A$ if there exists a nonzero vector $\mathbf{v} \in \mathbb{R}^n$ such that

$$A\mathbf{v} = \lambda\mathbf{v}.$$

The corresponding *eigenvector* $\mathbf{v}$ is the vector that is only *scaled, not rotated*, by the transformation $A$.

Equivalently, $\lambda$ is an eigenvalue if the following *characteristic equation* holds:

$$\det(A - \lambda I) = 0,$$

where $I$ is the $n \times n$ identity matrix.

Think of each column of $A$ as a separate person on a trail, and each entry as a step they take along that trail. The matrix $A$ mixes the trail walkers, so normally any direction of steps gets both *stretched and rotated*. An *eigenvector* is a special "trail direction" that only gets *stretched*, not turned, when applying $A$. The *eigenvalue* tells you *how much* that direction is stretched. Large eigenvalues correspond to the "dominant trail" that grows the fastest. Small eigenvalues correspond to directions that barely change. This is why in many systems, the largest eigenvalue often *drives the main behavior*, similar to how the biggest steps dominate the Frobenius norm in the trail analogy.

Notice the person who does not move at all — their steps are all zero. Because they have *no variance*, they are completely uninformative about the matrix. In linear algebra terms, their direction corresponds to a *zero eigenvalue*. This is why zero eigenvalues arise: they indicate directions that the matrix *completely flattens*. Practically, we do not care about these people because they contribute *nothing* to the overall movement or dominant directions of the system.

**Definition 2.19** (Matrix Operator (Spectral) Norm)**.** The *matrix operator norm*, also called the *spectral norm* or *induced 2-norm*, measures how much a matrix can *stretch* a vector when multiplied by it.

Formally, for $\mathbf{A} \in \mathbb{R}^{m\times n}$:

$$\|\mathbf{A}\|_2 = \max_{\mathbf{x}\neq 0} \frac{\|\mathbf{A}\mathbf{x}\|_2}{\|\mathbf{x}\|_2}.$$

Equivalently, it equals the *largest singular value* of $\mathbf{A}$:

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}.$$

**Intuition:**
- $\|\mathbf{A}\|_2$ captures the *maximum amplification factor* of $\mathbf{A}$.
- If you think of $\mathbf{A}$ as a transformation, it tells how much $\mathbf{A}$ can stretch any unit vector.
- It is widely used in numerical analysis, stability, and conditioning of linear systems.

**Worked example (2×2 case):**
Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}.$$

Compute $\mathbf{A}^\top \mathbf{A}$:

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}^\top \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 5 & 8 \\ 8 & 13 \end{bmatrix}.$$

The eigenvalues of $\mathbf{A}^\top \mathbf{A}$ are solutions to

$$\det(\mathbf{A}^\top \mathbf{A} - \lambda I) = 0 \quad \Rightarrow \quad \lambda^2 - 18\lambda + 1 = 0.$$

Hence

$$\lambda_{\max} = 9 + 4\sqrt{5}, \quad \|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}} = \sqrt{9 + 4\sqrt{5}} \approx 4.11.$$

So $\mathbf{A}$ stretches some unit vector by about a factor of 4.1.

In other words, imagine all possible combinations of walkers taking steps according to any unit vector $\mathbf{x}$ The operator norm tells you the combination that produces the largest overall movement. This is the dominant combined trail, the direction along which the walkers' weighted steps are amplified the most. This connects to previous concepts. Like the Frobenius norm, which sums the total effort across all walkers, the operator norm focuses on the single combination that experiences the maximum stretch. Zero eigenvalue directions correspond to walkers who do not move at all in that combination and therefore contribute nothing to the maximum stretch. The operator norm equals the largest singular value, which identifies the most dominant movement pattern among all walkers.

## 2.3 Calculus

Calculus is the study of change. It allows scientists to make precise statements about complex systems. In standard algebra we learn that the slope of a line is equal to the rise over run. **?@fig-linear-function** illustrates this idea.

This is the function $2x$, where we intuitively see that each movement along the x axis doubles whatever the input is for $x$. Suppose this is a price function where you are paying two dollars per pound of something. The rate of change for your pay is constant in the sense that you pay the same amount of money for every single pound that you buy. However, sometimes reality is a little more complex than nice linear functions. Consider the idea of kicking a football or soccerball.

Here, **?@fig-football-trajectory** shows us that the height of the ball changes at each point. More precisely, the speed at which the ball attains a given height changes at each point, as a function of time. At times point 0, the ball is with you, but by 2 seconds the ball will have attained a different height and be traveling a certain velocity at that height. Calculus allows us to study this phenomenon as well by using a derivative.

**Definition 2.20** (Derivative)**.** For a scalar function $f : \mathbb{R} \to \mathbb{R}$, the *derivative* measures the instantaneous rate of change:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

**Intuition:** How $f(x)$ changes for an infinitesimal change in $x$.

Derivatives have rules attached to them. For example, one is the power rule:

**Definition 2.21** (Power Rule)**.** If $f(x) = x^n$ for $n \in \mathbb{R}$:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} = nx^{n-1}.$$

**Worked example:** $\frac{\mathrm{d}}{\mathrm{d}x} x^3 = 3x^2$.

**?@fig-football-cvxpy** shows us the solution, but we will work it out by hand. Consider the football trajectory: $s(t) = -4t^2 + 16t + 3$. To find the time of maximum height, compute the derivative using the power rule: $\frac{\mathrm{d}s(t)}{\mathrm{d}t} = \frac{\mathrm{d}}{\mathrm{d}t}[-4t^2 + 16t + 3] = -8t + 16$. We set the derivative equal to zero to find the critical point: $-8t + 16 = 0 \Rightarrow t^* = \frac{16}{8} = 2$ seconds. Compute the maximum height by plugging $t^*$ into the original function: $s(t^*) = -4(2)^2 + 16(2) + 3 = -16 + 32 + 3 = 19$ meters. Thus, the ball reaches its *highest point at* $t = 2$ *s*, with a height of 19 meters.

**Definition 2.22** (Partial Derivative)**.** For a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$, the *partial derivative* with respect to $x_i$ is

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

**Intuition:** Measures how $f$ changes with $x_i$, holding other variables fixed.

Another important rule is the chain rule.

**Definition 2.23** (Chain Rule)**.** For $y = f(g(x))$:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}f}{\mathrm{d}g} \cdot \frac{\mathrm{d}g}{\mathrm{d}x}.$$

Consider a firm's profit given by revenue minus cost:

$$\Pi(x) = r(x) - c(x) = 75x - \Big[0.25(x+6)^2 + 191\Big].$$

The cost term can be viewed as a composition of functions. Let the outer function be

$$f(u) = 0.25u^2 + 191$$

and the inner function be

$$g(x) = x + 6,$$

so that

$$c(x) = f(g(x)).$$

Applying the chain rule, the derivative of cost with respect to $x$ is

$$\frac{\mathrm{d}c}{\mathrm{d}x} = \frac{\mathrm{d}f}{\mathrm{d}u} \cdot \frac{\mathrm{d}g}{\mathrm{d}x} = 0.5(x+6) \cdot 1 = 0.5(x+6).$$

The derivative of profit with respect to $x$ is then

$$\frac{\mathrm{d}\Pi}{\mathrm{d}x} = \frac{\mathrm{d}r}{\mathrm{d}x} - \frac{\mathrm{d}c}{\mathrm{d}x} = 75 - 0.5(x+6).$$

Setting this derivative equal to zero to find the maximum gives

$$75 - 0.5(x+6) = 0 \quad \Rightarrow \quad x^* = 144.$$

The maximum profit is

$$\Pi(144) = 75(144) - 0.25(150^2) - 191 = 10800 - 5625 - 191 = 4984.$$

The inner function is $g(x) = x + 6$, the outer function is $f(u) = 0.25u^2 + 191$, and the derivative of the cost term using the chain rule is $\frac{\mathrm{d}c}{\mathrm{d}x} = 0.5(x+6)$. The optimal quantity is

$x^* = 144$ and the maximum profit is $\Pi_{\max} = 4984$. To visualize this another way, consider the profit function as depending on revenue and costs, where costs depends on an inner function and outer function.

Realistically though, functions depend on many variables that we need to account for. To do this, we use the partial derivative.

**Definition 2.24** (Partial Derivative)**.** For a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$, the *partial derivative* with respect to $x_i$ is

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

**Intuition:** Measures how $f$ changes with $x_i$, holding other variables fixed.

Consider a small company that sells two products, Product X and Product Y. The company can spend a certain amount of resources on marketing each product, denoted by $x$ and $y$ respectively.

The expected profit (in thousands of dollars) from marketing is modeled as

$$f(x, y) = 3x + 4y - x^2 - y^2$$

where $3x+4y$ represents the linear returns from marketing, and $-x^2-y^2$ represents diminishing returns.

To maximize profit, we can use partial derivatives with respect to each marketing variable:

$$\frac{\mathrm{d}f}{\mathrm{d}x} = 3 - 2x, \quad \frac{\mathrm{d}f}{\mathrm{d}y} = 4 - 2y$$

Setting the partial derivatives equal to zero gives the optimal allocation:

$$3 - 2x = 0 \quad \Rightarrow \quad x^* = 1.5$$

$$4 - 2y = 0 \quad \Rightarrow \quad y^* = 2$$

The maximum profit is

$$f(1.5, 2) = 3 \cdot 1.5 + 4 \cdot 2 - 1.5^2 - 2^2 = 6.25$$

This can also be solved in `cvxpy`

**Definition 2.25** (Gradient)**.** The *gradient* of $f : \mathbb{R}^n \to \mathbb{R}$ is

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix}.$$

Consider the profit function of a company that markets two products, X and Y:

$$f(x, y) = 3x + 4y - x^2 - y^2$$

where $x$ and $y$ denote the marketing allocations for products X and Y. The gradient of $f(x, y)$ collects the partial derivatives with respect to $x$ and $y$:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 3 - 2x \\ 4 - 2y \end{bmatrix}.$$

At any point $(x, y)$, the gradient points in the direction of steepest increase of profit.

For the optimal allocation found earlier:

$$x^* = 1.5, \quad y^* = 2,$$

the gradient evaluates to

$$\nabla f(x^*, y^*) = \begin{bmatrix} 3 - 2(1.5) \\ 4 - 2(2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

confirming that $(x^*, y^*)$ is the critical point.

**Definition 2.26** (Gramian Matrix)**.** Given vectors $x_1, \ldots, x_n \in \mathbb{R}^m$, the Gramian matrix is the $n \times n$ matrix

$$\mathbf{G} = \begin{bmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \ldots & \mathbf{x}_1^\top \mathbf{x}_n \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \ldots & \mathbf{x}_2^\top \mathbf{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_n^\top \mathbf{x}_1 & \mathbf{x}_n^\top \mathbf{x}_2 & \ldots & \mathbf{x}_n^\top \mathbf{x}_n \end{bmatrix}.$$

Consider two vectors in $\mathbb{R}^3$:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{x}_2 = 2\mathbf{x}_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}.$$

Form the matrix

$$\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2] = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}.$$

Compute the Gramian:

$$\mathbf{G} = \mathbf{X}^\top \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}.$$

The correlation coefficient between the vectors is

$$\text{corr}(\mathbf{x}_1, \mathbf{x}_2) = \frac{\mathbf{x}_1^\top \mathbf{x}_2}{\|\mathbf{x}_1\|_2 \|\mathbf{x}_2\|_2} = \frac{28}{\sqrt{14}\sqrt{56}} = 1,$$

showing that $\mathbf{x}_1$ and $\mathbf{x}_2$ are perfectly aligned. To visualize what we mean by alignment, we can just plot these vectors:

We will oftentimes return to the Gramian in order to talk about the relationships between units (cities, states, or others for synthetic control methods)

## 2.4 Optimization

Much of this chapter has been building up to optimization. With the exception of some Bayesian SCMs, basic knowledge of optimization is crucial to being fluent in SCM research, instead of merely a user who uses preexisting code. Let's start with the idea of an objective function.

**Definition 2.27** (Objective Function)**.** An *objective function* is a scalar-valued function we aim to minimize or maximize:

$$f : \mathbb{R}^n \to \mathbb{R}.$$

Optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{or} \quad \max_{\mathbf{x}} f(\mathbf{x}).$$

An objective function may look like

$$f(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2$$

where the vector $\mathbf{y}_1$ is our target variable of interest, $\mathbf{Y}_0$ is some matrix, and $\mathbf{w}$ is what we call a decision variable, or, the vector of values that we seek to optimize the function with. In this case, we are using the Frobenius matrix to minimize the squared error between treated and the dot product of the matrix and some transformation of its values. The first step of minimizing an objective function is getting the first order conditions:

**Definition 2.28** (First-Order Condition (FOC))**.** The *first-order condition* is necessary for a local extremum of a differentiable function $f(\mathbf{x})$:

$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$

This must hold at any local minimum or maximum.

This is essentially taking the first derivative. Consider the target vector $\mathbf{y}_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$ and the matrix/vector $\mathbf{Y}_0 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, with a scalar decision variable $w$. Our objective function is the squared 2-norm of the difference:

$$f(w) = \|\mathbf{y}_1 - \mathbf{Y}_0 w\|_2^2.$$

By definition, the squared 2-norm of a vector is the sum of the squares of its entries. Therefore, if we subtract $\mathbf{Y}_0 w$ from $\mathbf{y}_1$, we get the vector

$$\mathbf{y}_1 - \mathbf{Y}_0 w = \begin{bmatrix} 2 - w \\ 4 - 2w \\ 6 - 3w \end{bmatrix},$$

and taking the squared 2-norm of this vector gives

$$f(w) = (2 - w)^2 + (4 - 2w)^2 + (6 - 3w)^2.$$

Expanding each square, we find

$$f(w) = (4 - 4w + w^2) + (16 - 16w + 4w^2) + (36 - 36w + 9w^2),$$

and combining like terms results in

$$f(w) = 14w^2 - 56w + 56.$$

From here we can complete the square, use the power rule, or use the quadratic formula; I prefer the power rule. Differentiating with respect to $w$ yields the first-order condition

$$f'(w) = 28w - 56 = 0,$$

To solve this, we add 56 to the right hand side and divide by 28.

$$w^* = 2.$$

Since the second derivative is positive, $f''(w) = 28 > 0$, we confirm that this is indeed a minimum. Now, in fairness, we could have just done this in our heads. But the reason I worked this out here is because we will be working a lot with objective functions. At least for me, I was very intimidated by them when I first saw them (and I still am all the time). So, I work through them to show you that the math behind this is not as arcane as it at first seems.

Frequently, we have optimization problems under constraints in real life. Often, there are limits to how much we can spend or how far we can go. These constraints change the feasibility set of our solutions"

**Definition 2.29** (Feasible Region)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be an objective function, and let the optimization problem be

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad \begin{cases} g_i(x) \leq 0, & i = 1, \ldots, m, \\ h_j(x) = 0, & j = 1, \ldots, p, \end{cases}$$

where $g_i(x)$ are inequality constraints and $h_j(x)$ are equality constraints.

The *feasible region* (also called the feasibility set) is defined as

$$\mathcal{W} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0,\ i = 1, \ldots, m;\ h_j(x) = 0,\ j = 1, \ldots, p\}.$$

In words, $\mathcal{W}$ is the set of all points that satisfy *all the constraints* of the optimization problem. The solution of the optimization problem must lie in $\mathcal{W}$. For example, with the problem

$$\min_{x_1, x_2} x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 + x_2 = 1,$$

the feasible region is

$$\mathcal{W} = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 + x_2 = 1\}.$$

## 2.5 Convexity

Convexity is an important idea that we will return to in later chapters. Here we define some basic terms, which we will build on later.

**Definition 2.30** (Convex Function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if for all $x, y \in \mathbb{R}^n$, $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda) f(y).$$

Differentiable functions satisfy $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$.

The intuition is that a convex function is bowl-shaped and always lies below the line connecting any two points on its graph. An example is $f(x) = x^2$ on $\mathbb{R}$, which has a minimum at zero and curves upward on both sides. Another example is $f(x) = e^x$, which grows exponentially but also satisfies the convexity property.

**Definition 2.31** (Convex Set)**.** A set $C \subset \mathbb{R}^n$ is convex if for any $x, y \in C$ and $\lambda \in [0, 1]$, the point $\lambda x + (1 - \lambda)y$ is also in $C$.

Intuitively, any line segment connecting points in the set stays entirely within the set. For example, the interval $[0, 1]$ in $\mathbb{R}$ is convex because any point between two numbers in the interval is also in the interval. Similarly, the unit disk $\{x \in \mathbb{R}^2 : \|x\| \leq 1\}$ is convex because any line segment between two points in the disk remains inside the disk.

**Definition 2.32** (Closed Set)**.** A set $C \subset \mathbb{R}^n$ is closed if it contains all its limit points. That is, if a sequence $(x_k)$ in $C$ converges to $x$, then $x$ must also be in $C$.

The intuition is that a closed set has no "missing boundary points." For example, the interval $[0, 1]$ is closed because its endpoints are included. Another example is $\{x \in \mathbb{R} : x \geq 0\}$, which contains all points that are limits of sequences within the set.

**Definition 2.33** (Bounded Set)**.** A set $C \subset \mathbb{R}^n$ is bounded if all its points lie within some finite distance from the origin. Formally, there exists $M > 0$ such that $\|x\| \leq M$ for all $x \in C$.

Intuitively, a bounded set fits inside a ball of finite radius. The interval $[0, 1]$ is bounded, as is the unit circle $\{x \in \mathbb{R}^2 : \|x\| = 1\}$.

**Definition 2.34** (Compact Set)**.** A set $C \subset \mathbb{R}^n$ is compact if it is both closed and bounded.

This means the set is finite in extent and contains all its boundary points. A compact set ensures that continuous functions defined on it achieve maximum and minimum values. Examples include the interval $[0, 1]$ and the closed unit disk $\{x \in \mathbb{R}^2 : \|x\| \leq 1\}$.

**Definition 2.35** (Convex Hull)**.** The convex hull of points $\{x_1, \ldots, x_J\}$ is

$$\text{conv}\{x_1, \ldots, x_J\} = \left\{ \sum_{j=1}^{J} w_j x_j \;\middle|\; w_j \geq 0, \sum_{j=1}^{J} w_j = 1 \right\}.$$

Intuitively, the convex hull is the smallest convex set containing all the points, formed by all weighted averages of the points. For instance, the convex hull of points $(0, 0), (1, 0), (0, 1)$ is the triangle defined by these vertices. If the points lie on a line, such as $x_1 = 0, x_2 = 2, x_3 = 5$, the convex hull is the interval $[0, 5]$.

## 2.6 Inequalities

While this book is not a real analysis book, I will occasionally present some theoretical results if they help explain why certain models or ideas behave the way they do. Typically, these are simple algebraic manipulations. In an appendix, I will go through the asymptotic bias bound

of the standard SCM much later on. To do this, we need inequalities that allow us to give formal bounds.

The first of these is the triangle inequality.

**Definition 2.36** (Triangle Inequality for Scalars)**.** For any real numbers $a$ and $b$, the triangle inequality states:

$$|a + b| \leq |a| + |b|.$$

This inequality allows us to bound the absolute value of a sum of scalars by the sum of their absolute values. To build intuition, imagine walking along a trail. Suppose you first take 3 steps backward, then 5 steps forward. Your net position relative to the start is 2 steps forward, even though you actually moved a total of 8 steps. The triangle inequality formalizes this idea: the net change cannot exceed the total distance covered.

In symbols, if $a = -3$ and $b = 5$, then

$$|a + b| = |-3 + 5| = |2| = 2, \quad |a| + |b| = 3 + 5 = 8,$$

so indeed $|a + b| \leq |a| + |b|$.

We can also apply the triangle inequality to vectors. Intuitively, the idea is the same: the length of the sum of two vectors cannot exceed the sum of their lengths.

**Definition 2.37** (Triangle Inequality for Vectors)**.** For any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the triangle inequality states:

$$\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

To build intuition, imagine walking along a trail in two dimensions. If you first walk along vector $\mathbf{x}$ and then along vector $\mathbf{y}$, your net displacement from the start is represented by $\mathbf{x} + \mathbf{y}$. The length of this net displacement (the straight-line distance from start to finish) is always less than or equal to the total distance you actually walked (the sum of the lengths of $\mathbf{x}$ and $\mathbf{y}$). For example, if $\mathbf{x} = (3, 0)$ and $\mathbf{y} = (0, 4)$, then

$$\|\mathbf{x} + \mathbf{y}\|_2 = \|(3, 4)\|_2 = 5, \quad \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2 = 3 + 4 = 7,$$

so indeed $\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2$. This inequality really helped me understand the $\ell_1$ norm.

**Definition 2.38** (Jensen's Inequality)**.** Let $X$ be a random variable and $f$ a convex function. Then:

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$

In our context, the convex function is $f(x) = |x|$, so

$$|\mathbb{E}[X]| \leq \mathbb{E}[|X|].$$

To build intuition, consider a trail example. Suppose you take two steps: $x = -2$ steps backward and $y = 4$ steps forward, with weights $\theta = 0.3$ and $1 - \theta = 0.7$. Then

$$|\theta x + (1-\theta)y| = |0.3(-2) + 0.7(4)| = 2.2,$$

while

$$\theta|x| + (1-\theta)|y| = 0.3 \cdot 2 + 0.7 \cdot 4 = 3.4.$$

Indeed, Jensen's inequality holds: $2.2 \leq 3.4$. The average magnitude is at least as big as the magnitude of the weighted average step.

Finally, we arrive at the great-granddaddy of inequalities: the Cauchy–Schwarz inequality. This inequality is fundamental in linear algebra and statistics, and it underpins many other results, including the triangle inequality for vectors.

**Definition 2.39** (Cauchy–Schwarz Inequality)**.** For any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the Cauchy–Schwarz inequality states:

$$|\mathbf{x}^\top \mathbf{y}| \leq \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2.$$

Intuitively, Cauchy–Schwarz says that the absolute value of the dot product of two vectors cannot exceed the product of their lengths. Geometrically, it captures the idea that the "projection" of one vector onto another can never be longer than the product of their magnitudes. For example, consider vectors $\mathbf{x} = (1, 2, 2)$ and $\mathbf{y} = (2, 0, 1)$. Then

$$\mathbf{x}^\top \mathbf{y} = 1 \cdot 2 + 2 \cdot 0 + 2 \cdot 1 = 4, \quad \|\mathbf{x}\|_2 = \sqrt{1^2 + 2^2 + 2^2} = 3, \quad \|\mathbf{y}\|_2 = \sqrt{2^2 + 0^2 + 1^2} = \sqrt{5}.$$

The product of the norms is $3 \cdot \sqrt{5} \approx 6.71$, and indeed

$$|\mathbf{x}^\top \mathbf{y}| = 4 \leq 6.71 = \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2.$$

This inequality is extremely useful because it allows us to bound dot products in terms of vector lengths.

We can also see how Cauchy–Schwarz leads naturally to the triangle inequality for vectors, using our trail analogy. Suppose you first walk along vector $\mathbf{x}$, then along vector $\mathbf{y}$. Your net displacement is $\mathbf{x} + \mathbf{y}$, and the triangle inequality says

$$\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

Cauchy–Schwarz helps prove this because

$$\|\mathbf{x}+\mathbf{y}\|_2^2 = (\mathbf{x}+\mathbf{y})^\top(\mathbf{x}+\mathbf{y}) = \|\mathbf{x}\|_2^2+2\mathbf{x}^\top\mathbf{y}+\|\mathbf{y}\|_2^2 \le \|\mathbf{x}\|_2^2+2\|\mathbf{x}\|_2\|\mathbf{y}\|_2+\|\mathbf{y}\|_2^2 = (\|\mathbf{x}\|_2+\|\mathbf{y}\|_2)^2,$$

where we applied Cauchy–Schwarz to bound $|\mathbf{x}^\top\mathbf{y}| \le \|\mathbf{x}\|_2\,\|\mathbf{y}\|_2$.

For a concrete numeric example, consider a 2D trail: let $\mathbf{x} = (3, 0)$ represent walking 3 steps east, and $\mathbf{y} = (0, 4)$ represent walking 4 steps north. Then your net displacement is

$$\mathbf{x} + \mathbf{y} = (3, 4), \quad \|\mathbf{x} + \mathbf{y}\|_2 = \sqrt{3^2 + 4^2} = 5.$$

The sum of the lengths of your individual steps is

$$\|\mathbf{x}\|_2 + \|\mathbf{y}\|_2 = 3 + 4 = 7.$$

Cauchy–Schwarz guarantees that your net displacement cannot exceed the total distance you walked:

$$\|\mathbf{x} + \mathbf{y}\|_2 = 5 \le 7 = \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

Thus, the triangle inequality for vectors is a direct consequence of Cauchy–Schwarz.

Minkowski's inequality generalizes the triangle inequality to sums of vectors (or sequences of vectors) and higher moments. It is particularly useful when we want to bound the "combined displacement" of multiple vectors at once.

**Definition 2.40** (Minkowski Inequality)**.** For vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^F$ and $g \ge 1$:

$$\Big(\sum_{i=1}^{n} \|\mathbf{x}_i + \mathbf{y}_i\|_2^g\Big)^{1/g} \le \Big(\sum_{i=1}^{n} \|\mathbf{x}_i\|_2^g\Big)^{1/g} + \Big(\sum_{i=1}^{n} \|\mathbf{y}_i\|_2^g\Big)^{1/g}.$$

Intuitively, imagine a group of people walking on the same trail at once. Each person takes a step represented by $\mathbf{x}_i$ and another step represented by $\mathbf{y}_i$. Minkowski's inequality says that the combined "effort" of all people moving along both sets of steps, measured in the $g$-norm, cannot exceed the sum of the efforts of each set of steps separately.

For a numeric illustration, suppose three people take steps $\mathbf{x}_1 = (1, 0)$, $\mathbf{x}_2 = (0, 2)$, $\mathbf{x}_3 = (1, 1)$, and corresponding steps $\mathbf{y}_1 = (0, 1)$, $\mathbf{y}_2 = (1, 0)$, $\mathbf{y}_3 = (1, 0)$, with $g = 2$. Then the total "combined displacement" is

$$\Big(\sum_{i=1}^{3} \|\mathbf{x}_i + \mathbf{y}_i\|_2^2\Big)^{1/2} = \Big(\|(1,1)\|_2^2 + \|(1,2)\|_2^2 + \|(2,1)\|_2^2\Big)^{1/2} = \sqrt{2+5+5} = \sqrt{12} \approx 3.46.$$

The sum of the separate "efforts" is

$$\Big(\sum_{i=1}^{3}\|\mathbf{x}_i\|_2^2\Big)^{1/2}+\Big(\sum_{i=1}^{3}\|\mathbf{y}_i\|_2^2\Big)^{1/2}=\sqrt{1+4+2}+\sqrt{1+1+1}=\sqrt{7}+\sqrt{3}\approx 4.45.$$

Indeed, the combined displacement $3.46 \le 4.45$, illustrating Minkowski's inequality. This shows how the total movement of multiple people along a trail is controlled by the sum of their individual efforts.

Finally, we generalize Cauchy–Schwarz to Hölder's inequality. While Cauchy–Schwarz handles the case of $p = q = 2$, Hölder's inequality allows us to bound sums of products for any exponents $p, q \ge 1$ satisfying $1/p + 1/q = 1$. This is very useful when dealing with weighted sums or expectations.

**Definition 2.41** (Hölder's Inequality)**.** For sequences of real numbers $(a_i)$ and $(b_i)$, and exponents $p, q \ge 1$ such that $1/p + 1/q = 1$, Hölder's inequality states:

$$\sum_i |a_i b_i| \le \left(\sum_i |a_i|^p\right)^{1/p}\left(\sum_i |b_i|^q\right)^{1/q}.$$

Intuitively, Hölder's inequality says that the "weighted overlap" between two sequences cannot exceed the product of their generalized lengths.

For a simple numeric example, consider two sequences $a = (1, 2)$ and $b = (3, 4)$ with $p = 3$ and $q = 3/2$ (so that $1/p + 1/q = 1$). Then

$$\sum_i |a_i b_i| = |1\cdot 3| + |2\cdot 4| = 3 + 8 = 11.$$

On the right-hand side, we compute the generalized $p$-norms:

$$\left(\sum_i |a_i|^p\right)^{1/p} = (1^3 + 2^3)^{1/3} = (1+8)^{1/3} = 9^{1/3} \approx 2.08,$$

$$\left(\sum_i |b_i|^q\right)^{1/q} = (3^{3/2} + 4^{3/2})^{2/3} \approx (5.196 + 8)^{2/3} = 13.196^{2/3} \approx 5.43.$$

Multiplying the norms gives $2.08 \cdot 5.43 \approx 11.3$, which indeed bounds the sum of products $11 \le 11.3$.

Rosenthal's inequality generalizes the idea of bounding the magnitude of a sum of independent random variables, similar to how the triangle, Cauchy–Schwarz, and Hölder inequalities help for deterministic sequences or vectors. In the random variable case, it controls the expected size of the sum, measured in terms of higher moments.

**Definition 2.42** (Rosenthal Inequality (Scalar Version))**.** Let $X_1, \ldots, X_n$ be independent, mean-zero random variables with finite $g$-th moments ($g \geq 2$). Then there exists a constant $C(g)$ such that:

$$\mathbb{E}\Big[\Big|\sum_{i=1}^{n} X_i\Big|^g\Big] \leq C(g) \max\left(\sum_{i=1}^{n} \mathbb{E}[|X_i|^g], \Big(\sum_{i=1}^{n} \mathbb{E}[X_i^2]\Big)^{g/2}\right).$$

This inequality bounds the $g$-th moment of a sum of independent mean-zero random variables.

Intuitively, imagine walking along a trail where each step is random — sometimes forward, sometimes backward — and the steps are independent of each other. Your net displacement after $n$ steps is random. Rosenthal's inequality tells us that the expected "size" of your total displacement (raised to the $g$-th power) can be bounded by either the sum of the $g$-th powers of your individual steps or by the $g/2$ power of the sum of the step variances, whichever is larger.

For a concrete numeric illustration, consider $n = 2$ independent random steps $X_1$ and $X_2$, where $X_1$ takes values 1 or $-1$ with equal probability, and $X_2$ takes values 2 or $-2$ with equal probability. Let $g = 4$. Then

$$\mathbb{E}[|X_1|^4] = 1, \quad \mathbb{E}[|X_2|^4] = 16, \quad \sum_i \mathbb{E}[|X_i|^4] = 17,$$

and

$$\mathbb{E}[X_1^2] = 1, \quad \mathbb{E}[X_2^2] = 4, \quad \left(\sum_i \mathbb{E}[X_i^2]\right)^{4/2} = (1+4)^2 = 25.$$

Hence, Rosenthal's inequality gives

$$\mathbb{E}[|X_1 + X_2|^4] \leq C(4) \cdot \max(17, 25) = C(4) \cdot 25,$$

showing that the fourth moment of the total displacement is controlled by either the sum of the fourth moments of the individual steps or by the squared sum of their variances. In other words, even though the steps are random and sometimes cancel each other out, Rosenthal gives a formal bound on how large the total displacement can be in expectation.

**Definition 2.43** (Vector-Valued Rosenthal Inequality)**.** Let $X_1, \ldots, X_n \in \mathbb{R}^F$ be independent, mean-zero random vectors with finite $g$-th moments ($g \geq 2$). Then there exists a constant $C(g)$ such that:

$$\mathbb{E}\Big[\Big\|\sum_{i=1}^{n} X_i\Big\|_2^g\Big] \leq C(g) \max\left(\sum_{i=1}^{n} \mathbb{E}\|X_i\|_2^g, \Big(\sum_{i=1}^{n} \mathbb{E}\|X_i\|_2^2\Big)^{g/2}\right).$$

To see this intuitively, imagine walking along a trail where each step is a random vector in 2D space, e.g. $X_1 = (1, 0)$, $X_2 = (0, 2)$. The sum $X_1 + X_2 = (1, 2)$ represents your net displacement. Rosenthal's inequality bounds the expected size of the net displacement in the $g$-th moment norm:

$$\mathbb{E}\|X_1 + X_2\|_2^g \leq C(g) \max\left(\mathbb{E}\|X_1\|_2^g + \mathbb{E}\|X_2\|_2^g, (\mathbb{E}\|X_1\|_2^2 + \mathbb{E}\|X_2\|_2^2)^{g/2}\right).$$

Even if the steps sometimes cancel out in some directions, this inequality ensures the expected size cannot blow up arbitrarily.

## 2.7 Conclusion

These definitions and worked examples form the mathematical toolbox for synthetic control. Analysts can now reason rigorously about vectors, matrices, derivatives, convexity, and optimization, while connecting directly to SCM applications. I have simply defined these for use later on, so we will become more acquainted with these over time. A brief word of warning—and encouragement—is in order. While this book aims to make SCM accessible to a wide audience, mathematics will be, in a sense, non-negotiable.

One of my favorite econometricians once asked hypothetically in his lecture notes, "Do I really need the fancy mathematics to understand econometrics?" (Shi, n.d.). The answer, as he put it, is both yes and no: not all practitioners need measure theory to apply econometrics effectively. By extension, not all practitioners who use causal inference and SCM need measure theory or post-graduate understanding of optimization and real analysis. In fact, my first degrees were in political science where I had no exposure to linear algebra, calculus, set theory, or discrete mathematics. However, without some degree of mathematical precision, core ideas like weighted averages, convex hulls, interpolation and extrapolation biases, and other important concepts cannot be fully defined. In the same spirit, this text strives for balance. I will avoid unnecessary abstraction, but I will also insist on clarity and correctness where it matters most. Only by taking advantage of the language of probability and statistics can the full value of the SCM be made known.

# Chapter 3

# Averages

> "I'm too profound to go Back and forth, with no average dork"
> — *Solana Imani Rowe*

An average is one of the first summary statistics we learn—often in elementary or middle school, depending on where we grew up. Averages are everywhere in data analysis. We regularly compute metrics like average revenue, averages of income, average of prices, and for many other metrics. Yet, few people pause to consider what an average actually *is* as a concept or why we use them. In this chapter, we cover this in more detail, from simple arithmetic means to general weighted averages—as a foundation for understanding how computers construct synthetic controls in practice. There is an entire chapter for averaging because this is all a synthetic control is. People will say "fake unit" or "weighted mix" without being precise about what they mean, when at simplest we are just computing an average.

## 3.1 Averaging in Statistics

Averages arise naturally when we try to find a single value that best represents a collection of numbers. In the old days, Egyptians and Babylonians used averages to summarize construction and financial matters. In modern times, we use averages to summarize things like test grades, incomes, and other variables we care about. To formalize this, suppose we have a sequence of data points $(x_1, x_2, \ldots, x_n)$ of length $n$. We want to choose a single number $\mu$ that is as close as possible to all of them. Here, $\mu$ is a placeholder variable representing a potential candidate for the 'best' summary value; we will later find that the minimizing value, denoted $\bar{x}$, is the arithmetic mean. Because we seek a single number as close as possible to all values, this becomes our first example of an optimization problem—minimizing a function with respect to a variable. Recalling the definition of an *objective function*, let our observed values be $x_i$ and the number that "is as close as possible to all the values" be the Greek letter $\mu$ (m-yoo). Because we are looking for the value that minimizes the distance, we can write $x_i - \mu$ as a shorthand for this deviation. To minimize the total deviation across all points, we sum over each element of the sequence $i$: $\sum_{i=1}^{n}(x_i - \mu)$. Finally, to penalize values of $\mu$ that are further away from the observed data, we square each term. This gives us the optimization problem:

**Definition 3.1. Objective function**

A *scalar-valued* function of a choice variable that we minimize (or maximize). We write the choice variable under the operator: $\min_\mu f(\mu)$ reads "search over $\mu$ for the value that makes $f$ smallest."

$$\min_\mu \sum_{i=1}^{n} (x_i - \mu)^2.$$

This function is *convex* and differentiable. We can optimize this function by taking the derivative with respect to $\mu$. We begin by applying the **chain rule** term by term.

**Definition 3.2. The power and chain rules**

The **power rule** differentiates a power: $\frac{\mathrm{d}}{\mathrm{d}u} u^k = k\, u^{k-1}$. The **chain rule** differentiates a function of a function: if $g$ depends on $u$ and $u$ depends on $\mu$, then $\frac{\mathrm{d}g}{\mathrm{d}\mu} = \frac{\mathrm{d}g}{\mathrm{d}u} \cdot \frac{\mathrm{d}u}{\mathrm{d}\mu}$ — outer derivative times inner.

For each term $(x_i - \mu)^2$, there are two parts to consider: the outer function and the inner function. The outer function is $f(u) = u^2$, where $u = x_i - \mu$. Its derivative (by the **power rule**) is

$$\frac{\mathrm{d}}{\mathrm{d}u} u^2 = 2u = 2(x_i - \mu)$$

The inner function is $u(\mu) = x_i - \mu$. Its derivative with respect to $\mu$ is

$$\frac{\mathrm{d}}{\mathrm{d}\mu}(x_i - \mu) = -1$$

Multiplying the outer and inner derivatives gives the derivative of a single term:

$$\frac{\mathrm{d}}{\mathrm{d}\mu}(x_i - \mu)^2 = 2(x_i - \mu) \cdot (-1) = -2(x_i - \mu)$$

Summing over all $i$ gives the derivative of the entire objective function:

$$\frac{\mathrm{d}}{\mathrm{d}\mu} \sum_{i=1}^{n} (x_i - \mu)^2 = \sum_{i=1}^{n} [-2(x_i - \mu)] = -2 \sum_{i=1}^{n} (x_i - \mu)$$

Setting this derivative equal to zero gives the first-order condition for the minimum

$$-2 \sum_{i=1}^{n} (x_i - \mu) = 0.$$

We now solve for $\mu$. First, just divide both sides by $-2$:

$$\sum_{i=1}^{n}(x_i - \mu) = 0.$$

Next, split the sum into two parts:

$$\sum_{i=1}^{n} x_i - \sum_{i=1}^{n} \mu = 0.$$

Since $\mu$ is constant with respect to the index $i$, the second sum simplifies to multiplication:

$$\sum_{i=1}^{n} x_i - n\mu = 0.$$

Now we add $n\mu$ to both sides:

$$\sum_{i=1}^{n} x_i = n\mu.$$

Finally, divide both sides by $n$ to isolate $\mu$:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

**Definition 3.3. Arithmetic average**

For some $(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$, the arithmetic average is $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$, where $n$ is the length of the sequence and $\frac{1}{n}$ is the weight.

Thus, the value of $\mu$ that minimizes the total squared deviation is the arithmetic mean of the data. The arithmetic mean is thus not an arbitrary formula: it is the unique value that minimizes the total squared distance between itself and the data.

But, recall the definition of an objective function: I mentioned how it is a **scalar valued** function, because it returns a scalar. That scalar is the value of the function where we set $\mu$ equal to the optimal value. Substituting this minimizing value back into the objective function, we get the total deviation around the mean:

$$f(\bar{x}) = \sum_{i=1}^{n}(x_i - \bar{x})^2.$$

Because this quantity grows with $n$, it is often more useful to consider the average squared deviation (also called the residuals):

$$\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2,$$

which we call the population variance. The variance measures how far the data tend to fall from their best-fitting constant value. We use squared deviations because they produce a smooth, differentiable, and convex objective that penalizes larger errors more heavily. Note that the absolute deviations, instead of the squared deviations, would return the median, not the mean. The cost of squaring the residuals is that variance is expressed in squared units; taking the square root restores the original units and gives the standard deviation. When variance is estimated from a sample rather than the entire population, the denominator is typically replaced by $n - 1$ to obtain an unbiased estimator.

### 3.1.1 So What?

Now, you may be saying "Hey Jared, this is overkill. Why bother doing all of this for something as simple as an average?" The derivation shows us that the average is the solution to an optimization problem. That single insight connects simple statistics to the machinery of modern econometrics. Variance, standard deviation, mean squared error, regression coefficients, and the synthetic control weights are all built on the same principle: finding a number/set of numbers that minimizes some measure of error. The arithmetic mean is the simplest, most familiar instance of this principle. Every estimator in this book is an application of averaging, and this fact will become crucial in later chapters when we take more sophisticated averages.

## 3.2 Kinds of Arithmetic Averages

Okay, so we now have our definition. Now let's apply this to the other objects we already know about, scalars, vectors, and matrices.

### 3.2.1 Scalars

For the simplest case, we can simply apply the arithmetic mean to scalars. Suppose you have 12 apples and your friend has 18. Denote this sequence as $A = (12, 18) \in \mathbb{N}^2$. There are two elements in this sequence. Because the length of the sequence is 2, our weight is $\frac{1}{2}$. The way to compute the arithmetic average is

$$\bar{x} = \frac{1}{2}(12 + 18) = (0.5 \times 12) + (0.5 \times 18) = 6 + 9 = 15.$$

On average, you and your friend each have 15 apples. We can do this in `NumPy`

```
import numpy as np

A = np.array([12, 18])
```

```
mean_1 = np.mean(A)

weights = np.full(len(A), len(A)**-1)
mean_2 = np.average(A, weights=weights)

print(f"Mean of Apples: {mean_1}, {mean_2}")
```

See? Nothing special happened here, we just took the empirical mean of this sequence. Just to show that the above calculus derivation was not voodoo or an optical illusion, we can even just plot the objective function itself to see the scalar outputs at different values:

```
import numpy as np
import matplotlib.pyplot as plt

# Data points

A = np.array([12, 18])

# Objective function: sum of squared deviations

mu_values = np.linspace(10, 20, 400)
f_mu = np.sum((A[:, None] - mu_values)**2, axis=0)

# Plot

plt.figure(figsize=(6,4))
plt.plot(mu_values, f_mu, label=r'$f(\mu) = \sum (x_i - \mu)^2$')
plt.axvline(np.mean(A), color='red', linestyle='--', label=r'$\bar{x} = 15$')
plt.scatter(np.mean(A), np.sum((A - np.mean(A))**2), color='red')
plt.xlabel(r'$\mu$')
plt.ylabel(r'$f(\mu)$')
plt.title('Objective Function for Arithmetic Mean')
plt.legend()
plt.grid(True)
plt.show()
```

Figure 3.1

The next context in which we often encounter averages is when estimating treatment effects. Suppose we are interested in the Average Treatment effect on the Treated (ATT) for a single treated unit ($j = 1$) observed over two post-treatment periods ($|\mathcal{T}_2| = 2$). Let the observed (treated) outcome be denoted by $y_{1t}(1)$ and the predicted or counterfactual outcome by $y_{1t}(0)$. The treatment effect in each period is then $\tau_t = y_{1t}(1) - y_{1t}(0)$. Suppose the observed sequence is $y_1(1) = (533, 568)$ and the counterfactual sequence is $y_{1t}(0) = (400, 446)$. The resulting difference of these (the treatment effect sequence) is $\tau = y_1(1) - y_{1t}(0) = (133, 122)$.

To compute the average treatment effect on the treated across these two post-treatment periods (formally defined later in the book), we apply the same arithmetic mean formula as before to this sequence of treatment effects. This looks like:

$$\bar{\tau}_{\text{ATT}} = \frac{1}{2}\sum_{t=1}^{2} \tau_t = \frac{1}{2}(133 + 122) = \tag{3.1}$$

$$(0.5 \times 133) + (0.5 \times 122) = 66.5 + 61 = 127.5. \tag{3.2}$$

Thus, on average, the treated unit's observed outcomes are about 127.5 units higher than what the counterfactual model predicted. Another thing we can do in Python:

```python
import numpy as np

# Observed outcomes
y1_treated = np.array([533, 568])

# Counterfactual outcomes
y1_control = np.array([400, 446])

# Treatment effect sequence
tau = y1_treated - y1_control

# Equal weights
weights = np.full(len(tau), len(tau)**-1)

# Average Treatment Effect on the Treated (ATT)
ATT = np.average(tau, weights=weights)

print(f"Treatment effects by period: {tau}")
print(f"Average Treatment Effect on the Treated (ATT): {ATT}")
```

Because each value in the sequence contributes equally to the final result, the arithmetic mean can be seen as a weighted sum where each weight for the $i$-th element is $\frac{1}{n}$. In general, for a sequence $(x_1, x_2, \ldots, x_n)$, the arithmetic mean can be written as

$$\bar{x} = \sum_{i=1}^{n} w_i x_i, \quad w_i = \frac{1}{n} \, \forall \, i$$

**Problem 3.1. Mean of Small Numbers**

Compute the arithmetic mean of the numbers: 4, 8, 12.

**Problem 3.2. Mean of Single-Digit Values**

Find the arithmetic mean of the sequence: 1, 3, 5, 7, 9.

**Problem 3.3. Mean of Exam Scores**

A student scored 70, 75, and 80 on three tests. Compute the arithmetic mean score.

**Problem 3.4. Mean of Monthly Sales**

A small shop made the following sales in dollars over 4 months: 200, 250, 300, 350. Find the average monthly sales.

**Problem 3.5. Mean of Age Values**

The ages of five friends are 20, 22, 24, 26, and 28. Compute the arithmetic mean age.

### 3.2.2 Vectors

The same idea generalizes easily to vectors. In higher dimensions, averaging is simply applying the same principle along rows or columns. We can represent the simple apple example as a row vector:

$$\mathbf{x} = \begin{bmatrix} 12 & 18 \end{bmatrix} \in \mathbb{R}^{1\times 2}.$$

This is the same example as before, just expressed in vector form. Here, each column corresponds to a person's value.

**Definition 3.4. Dot product**

For a row vector $\mathbf{a}$ and a column vector $\mathbf{b}$ of the same length, the *dot product* multiplies them entry by entry and sums the results, $\mathbf{ab} = \sum_i a_i b_i$. It turns "multiply each value by a weight and add them up" — an average — into a single product.

Using the *dot product* and the idea of the average as a weighted sum, we introduce a weight vector that assigns proportional and equal weight to each person:

$$\mathbf{w} = \tfrac{1}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \tfrac{1}{2}\mathbf{1}_2 \in \mathbb{R}^{2\times 1}.$$

Why proportional and equal? Because that is simply the definition of the arithmetic average. The same way we used $\frac{1}{2}$ for the scalar example because of the fact that there were two units, we use the same principle for the vector setting, where our units are columns. Here, $\mathbf{1}_2$ is a column vector of ones:

$$\mathbf{1}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Multiplying by $\frac{1}{2}$ ensures that each entry has the weight $\frac{1}{n}$, just like in the scalar arithmetic mean. The average is then given by the matrix product between the data vector and the weight vector:

$$\mathbf{xw} = \begin{bmatrix} 12 & 18 \end{bmatrix} \left( \frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \frac{1}{2}(12 + 18) = 15.$$

We get a scalar, in this case 15. We have two people (one in each column) in the vector, and we multiply every value in that column by the weight vector value, add these up, and this is the mean. The way we compute this in Python is:

```
import numpy as np

# --- Example 1: Apples ---
A = np.array([12, 18])
weights_apples = np.full(len(A), len(A)**-1)

# Dot Product
mean_apples = weights_apples @ A

print(f"Mean of Apples (dot product): {mean_apples}")
```

We can represent the same treatment effect example as a row vector:

$$\boldsymbol{\tau} = \begin{bmatrix} 133 & 122 \end{bmatrix} \in \mathbb{R}^{1\times 2}.$$

Here, each column corresponds to a time period, and the value represents the treatment effect for that period. To compute the average, we use the same equal-weight vector as before, $\frac{1}{2}\mathbf{1}_2$. Again, why? We have two post-treatment periods, and per the definition of the arithmetic mean, we need the weight to be equal to the number of elements that we have. So when we do this, we get:

$$\boldsymbol{\tau}^\top \left( \tfrac{1}{2}\mathbf{1}_2 \right) = \begin{bmatrix} 133 & 122 \end{bmatrix} \cdot \left( \frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 133 \times 0.5 & 122 \times 0.5 \end{bmatrix} = \begin{bmatrix} 66.5 & 61 \end{bmatrix}.$$

Summing the entries gives:

$$\mathbf{1}_2^\top \begin{bmatrix} 66.5 \\ 61 \end{bmatrix} = 66.5 + 61 = 127.5.$$

The average treatment effect across the two periods is 127.5, the same number as before. Thus, arithmetic averaging is simply taking a weighted sum where each weight is the reciprocal of the number of entries.

This can also be done with both examples using Python:

```python
import numpy as np

# --- Example 2: ATT ---
y1_treated = np.array([533, 568])
y1_control = np.array([400, 446])
tau = y1_treated - y1_control

weights_tau = np.full(len(tau), len(tau)**-1)
ATT = weights_tau @ tau  # same as np.dot(weights_tau, tau) or np.mean(y1_treated - y1_c

print(f"Treatment effects by period: {tau}")
print(f"Average Treatment Effect on the Treated (dot product): {ATT}")
```

Of course the above example is pedagogical. The real way we'd do this in code/a project is:

```python
import numpy as np

y1_treated = np.array([533, 568])
y1_control = np.array([400, 446])

print(np.mean(y1_treated - y1_control))
```

**Problem 3.6. Vector Mean of Small Numbers**

Represent the sequence $[3, 6, 9]$ as a vector and compute the arithmetic mean using equal weights.

**Problem 3.7. Vector Mean of Test Scores**

A student scored $[65, 70, 75, 80]$ on four tests. Represent these as a vector and compute the arithmetic mean using equal weights.

**Problem 3.8. Vector Mean of Daily Steps**

Your daily steps for a week are $[5000, 6000, 5500, 7000, 6500, 6000, 5800]$. Represent this as a vector and compute the arithmetic mean with equal weights.

**Problem 3.9. Vector Mean of Monthly Sales**

A store's sales for three months are $[1200, 1500, 1300]$. Represent this as a vector and compute the average using equal weights.

**Problem 3.10. Vector Mean of Heights**

The heights (in cm) of four people are $[160, 165, 170, 175]$. Represent these as a vector and compute the arithmetic mean using equal weights.

### 3.2.3 Matrices

Let's extend the idea to multiple time periods. Suppose today you have 12 apples and tomorrow 30, while your friend has 18 today and 22 tomorrow. We can represent this as a matrix:

$$\mathbf{Y} = \begin{bmatrix} 12 & 18 \\ 30 & 22 \end{bmatrix} \in \mathbb{R}^{2\times 2}.$$

Each row corresponds to a time period, and each column corresponds to a person.

**Definition 3.5. Matrix–vector product**

A *matrix* is a rectangular grid of numbers; here rows are time periods, columns are units. The *matrix–vector product* $\mathbf{Yw}$ takes the dot product of each row of $\mathbf{Y}$ with $\mathbf{w}$, one number per row — so weighting the columns by $\mathbf{w}$ averages across units, one time period at a time.

To find the average number of apples per time period, we multiply $\mathbf{Y}$ by the weight vector

$$\mathbf{w} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

The result is a column vector, where each entry is the average across people for that time period:

$$\bar{\mathbf{x}} = \mathbf{Yw} = \begin{bmatrix} 12 & 18 \\ 30 & 22 \end{bmatrix} \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} = \begin{bmatrix} 15 \\ 26 \end{bmatrix}.$$

Thus, the first entry 15 is the average for today, and the second entry 26 is the average for tomorrow. In Python, we'd do:

```
import numpy as np

# --- Apples Example ---
Y = np.array([[12, 18],
              [30, 22]])

w_units = np.full((Y.shape[1], 1), Y.shape[1]**-1)  # column weights (for people)
avg_by_time = Y @ w_units       # average across people for each time period

print("Average apples per time period:")
print(avg_by_time)
```

Suppose we have two treated units over two post-treatment periods. Let the predicted outcomes and observed outcomes be:

$$\hat{\mathbf{Y}} = \begin{bmatrix} 400 & 380 \\ 446 & 420 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 533 & 490 \\ 568 & 510 \end{bmatrix}.$$

Here, each row corresponds to a post-treatment period, and each column corresponds to a treated unit. The treatment effects are the differences between the observed outcomes and their predicted outcomes:

$$\boldsymbol{\tau} = \mathbf{Y} - \hat{\mathbf{Y}}.$$

We typically are concerned with the average treatment effect on the treated units, where we average first across units (columns) and then across time periods. To do this, we have weights for each unit (that is, the number of columns) **and** for each particular time point,

$$\mathbf{w}_{\text{units}} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \quad \mathbf{w}_{\text{time}} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

This is simply a generalization of th rules we've been developing so far, where units get weight when we average across them, and extending that to the setting where time periods also get weight defined by their length. So, unlike where we have one treated unit and one predicted series, this generalizes very nicely to the setting where we have multiple treated units too:

$$\text{ATT} = \mathbf{w}_{\text{time}}^{\top} \boldsymbol{\tau}\, \mathbf{w}_{\text{units}} = \mathbf{w}_{\text{time}}^{\top} (\mathbf{Y} - \hat{\mathbf{Y}}) \mathbf{w}_{\text{units}}.$$

We start with

$$\text{ATT} = \begin{bmatrix} 1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 133 & 110 \\ 122 & 90 \end{bmatrix} \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

The first step is to multiply the weight vector on the left (the time vector) by the matrix of treatment effects:

$$\begin{bmatrix} 1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 133 & 110 \\ 122 & 90 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \cdot 133 + \frac{1}{2} \cdot 122 & \frac{1}{2} \cdot 110 + \frac{1}{2} \cdot 90 \end{bmatrix} = \begin{bmatrix} 127.5 & 100 \end{bmatrix}.$$

Okay now this one is easy. Now we just multiply the resulting row vector by the column weight vector, or the average of the ATTs between each unit:

$$\begin{bmatrix} 127.5 & 100 \end{bmatrix} \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} = 127.5 \cdot 1/2 + 100 \cdot 1/2 = 63.75 + 50 = 113.75.$$

Again in `NumPy`, we'd do

Averaging is just a form of a dot product that we take across sequences/vectors and matrices. In regression and synthetic control methods, this same idea extends from scalars to matrices. Suppose $\mathbf{y}_1 \in \mathbb{R}^T$ is a target time series and $\mathbf{Y}_0 \in \mathbb{R}^{T \times J}$ is a matrix whose columns contain $J$ potential control series. The goal is to approximate $\mathbf{y}_1$ using a weighted combination of the columns of $\mathbf{Y}_0$. We can express this as minimizing the squared error

$$f(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 = \sum_{t=1}^{T} \left( y_{1t} - \sum_{j=1}^{J} w_j Y_{0,tj} \right)^2 .$$

The solution $\mathbf{w}$ gives the weights that produce the best-fitting linear combination of vectors in the least-squares sense. The mean, variance, and mean squared error are therefore all instances of a common idea: each is defined by minimizing the total squared deviation from a reference point. The mean is the minimizer, the variance is the minimized value of the objective, and the mean squared error is the same idea applied to predictions or reconstructions rather than to raw data. Understanding how averaging arises from this optimization perspective provides a unified way to interpret many methods in statistics, including regression and synthetic control, as procedures that compute structured weighted averages.

**Problem 3.11. Matrix Representation of Student Scores**

Two students take two exams. Their scores are:

| | Akiko | Tsumugi |
|---|---|---|
| Exam 1 | 80 | 70 |
| Exam 2 | 90 | 85 |

Represent this as a matrix and compute the average score per exam. Then, compute the average score per student instead.

**Problem 3.12. Matrix of Daily Temperatures**

The temperatures (°C) in two cities over three days are:

| | City A | City B |
|---|---|---|
| Day 1 | 20 | 25 |
| Day 2 | 22 | 28 |
| Day 3 | 21 | 26 |

Represent this as a matrix and compute: 1. The average temperature per day. 2. The average temperature per city.

**Problem 3.13. Matrix of Weekly Sales**

A store tracks sales for two products over four weeks:

| | Product X | Product Y |
|---|---|---|
| Week 1 | 100 | 120 |
| Week 2 | 110 | 130 |
| Week 3 | 90 | 140 |
| Week 4 | 105 | 125 |

Represent this as a matrix. Compute: 1. Average sales per week. 2. Average sales per product.

**Problem 3.14. Matrix of Monthly Rainfall**

Monthly rainfall (mm) for two cities over three months:

| | City X | City Y |
|---|---|---|
| Jan | 50 | 60 |
| Feb | 45 | 55 |
| Mar | 55 | 65 |

Represent this as a matrix. Compute: 1. Average rainfall per month. 2. Average rainfall per city.

**Problem 3.15. Matrix of Machine Outputs**

Two machines produce units per day for three days:

| | Machine 1 | Machine 2 |
|---|---|---|
| Day 1 | 200 | 220 |
| Day 2 | 210 | 230 |
| Day 3 | 205 | 225 |

Represent this as a matrix. Compute: 1. Average output per day. 2. Average output per machine.

## 3.3 What Are We Weighting For? Introducing Weighted Averages

Averages allow us compare units (whether people, firms, or countries) across time or groups. This is the reason economists use them so much. In the treatment effect example we just did, we can compare the average treatment effect for one unit to that of another unit. However so far there is a detail I have omitted: we have so far taken it as an article of faith that the weight vector should be $\frac{1}{n}$. But why should this be? In other words, does it have to always be true that $n$ needs to be the length of the columns or rows? Well, no! As it turns out, the weights can even vary across units and time periods! However, the moment we leave the special case where the weights are the reciprocal of the size of the entries, now the choice of weights matters. For comparison purposes, not all units should get the same weight.

Consider a simple example: you earn \$60k/year. Your friend earns \$70k/year. The arithmetic mean is:

$$\mathbf{x} = \begin{bmatrix} 60000 & 70000 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \end{bmatrix},$$

which evaluates to:

$$\bar{x} = \mathbf{xw} = 0.5 \cdot 60000 + 0.5 \cdot 70000 = 65000 \text{ / year.}$$

Now suppose Mansa Musa joins, earning \$107 billion/year. Mansa Musa's income must be treated equally to you two. So, now the average becomes:

$$\mathbf{x} = \begin{bmatrix} 60000 & 70000 & 1.07 \times 10^{11} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix},$$

$$\bar{x} = 0.33 \cdot 60000 + 0.33 \cdot 70000 + 0.33 \cdot 1.07 \times 10^{11} \approx 35.7 \text{ B / year.}$$

In Python, the way we'd compute this is:

```
import numpy as np

# --- Example 1: Two friends ---
x1 = np.array([60000, 70000])
w1 = np.full((len(x1), 1), 1 / len(x1))

mean_equal = x1 @ w1

print(f"Average income: ${mean_equal.item():,.0f}\n")
```

```
# --- Example 2: Adding Mansa Musa ---
x2 = np.array([60000, 70000, 1.07e11])
w2 = np.full((len(x2), 1), 1 / len(x2))

mean_equal_big = x2 @ w2

print(f"Average income: ${mean_equal_big.item():,.2f}")
```

Clearly, the average is dominated by this extreme outlier, giving a value that is no longer representative of the original group. If the bartender next day bragged to his investors about earning 50k that night and wanted to expand his business, they would say something like: "It is not that you earned 50k organically, a mega-wealthy king was in town and happened to be there that night, that is not a typical night for you. If we looked at your pre-Mansa period, this is almost certainly an outlier relative to your own previous growth as well as the growth of you local competitors."

They would say what we know by simple intuition: on nights where wealthy people are patrons, earnings will very likely be anomalous with respect to other normal nights and cannot be used for decision-making. In other words, if we want a meaningful "bar average," Mansa as an individual should likely get much less weight (if any) than the other patrons. The arithmetic mean assumes equal and proportional contribution from each unit, which may not be appropriate for meaningful comparisons.

This leads us to weighted averages. Recall the idea of the weight that I have been referring to, $\frac{1}{n}$. This is essentially the fraction that we scale the original data by for the weighted sum. In this case, the weight is proportional to the size of the number of units we have at hand. Of course, you may now be asking "Well, are other kinds of averages possible? Are we just left with the arithmetic mean?" As it turns out, yes and no respectively. The weight is a mere convention of arithmetic means; there is no law of nature or script from the Heavens that says the weight for each element must be $1/n$ Practically, we can assign weights however we like. So, instead of treating Mansa as equal to you two, or treating the bar that gets Mansa similar to other bars, we can give each person/bar a weight $w_i \in [0, 1]$.

**Definition 3.1** (Weighted Average)**.** For a sequence of numbers $(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ and a vector of weights $\mathbf{w} = (w_1, w_2, \ldots, w_n) \in \mathbb{R}^n$, the weighted average is defined as:

$$\mathbf{x}^\top \mathbf{w} = \sum_{i=1}^{n} w_i x_i$$

The classical arithmetic mean is a special case where all weights are equal to $1/n$.

When we think about the weights $\frac{1}{w_i}$, it is best to think about it as a decision variable that decides how much an individual/unit counts in the average. Now reconsider the average with this in mind: with a weighted average, Mansa can get a tiny weight (if any), and you and your friend get larger weights, giving a more reasonable "average bar income." If we assign Mansa weight 0 and you and your friend weight 0.5 each, we obtain:

$$\mathbf{w} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0 \end{bmatrix}, \quad \bar{x} = \mathbf{x}\mathbf{w} = 0.5 \cdot 60000 + 0.5 \cdot 70000 = 65000 \,/\text{year}.$$

Now the average reflects the typical case, making it a useful comparison for "regular" units. In Python, the way we'd do this is:

```
import numpy as np

# Incomes: You, Friend, Mansa Musa
x = np.array([60000, 70000, 1.07e11])

# Assign weights
w = np.array([0.5, 0.5, 0])

# Compute weighted average
weighted_mean = x @ w

print(f"Weighted average: ${weighted_mean:,.0f}.")
```

Now suppose the weight vector is:

$$\mathbf{w} = \begin{bmatrix} 0.15 \\ 0.85 \\ 0 \end{bmatrix}.$$

The weighted average is computed as:

$$\bar{x}_{\text{weighted}} = 0.15 \cdot 60000 + 0.85 \cdot 70000 + 0 \cdot 1.07 \times 10^{11} = 68500$$

So the weighted average income is \$68,500/year. We can see that this is closer to your friend's income compared to the case where you both shared equal weight, and certainly very different from the setting where Mansa counts at all.

```
import numpy as np

x = np.array([60000, 70000, 1.07e11])
w = np.array([0.15, 0.85, 0]).reshape(-1, 1)

weighted_mean = x @ w
print(f"Weighted average income: ${weighted_mean.item():,.0f}")
```

Now, consider a panel data version of the above bar example, where now we are interested in comparing one bar to ten other bars:

Here, we can see the the average where the Mansa bar is included pulls the average upwards to roughly 20,000 at the end of the time series. In applied work, group level averages appear all the time, where we compare a single set of units to the mean of a larger group. This idea is also frequent in recent news articles where comparisons are made. Examples like crime rates, healthcare spending, and gasoline prices compare a unit to a larger mean to make inferences from them.

**Problem 3.16. Impact of an Outlier on the Mean**

One of your friends suddenly receives a lottery prize of $5 million. Compute the new arithmetic mean. Comment on how representative this new mean is of the original group.

**Problem 3.17. Weighted Averages**

Suppose you want some of the first three students to have more influence than the last one. Assign an arbitrary set of weights and compute the weighted average. Explain the difference with the simple average.

**Problem 3.18. Weighted Averages, Applied**

From this problem, assign Akiko weight 0.6 and Tsumugi weight 0.4 for the average score for each exam. Comment on how the result changes.

**Problem 3.19. Limitations of the Arithmetic Mean**

Explain why simply taking the arithmetic mean can be misleading. Give a real-life example.

## 3.4 Conclusion

In this chapter, we have explored averages in their simplest forms—scalars, vectors, and matrices—and introduced the notion of weighted averages. The key takeaway is that averages are tools for comparison, but their usefulness depends entirely on the weights we assign. In real-world applications, we rarely know the "correct" weights in advance.

Sometimes, people have told me in the context of my blog have suggested that much of this content, especially around synthetic controls, would be easier to digest if all technical language were removed. While this might be true for a general audience, it misunderstands who my blog/this book is written for. The primary audience is not business stakeholders, but 1) the employees and practitioners who use data to generate empirical insights for businesses and 2) graduate students who wish to have a good understanding of SCM.

With this in mind, I want to underscore how crucial the idea of averaging is for a data practitioner who seeks to use SCM. At some point, correct explanations cannot be fully conveyed without mathematical formality. Averages are so central to the next chapters that a thorough understanding is essential to grasping and applying the ideas presented. Soon, we

will explore how to generate weights in order to construct meaningful comparisons. This, however, is impossible without a firm grasp of averages. Understanding them is not just an academic exercise: averaging forms the foundation for most comparisons in econometrics, and SCM is no exception.

**Problem 3.20. Mean as Minimizer of Squared Error**

Consider the numbers $x = [4, 7, 10, 6]$.

1. Compute the arithmetic mean $\bar{x}$.

2. Verify that $\bar{x}$ minimizes the sum of squared deviations by computing $\sum_{i=1}^{4}(x_i - \mu)^2$ for $\mu = 5, 6.75, 8$.

**Problem 3.21. Variance as Average Squared Deviation**

Using the same numbers $x = [4, 7, 10, 6]$:

1. Compute the total squared deviation from the mean.

2. Compute the variance (average squared deviation).

3. Interpret the variance: does it indicate the data are spread out or tightly clustered around the mean?

**Problem 3.22. Mean Squared Error for Predictions**

A model predicts values $[12, 15, 14, 16]$ while the observed values are $[13, 14, 16, 15]$.

1. Compute the error for each observation.

2. Compute the squared error for each observation.

3. Compute the mean squared error (MSE).

4. Interpret the MSE in terms of prediction accuracy.

**Problem 3.23. Weighted Average in Vector Form**

Suppose we have a vector of observations $\mathbf{y} = [3, 5, 8]$ and a uniform weight vector $\mathbf{w} = [1/3, 1/3, 1/3]^\top$.

1. Compute the weighted average using the dot product $\mathbf{y} \cdot \mathbf{w}$.

2. Compute the total squared deviation of $\mathbf{y}$ from this weighted average.

**Problem 3.24. Vectorized MSE in Regression**

Let $\mathbf{y}_1 = [5, 7, 6]$ be observed outcomes, and let $\mathbf{Y}_0 = \begin{bmatrix} 4 & 6 \\ 6 & 8 \\ 5 & 7 \end{bmatrix}$ be two predictor vectors.

If the weight vector is $\mathbf{w} = [0.4, 0.6]^\top$:

1. Compute the predicted values $\mathbf{Y}_0\mathbf{w}$.
2. Compute the residual vector $\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}$.
3. Compute the Mean Squared Error (MSE) using the formula $\frac{1}{n}\|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$.

# Chapter 4

# Intro to mlsynth

This is not a book about Python. However, Python is the language I use to estimate synthetic control methods. In particular, I developed mlsynth, the largest synthetic control package in the world. While this book is not about mlsynth itself, I frequently use its functions to simplify operations and streamline examples. This chapter introduces the basics of mlsynth. I cover the key functions and tools that readers will need to follow along with the analyses in this book.

## 4.1 Installing mlsynth

Before using any mlsynth functions, the package must be installed. It can be installed directly from GitHub using pip with the following command:

```
pip install git+https://github.com/jgreathouse9/mlsynth.git
```

mlsynth requires Python version 3.9 or higher. It also depends on standard scientific Python packages including numpy, pandas, matplotlib, scipy, scikit-learn, statsmodels, cvxpy, pydantic, and scikit-optimize. Once installed, functions from mlsynth can be imported into a Python script or notebook. For example:

```
from mlsynth.utils.helperutils import dataprep, sc_diagplot
```

## 4.2 The mlsynth API

mlsynth provides a large number of functions internally, but I went through great pains to ensure that users do not need to interact with most of them if they do not want to. The package is designed so that analysts mainly work with one standardized API: the high-level estimator classes. Each estimator, has a fit method that handles all the underlying data preparation, estimation, effect calculation, and plotting automatically.

Before running any estimator, it helps to understand how mlsynth organizes the panel data. All class estimators in mlsynth share a common structure: they take a configuration dictionary that fully describes the panel. A dictionary is a data structure that stores key-value pairs, meaning each "key" identifies a piece of information, and the "value" holds the corresponding data. This allows the functions to access exactly what they need by referring to the keys. All estimators require the user to provide a data frame, the name of the outcome column, treatment, unit, and time columns. For example, here is the workflow for Proposition 99:

```
import pandas as pd
from mlsynth import FDID

url = "https://raw.githubusercontent.com/jgreathouse9/mlsynth/refs/heads/main/basedata/s
data = pd.read_csv(url)

# base config
config = {
    "df": data,
    "outcome": data.columns[2],
    "treat": data.columns[-1],
    "unitid": data.columns[0],
    "time": data.columns[1],
    "display_graphs": True,
    "save": False,
    "counterfactual_color": ["red"]
}
```

Here, the configuration dictionary fully describes the panel structure. df points to the loaded data. outcome identifies the variable to be modeled, cigsale, while treat points to the treatment indicator. unitid and time identify the cross-sectional units and time periods, respectively. Options for plotting and saving rests can also be specified.

Once this configuration is passed to the estimator, mlsynth handles the rest. The data are pivoted into a wide format, treated and donor units are identified, pre- and post-treatment periods are determined, and the counterfactual outcome is estimated. The treatment effect is then calculated and, if requested, plotted.

The key point is that the user never needs to manually manipulate the panel, compute donor matrices, or separate pre- and post-treatment periods. The configuration dictionary encapsulates all of these inputs, and the estimator performs the necessary steps internally. This makes running a synthetic control analysis with mlsynth both fast and reliable.

## 4.3 Preparing data with dataprep

dataprep is the lifeblood of mlsynth. Many synthetic control methods require the data to be in a wide format, with rows representing time periods and columns representing units. Treated units and donor units must be clearly separated, and pre-treatment and post-treatment

periods need to be identified. The dataprep function completely automates this process. It converts a long panel dataset into a wide format, such that each column is a unit and each row a time periods. It identifies treated units and donors, separates the data into pre- and post-treatment periods, and even handles cases with multiple treated units by grouping them into cohorts based on treatment start times.

The function takes a panel data frame and the names of the columns for the unit identifier, time period, outcome variable, and treatment indicator. It returns a dictionary containing the prepared data. For a single treated unit, the dictionary includes the treated unit name, a wide outcome matrix, the treated unit outcome vector, donor names, a donor outcome matrix, total periods, and the number of pre- and post-treatment periods. For multiple treated units, the dictionary contains the wide outcome matrix, a cohort dictionary with treated units for each cohort, donor matrices for each cohort, and the corresponding pre- and post-treatment periods. dataprep also performs checks to ensure that the panel is strongly balanced, meaning every unit has an observation for every time period and that there are no duplicate unit-time observations. This ensures that subsequent analyses using synthetic control methods are valid and comparable across units.

For example, consider the Basque data. After loading the panel into a DataFrame, we can call `dataprep` as follows:

```
from mlsynth.utils.datautils import dataprep
import pandas as pd

url =("https://raw.githubusercontent.com/jgreathouse9/mlsynth/refs/heads/main/basedata/b

data = pd.read_csv(url)

prepped_data = dataprep(
    data,
    "regionname",
    "year",
    "gdpcap",
    "Terrorism"
)

y, Y0, T0, time = prepped_data["y"], prepped_data["donor_matrix"], prepped_data["pre_per
```

dataprep returns a dictionary containing several key objects. y is the outcome vector for the treated unit, which in this case represents GDP per capita for the Basque Country. Y0 is the donor matrix, containing the same outcome variable for all donor regions that were not treated. T0 gives the number of pre-treatment periods, which defines the time window used to fit the synthetic control. time contains the time labels, such as years, for reference in plotting or analysis.

Using dataprep allows analysts to quickly extract the essential elements of the panel without manually pivoting the DataFrame or separating treated and donor units. This makes

subsequent estimation with any mlsynth estimator much simpler and less error-prone.

## 4.4 Visualizing Treated vs Donor Trends

After preparing the data for a synthetic control analysis, it is often useful to visually inspect how the treated unit compares to the donor pool before and after treatment. The `sc_diagplot` function provides this diagnostic view.

```
import pandas as pd
from mlsynth.utils.helperutils import sc_diagplot

# Load Proposition 99 smoking data
url = "https://raw.githubusercontent.com/jgreathouse9/mlsynth/refs/heads/main/basedata/s
data = pd.read_csv(url)

# Define a configuration dictionary for the treated unit
plot_config = [{
    "df": data,
    "unitid": "state",          # Column identifying cross-sectional units
    "time": "year",             # Column identifying time periods
    "outcome": "cigsale",       # Outcome variable to plot
    "treat": "Proposition 99",      # Treatment name
    "display_graphs": True,     # Whether to show the plot
    "save": False,              # Set a file path to save instead of showing
    "counterfactual_color": ["red"]
}]

# Generate the diagnostic plot
sc_diagplot(plot_config)
```

In synthetic control analyses, it is critical to understand how the treated unit relates to the donor pool. `sc_diagplot` provides a straightforward way to inspect this relationship.

The plot shows the trajectory of the treated unit, the individual donor units, and the mean of all donors. Visualizing the donor pool helps identify potential donors that closely match the treated unit before treatment. Plotting the mean of donors provides a reference for the average behavior of the control group, which is essentially the baseline against which the treatment effect will be estimated.

By comparing the treated unit to both the individual donors and their mean, analysts can check the pre-treatment fit and assess whether the donor pool overall provides a credible counterfactual. This step is important because the quality of the synthetic control estimate depends on the treated unit being well-represented by a combination of donors. Visual inspection can reveal mismatches, outliers, or trends that may influence the estimated treatment effect, giving analysts insight on their results before formal estimation.

# Part II

# Estimation and Inference

# Chapter 5

# Potential Outcomes and Comparative Case Studies

> "If there's another universe, please make some noise. Give me a sign..."
> — Solana Imani Rowe

This chapter introduces the potential outcomes framework—the cornerstone of causal inference—and connects it to the comparative case study framework that underlies SCM (Rubin, 2008). Together, they provide a formal way to think about causality when experiments are infeasible. In principle, causal inference is about constructing another universe: one where what did happen, did not happen. The goal is to use observable data and statistical structure to make that alternate universe as realistic and plausible as possible. Note that this chapter cannot cover all of the basics– there is already a very mature body of work that treats this in more detail. Because I do not wish to reinvent wheels, I refer readers to much more thorough texts such as Cunningham's *Mixtape*, Huntington-Klein's *The Effect*, and Facure's *Causal Inference for the Brave and True* (Cunningham, 2021; Facure Alves, 2022; Huntington-Klein, 2025). The mathematical foundation introduced in the previous chapter now becomes essential. It allows us to express these ideas formally, to see why counterfactual reconstruction is even possible, and to understand the role of similarity between units. We begin by defining the key ideas behind potential outcomes, clarifying why causal effects are unobservable for any individual unit, and introducing the notation and modeling ideas that makes these concepts precise.

We then turn to the comparative case study framework, which shows how researchers and practitioners attempt to approximate those missing outcomes in practice. When policymakers, marketers, or data scientists evaluate an intervention—be it a policy reform, a new advertising campaign, or an infrastructure project—they face the same profound challenge: we only ever observe one version of history. Randomized experiments, while ideal, are often impossible, unethical, or prohibitively expensive. The comparative case study approach provides a structured way to reason about "what would have happened otherwise," using other units or regions as stand-ins for the unobserved counterfactual. At the end of this chapter, we will connect this logic to real examples—such as the case of West Germany's reunification.

## 5.1 The Fundamental Problem of Causal Inference

Academic and business scientists face a fundamental challenge: for any given unit (e.g., a country, region, or individual), we can only observe one outcome at a time (Holland, 1986). If a unit receives a single treatment, we see the version of the outcome we see given that the treatment has happened; if it does not, we see the untreated outcome. Formally, for unit $j$ at time $t$ we have:

$$y_{jt} = d_j y_{jt}(1) + (1 - d_j) y_{jt}(0),$$

where $d_j \in \{0, 1\}$ indicates treatment status. The key algebraic fact here formalizes the intuition above: given that a treatment happens, the untreated outcomes $y_{jt}(0)$ go away. Given that the treatment has not yet been enacted, we do not see $y_{jt}(1)$, or the treated outcomes. Another key way of expressing this is

$$y_{jt} = \begin{cases} y_{jt}(1) = f_j(t), & t \in \mathcal{T}_1,\ j \in \mathcal{N}, \\ y_{jt}(1) = f_j(t), & t \in \mathcal{T}_2,\ j \in \mathcal{N}_0, \\ y_{jt}(1) = f_j(t) + \tau_{jt}, & t \in \mathcal{T}_2,\ j \in \mathcal{N}_1 \\ y_{jt}(0) = f_j(t), & t \in \mathcal{T}_2,\ j \in \mathcal{N}_1 \end{cases}$$

where $f_j(t)$ represents the *generic outcome-generating process* in the absence of treatment, and $\tau_{jt}$ is the *treatment effect*, potentially varying across time and units. The algebra of this suggests that we can recover the potential outcome by having an estimate of $y_{jt}(0)\ \forall\ t \in \mathcal{T}_2$. Given some estimate for $y_{jt}(0)$, we the may compute the treatment effect of the policy or intervention. Since individual causal effects are unobservable, we often estimate average effects across units or time. Define:

**Definition 5.1** (Average Treatment Effect)**.** The Average Treatment Effect (ATE) measures the expected difference in potential outcomes if all units were treated versus if none were treated:

$$\text{ATE} = \mathbb{E}[\, y_{jt}(1) - y_{jt}(0)\,].$$

**Definition 5.2** (Average Treatment Effect on the Treated)**.** The Average Treatment Effect on the Treated (ATT) measures the expected treatment effect *for those units that actually received the treatment*:

$$\text{ATT} = \mathbb{E}[\, y_{jt}(1) - y_{jt}(0) \mid d = 1\,],$$

These averages rely on comparing treated and untreated units, assuming we can approximate the counterfactual. Practically, however, these hold only in expectation. We ususally work with the in-sample analogs of these:

**Definition 5.3** (Average Treatment Effect)**.** The sample analog of the ATE is the difference in the average observed outcomes between treated and untreated units:

$$\widehat{\text{ATE}} = \frac{1}{N_1} \sum_{j:d_j=1} y_{jt} - \frac{1}{N_0} \sum_{j:d_j=0} y_{jt},$$

where $N_1$ and $N_0$ are the number of treated and untreated units, respectively.

Worked example: If your observed outcomes are $10, 12, 14$ and the outcomes for untreated units are $7, 8, 9$, then the sample $\widehat{\text{ATE}}$ is

$$(10 + 12 + 14)/3 - (7 + 8 + 9)/3 = 12 - 8 = 4.$$

**Definition 5.4** (Average Treatment Effect on the Treated)**.** The sample analog of the ATT focuses on treated units, using untreated units to approximate their counterfactual outcomes:

$$\widehat{\text{ATT}} = \frac{1}{N_1} \sum_{j:d_j=1} y_{jt}(1) - \frac{1}{N_1} \sum_{j:d_j=1} \widehat{y}_{jt}(0),$$

where $\widehat{y}_{jt}(0)$ denotes the estimated counterfactual outcome for treated unit $j$ in period $t$.

Worked example: If your observed outcomes are $15, 18, 20$ but absent treatment they would have been $12, 14, 16$, then the sample $\widehat{\text{ATT}}$ is

$$(15 + 18 + 20)/3 - (12 + 14 + 16)/3 = 17.67 - 14 = 3.67.$$

**Problem 5.1. Potential Outcomes**

Explain in your own words the fundamental problem of causal inference. Why can we never observe both potential outcomes $y_{jt}(1)$ and $y_{jt}(0)$ for the same unit at the same time?

**Problem 5.2. ATE vs. ATT**

Define the Average Treatment Effect (ATE) and the Average Treatment Effect on the Treated (ATT). How do they differ conceptually and in terms of their estimation?

**Problem 5.3. Causal Interpretation**

In the potential outcomes framework, how would you explain the difference between saying "the policy caused an increase in $y$" and saying "$y$ increased after the policy"?

## 5.2 Counterfactuals and the Role of Controls

Above, I introduced the idea of outcomes $y_{jt}$, but I did not mention where they came from. It turns out that being able to reason about this is central to understanding causal inference models more generally. A useful place to start is the idea of the linear factor model (Filipovic & Schneider, 2025).

**Assumption 5.1. Linear Factor Model**

$\forall j \in \mathcal{N}$, $t \in \mathcal{T}_1$, pre-treatment outcomes are generated like

$$y_{jt}(0) = \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

where $\boldsymbol{\lambda}_t \in \mathbb{R}^F$ is a vector of *common factors* affecting all units, $\boldsymbol{\mu}_j \in \mathbb{R}^F$ are *unit-specific factor loadings*, and $\varepsilon_{jt}$ are zero-mean *idiosyncratic errors*. Intuitively, $\boldsymbol{\lambda}_t$ captures time-varying shocks or trends shared across all units, while $\boldsymbol{\mu}_j$ determines how sensitive each unit is to those factors.

If you're anything like me, you're asking what the heck this means practically, so I'll do a small illustration. Suppose there are $F = 2$ factor loadings — one capturing geography and another capturing culture. For a treated unit and three donor units, this might might look like this:

| Unit | Geography ($\mu_{j1}$) | Culture ($\mu_{j2}$) |
|---|---|---|
| Treated (1) | 1.20 | 0.50 |
| Donor A | 1.05 | 0.40 |
| Donor B | 0.80 | 0.55 |
| Donor C | 0.60 | 0.70 |

Each column represents a *time-invariant latent component* that affects all periods. Units differ, however, in how much they are influenced by each factor (or, *load* on each factor*. For example, the treated unit loads relatively heavily on Geography (1.20) but moderately on Culture (0.50), while Donor C loads more on Culture (0.70) than on Geography (0.60).

At each time period, the common factors $\boldsymbol{\lambda}_t$ capture *time-varying shocks* that influence all units simultaneously, such as national economic trends, policy changes, or global events. For example, over four periods:

| Time $t$ | Seasonal Factor ($\lambda_{t1}$) | Cyclical Factor ($\lambda_{t2}$) |
|---|---|---|
| $t_1$ | 1.0 | 0.1 |
| $t_2$ | 0.5 | 0.3 |
| $t_3$ | -0.2 | 0.8 |
| $t_4$ | -0.8 | 0.4 |

Here, $\boldsymbol{\lambda}_t$ varies over time but is *common to all units*, which is why these are called "common factors." Multiplying the factor loadings $\boldsymbol{\mu}_j$ by the time-specific factors $\boldsymbol{\lambda}_t$ shows how each unit responds differently to the same underlying shocks, with $\varepsilon_{jt}$ capturing unit-specific noise. **?@fig-toylfm** gives a slightly extended version of this toy example, plotted so we can see.

Linear factor models can capture many types of temporal dynamics: seasonal factors repeat periodically within a year, capturing patterns like quarterly sales cycles or flu seasons; cyclical

factors represent longer-term economic or social cycles; and periodic or oscillatory factors capture phenomena that recur with a known period, such as day-night cycles or weekly traffic patterns.

It is also possible to define grouped factor models (Liao et al., 2025), where units are organized into groups that share similar baseline structures. In this setting, the overall model can be thought of as a baseline linear factor model, but the details — such as factor loadings or even the number of active factors — may vary within each group. For example, different regions in a country might have a shared set of seasonal and cyclical factors, but the magnitude of their loadings on each factor differs by region. This flexibility allows linear factor models to accommodate complex multivariate data while remaining interpretable: the common factors capture shared variation, the factor loadings capture heterogeneity in responses, and the idiosyncratic errors capture unit-specific randomness.

**Assumption 5.2. Affine Factor Combination**

Suppose the treated unit's factor loadings lie in the affine span of the donor loadings:

$$\boldsymbol{\mu}_1 = \alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w},$$

for some intercept $\alpha \in \mathbb{R}$ and weights $\mathbf{w} \in \mathbb{R}^{N_0}$, where $\mathbf{1}_F$ is a vector of ones of length $F$.

Oftentimes, we assume that the treated unit's factor loadings may be approximated by the loadings of the control group. It says that the treated unit's long-run structure — captured by its factor loadings $\boldsymbol{\mu}_1$ — can be expressed as a combination of the donor units' loadings. The scalar $\alpha$ accounts for overall level differences (an intercept or baseline effect), while $\mathbf{M}^\top \mathbf{w}$ represents a weighted blend of donor characteristics.

Much of the synthetic control literature can be interpreted through the lens of factor models and related data-generating processes. While this is not a book on econometric theory, sometimes I will algebraically manipulate the factor model to show how thinking about it carefully allows us to consider the problem of causal impact estimation in a more refined manner. The following result, Lemma 5.1, seeks to convey the utility of the factor model as a basis for making comparisons. This demonstration underlies the rest of the book, as it explains why, in principle, comparative case studies are possible.

**Lemma 5.1** (Reconstruction of Treated Outcomes)**.** *Suppose the untreated outcomes follow the linear factor model. Denote the treated unit's factor loading as well as the matrix of donor factor loadings as*

$$\boldsymbol{\mu}_1^\top \in \mathbb{R}^{1\times F}, \qquad \mathbf{M} = \begin{bmatrix} \boldsymbol{\mu}_2^\top \\ \vdots \\ \boldsymbol{\mu}_{N_0+1}^\top \end{bmatrix} \in \mathbb{R}^{N_0 \times F} \tag{5.1}$$

*Further, let the common factors be:*

$$\boldsymbol{\lambda}_t \in \mathbb{R}^{F\times 1}, \qquad \boldsymbol{\Lambda} = \begin{bmatrix}\boldsymbol{\lambda}_1 & \boldsymbol{\lambda}_2 & \cdots & \boldsymbol{\lambda}_T\end{bmatrix} \in \mathbb{R}^{F\times T}. \tag{5.2}$$

*Now, let the donor pool and target vector be*

$$\mathbf{y}_1 = \begin{bmatrix} y_{11}(0) \\ \vdots \\ y_{1T}(0)\end{bmatrix} \in \mathbb{R}^{T\times 1}, \qquad \mathbf{Y}_0 = \begin{bmatrix} y_{21}(0) & \cdots & y_{2T}(0) \\ \vdots & \ddots & \vdots \\ y_{N_0+1,1}(0) & \cdots & y_{N_0+1,T}(0)\end{bmatrix}^\top \in \mathbb{R}^{T\times N_0}, \tag{5.3}$$

*Let the affine combination assumption hold. Then the treated unit's outcomes can be expressed as a linear combination of the donor outcomes plus an intercept and a residual:*

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0\mathbf{w} + \boldsymbol{\varepsilon}_1^*, \quad \textit{where} \quad \boldsymbol{\varepsilon}_1^* = \boldsymbol{\varepsilon}_1 - \mathbf{E}_0\mathbf{w},$$

*with* $\mathbf{y}_1 = (y_{11}(0), \ldots, y_{1T}(0))^\top \in \mathbb{R}^T$, $\boldsymbol{\varepsilon}_1 = (\varepsilon_{11}, \ldots, \varepsilon_{1T})^\top$, *and* $\mathbf{E}_0 \in \mathbb{R}^{T\times N_0}$ *the matrix of donor idiosyncratic errors.*

**Corollary 5.1.** *Notice that the common factors* $\boldsymbol{\lambda}_t$ *do not explicitly appear in the lemma. There are two reasons for this. First, the common factors are* unobserved*, so we cannot directly use or adjust for them. Second, by definition, common factors affect all units in the same way at each time* $t$*, so they do not help distinguish the treated unit from the donors. Any linear combination of donor outcomes automatically carries the same influence of the common factors, meaning that these shared shocks cancel out when forming a weighted approximation of the treated unit's outcomes. The variation we can exploit for comparison comes entirely from the unit-specific loadings* $\boldsymbol{\mu}_j$ *and the idiosyncratic errors* $\varepsilon_{jt}$*, which differ across units.*

The linear factor model provides a powerful lens for interpreting comparative case studies. In the potential outcomes framework, the challenge was that the counterfactual outcome $y_{1t}(0)$ is unobserved. The factor model tells us why it might nevertheless be *reconstructable* as an estimate: if units share a common latent structure — that is, if their outcomes depend on a set of unobserved but shared factors $\boldsymbol{\lambda}_t$ — then the treated unit's trajectory before intervention can be expressed as a combination of the donor trajectories.

If the treated unit's factor loadings $\boldsymbol{\mu}_1$ can be approximated by an affine combination of the donors' loadings, the donors already encode information about how the treated unit would have responded to each common factor. Consequently, the untreated potential outcome of the treated unit, $y_{1t}(0)$, can be reconstructed as a weighted combination of donor outcomes. This is the formal justification behind comparative case studies.

**Problem 5.4. Linear Factor Model Interpretation**

Consider the linear factor model:

$$y_{jt}(0) = \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}.$$

a) Explain what the vectors $\boldsymbol{\lambda}_t$ and $\boldsymbol{\mu}_j$ represent.

b) Give an example of a real-world latent factor that could influence outcomes across units.

**Problem 5.5. Affine Factor Combination**

Describe the affine combination assumption:

$$\boldsymbol{\mu}_1 = \alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w}.$$

What does it imply about the relationship between the treated unit and the donor units?

**Problem 5.6. Reconstruction of Treated Outcomes**

Based on this Lemma, write the treated unit's outcome vector as a linear combination of donor outcomes plus an intercept and residual. Explain intuitively what each component represents.

## 5.3 What Is a Comparative Case Study?

Case studies in general are used to draw conclusions about a phenomenon of interest (say, economic development) by comparing some unit of interest to a set of others. In the classic text by Tocqueville et al. (2012) (a qualitative case study), the United States of America is compared to England, France, and elsewhere in Europe. Quantitative case studies use empirical data about the world to generate counterfactuals. Quantitative case studies approximate a counterfactual by drawing on the observable outcomes of other units that can stand in for the treated unit under alternative circumstances. The core idea here, as with Alexis de Tocqueville, is to find one or more units that resemble the treated unit as closely as possible prior to the intervention and to use their trajectories as a window into the outcome that might have occurred.

At the heart of the comparative case study framework is the importance of the similarity between the main unit of interest to the units we are comparing it to. Thus, it must begin with a clear identification of the treated unit and the intervention of interest, followed by a careful examination of pre-intervention outcomes and relevant background characteristics. Analysts then search for comparators, either single units or combinations thereof, that replicate the patterns of the treated unit as closely as possible before the intervention. Finally, the comparison of post-intervention trajectories provides an estimate of the intervention's effect, offering a way to reason about what would have happened under alternative circumstances. This framework is not a simple exercise in matching, but a thoughtful integration of domain knowledge, empirical observation, and careful judgment.

SCM builds on this by leveraging panel data (where multiple units (e.g., countries, regions, or markets) are observed over time) to compare a single treated unit against a set of untreated

control units. This allows us to construct a more accurate counterfactual trajectory for what might have happened absent the intervention.

## 5.4 West Germany

To make this concrete, consider the study of German reunification by Abadie et al. (2015) which is one of the most influential applications of the framework. Studying the impact of political and economic reunification on West German GDP per capita, their goal here is to see how GDP would have evolved in the absence of reunification. Of course, they face the challenge that no alternate West Germany exists. To approach this problem, they examined the set of OECD countries whose economic indicators, trade patterns, industrial composition, and growth trends before 1990 closely resembled those of West Germany.

We can begin from the perspective of the linear factor model. Here, each country's trajectory of GDP per capita can be thought of as reflecting a mixture of latent "factors" (maybe economic, geographic, institutional, and cultural ones) that shape its evolution over time. The donor countries (Australia, Austria, Belgium, Denmark, France, Greece, Italy, Japan, Netherlands, New Zealand, Norway, Portugal, Spain, Switzerland, the United Kingdom, and the United States) differ in how they load on these underlying dimensions. For instance, France, Austria, and Belgium may be understood as having high geographic and historical loadings relative to West Germany: they share borders, trade patterns, and deep pre-war economic ties, and Austria in particular was once part of the German Empire itself. These countries likely move closely with Germany on factors such as industrial composition, continental integration, and exposure to European macroeconomic shocks. By contrast, countries like Australia, New Zealand, or the United States, though still members of the OECD, are geographically and institutionally distant. Their loadings on these shared European factors may be weaker or even orthogonal.

Of course, this discussion is purely theoretical. We never observe the factor loadings outside of synthetic study, nor do we know how many underlying loadings exist in reality. The factor model is simply a conceptual device that helps us think clearly about why some countries make better comparisons than others. The guiding idea in comparative case studies is therefore to select units that are as similar as possible to the treated unit along these unobserved background dimensions — the economic, political, and historical forces that jointly shape their trajectories. When done thoughtfully, such comparisons help approximate the unobservable counterfactual world where the treatment did not occur.

To make this concrete, let's plot the outcome of interest. Consider **?@fig-west-germany-plot-one** which plots the GDP trajectory of West Germany versus the 16 control units. Simply by looking at the plot, which of these units are most likely to be similar to West Germany in the pre-intervention period? The answer unfortunately is not so obvious, as we can see that multiple units are close to the trajectory of West Germany before Reunification took place. The key thing to note here is that the pure empirical mean of OECD nations does not sufficiently resemble West Germany, in level or in trend. This means that the standard difference-in-differences estimator would be biased from parallel pre-intervention trends violations. Abadie et al. (2015) uses the SCM to estimate the impact of the reunification,

wherein only some of these OECD countries are assigned weight as a comparison for West Germany.

## 5.5 Mexican Drug War

Next, we can consider the setting of the Mexican Drug War. In 2008, the Mexican government invaded several states using the military to combat against the drug traffickers. We can view each state's homicide trajectory as arising from a set of latent factors — geographic, economic, and strategic — that influence violence over time. Chihuahua, for instance, exhibits consistently high homicide rates relative to many other states. From the factor model perspective, this pattern can be interpreted as Chihuahua having high loadings on factors related to border proximity, drug trafficking routes, and urban centers such as Ciudad Juárez. These factors capture the persistent structural and strategic drivers of violence, rather than transient shocks or random fluctuations.

Other states, particularly those farther from international borders or major trafficking corridors, load less heavily on these factors, which explains why their quarterly homicide rates are generally lower or less volatile. The empirical mean of other states — plotted in blue in **?@fig-mexhomdata** — can be thought of as a rough, unweighted average of these other latent trajectories. Chihuahua's consistently higher trajectory highlights the importance of underlying structural differences in shaping outcomes.

As with our discussion of West Germany, this is a conceptual exercise. We do not observe these latent factors or the exact loadings, and we do not know how many factors truly govern violence dynamics. The point of the linear factor model is to provide a lens through which we can interpret observed heterogeneity: units that share certain characteristics — geography, strategic importance, socio-economic conditions — are likely to move together on some latent dimensions, while units with different characteristics load differently. Comparative case studies aim to leverage this structure, ideally selecting control units whose latent trajectories resemble those of the treated unit prior to intervention. In Chihuahua's case, this might mean comparing it to other northern border states or urban hubs with similar strategic conditions, rather than to all Mexican states indiscriminately.

## 5.6 Marketing Measurement

A similar logic underlies how marketers evaluate campaigns in the absence of clean experiments. The challenges faced in marketing contexts mirror those encountered in policy evaluation, though the scale and timing can differ. When a retailer launches a new advertising campaign, managers are immediately confronted with the question of causality: are observed increases in sales truly the result of the campaign, or would sales have risen regardless due to underlying trends in demand, seasonality, or other market forces?

From a theoretical perspective, each region or store can be thought of as having latent factors that influence sales — for instance, local demographics, store size, historical demand patterns, or competitive intensity. Some regions may load heavily on these factors, while others less

so. To understand the campaign's effect, marketers often rely on holdout studies, sometimes referred to as geolift experiments. In a geolift study, a subset of regions or customer segments is deliberately excluded from the campaign and treated as a control group, while the remaining regions receive the intervention (for example, an increase in media spend compared to the business-as-usual scenario).

The goal is to see the effect of media spending on a given metric (called "conversions" in marketing jargon). If treated regions show higher conversions than holdout regions, this suggests the campaign caused the lift. Importantly, this comparison relies on the assumption that the latent factors governing sales in the treated regions are reasonably approximated by those in the holdout regions — that is, that the untreated units provide a plausible counterfactual trajectory for what would have happened absent the intervention. Random fluctuations, seasonal patterns, and other external shocks complicate this comparison, just as idiosyncratic noise affects observed outcomes in other contexts.

In short, the principle is the same as in policy evaluation or the study of violence in Chihuahua: differences in outcomes are interpreted through the lens of underlying latent factors. The better the holdout units resemble the treated units on these unobserved dimensions, the more credible the causal inference. Selecting control regions therefore requires careful judgment, informed by both past data and domain knowledge, rather than a purely mechanical choice.

## 5.7 Beyond Simple Control Selection

Even when the treated unit and a plausible set of comparison units are clearly identified, important design questions remain for researchers conducting quantitative case studies. Selecting which units to include is only part of the challenge; deciding how many units to use, and how to combine them, is equally consequential. Using too few units risks anchoring a counterfactual on idiosyncratic examples that happen to look similar by chance (such as, units that are only incidentally similar where other comparison units would be better). Using too many comparison units may blur distinctions and dilute meaningful resemblance between the treated unit and the comparison units.

A similar tension arises when analysts incorporate additional information beyond the main outcome of interest, such as auxilary covariates that capture relevant background characteristics. For example, if we are the data science unit of a hotel chain, we may wish to include temperature in our analyses because it can be a proxy for seasonality. Including covariates can improve comparability, but they also introduce new decisions: which variables matter most (and what would it mean in principle for them to matter most?). Furthermore, how many auxilary covariates should we use? In policy analysis, for instance, should unemployment, education, and industrial structure all be treated as equally important predictors of economic growth? In marketing, should store size, regional demographics, and past sales volatility all receive comparable emphasis when identifying holdout regions? Each of these choices shapes the resulting counterfactual.

The difficulties do not stop there. A core assumption of the comparative case framework is that the treated and comparison units evolve independently—that there are no spillovers or

cross-unit effects. Yet in many real settings, this assumption is fragile. Policy interventions can influence neighboring jurisdictions through migration, trade, or imitation. Marketing campaigns may spill over across geographic boundaries as customers travel, communicate, or respond to national media. When such spillovers occur, the comparison units no longer represent what would have happened without the intervention; they themselves are partly treated. Recognizing, diagnosing, and mitigating these influences is one of the most subtle and important tasks in applied comparative analysis.

## 5.8 Looking Ahead

Across policy evaluation, historical case studies, and marketing measurement, the central challenge is the same: we can observe only one version of history for any given unit, yet we wish to reason about what would have happened under alternative circumstances. Comparative case studies provide a structured way to approximate these unobserved counterfactuals by leveraging other units as stand-ins, under the assumption that these units share relevant background structure with the treated unit.

The linear factor model offers a conceptual lens for understanding why such approximations can work. In West Germany, latent factors may represent geography, industrial structure, and historical ties; France, Austria, and Belgium load heavily on these factors, making them plausible comparators. In Chihuahua, latent factors capture geographic proximity, trafficking routes, and urban centers like Juárez; northern border states or other strategic hubs are likely to share these underlying influences. In marketing, latent factors might include demographics, store characteristics, or historical demand patterns; holdout regions with similar factor loadings provide credible counterfactuals for the treated regions.

In all three cases, we never observe the factors or their loadings directly. The power of the factor model lies not in seeing the unobservables but in structuring our thinking: it identifies the dimensions along which units must be similar for comparisons to be meaningful, and clarifies why some units are better matches than others. Comparative case studies are, at their heart, an exercise in reasoning about these latent structures, selecting controls, and incorporating statistical adjustments that best approximate the unobserved counterfactual world.

Finally, this perspective underscores a core principle: constructing credible counterfactuals requires judgment, domain knowledge, and careful consideration of context. Whether analyzing historical events, criminal violence, or advertising campaigns, the goal is to approximate what would have happened in another version of the world — a world we cannot see but can approach through thoughtful comparison and careful use of observable data. This is the essence of causal inference, and it is the guiding logic behind every comparative case study we encounter. This idea will become more formalized in later chapters, as we work with more practical examples.

**Problem 5.7. Comparative Case Study Concept**

What is the core idea of a comparative case study? How does it relate to the potential outcomes framework?

**Problem 5.8. Selecting Comparison Units**

In designing a comparative case study, what factors make some units more appropriate comparators than others? Give at least two examples.

**Problem 5.9. Latent Factors in Real-World Examples**

Using the Chihuahua example, explain how latent factors (e.g., geography, drug trafficking routes) influence the homicide trajectory. Why does understanding these latent factors matter for constructing a counterfactual?

**Problem 5.10. Marketing Application**

Explain how the logic of potential outcomes and latent factors applies in a marketing context. How can holdout regions be used to estimate the effect of a new advertising campaign?

**Problem 5.11. Bonus Question: Evaluating a Marketing Campaign**

Imagine you are evaluating a marketing campaign across multiple stores, some treated and some held out as controls.

a) Explain how the linear factor model could help you justify which stores to use as controls.

b) Discuss potential challenges if treated stores are geographically clustered and subject to correlated shocks not present in the control stores. How would this affect your ATT estimates?

*Proof.* We start from the treated unit's factor model in vector form:

$$\mathbf{y}_1 = \mathbf{\Lambda}\boldsymbol{\mu}_1 + \boldsymbol{\varepsilon}_1.$$

Substituting the affine combination $\boldsymbol{\mu}_1 = \alpha\mathbf{1}_F + \mathbf{M}^\top\mathbf{w}$ gives

$$\mathbf{y}_1 = \mathbf{\Lambda}(\alpha\mathbf{1}_F + \mathbf{M}^\top\mathbf{w}) + \boldsymbol{\varepsilon}_1.$$

Using the distributive property of matrix multiplication:

$$\mathbf{y}_1 = \alpha\mathbf{\Lambda}\mathbf{1}_F + \mathbf{\Lambda}\mathbf{M}^\top\mathbf{w} + \boldsymbol{\varepsilon}_1.$$

Define $\mathbf{1}_T := \mathbf{\Lambda}\mathbf{1}_F \in \mathbb{R}^T$, so that the first term becomes $\alpha\mathbf{1}_T$. For the second term, note that

$$\mathbf{\Lambda M}^\top \mathbf{w} = (\mathbf{M\Lambda})^\top \mathbf{w}.$$

By definition,

$$\mathbf{Y}_0 = (\mathbf{M\Lambda})^\top + \mathbf{E}_0,$$

which implies $(\mathbf{M\Lambda})^\top = \mathbf{Y}_0 - \mathbf{E}_0$. Substituting gives

$$(\mathbf{M\Lambda})^\top \mathbf{w} = \mathbf{Y}_0 \mathbf{w} - \mathbf{E}_0 \mathbf{w}.$$

Combining all terms:

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0 \mathbf{w} + (\boldsymbol{\varepsilon}_1 - \mathbf{E}_0 \mathbf{w}).$$

Defining the combined residual $\boldsymbol{\varepsilon}_1^* := \boldsymbol{\varepsilon}_1 - \mathbf{E}_0 \mathbf{w}$, we have

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0 \mathbf{w} + \boldsymbol{\varepsilon}_1^*.$$

This shows that the treated unit's outcomes are a linear combination of the donor outcomes plus an intercept and a modified residual. ■ □

# Chapter 6

# Using The Control Group Mean

Now that we have spoken a little about averaging as an idea, we can finally formalize this in a panel data setting such that it is useful. Before, I was sort of vague about using averages for comparison; I gave a plot, but I never said how we would more formally do this. This chapter explains how we would do this as a first pass.

## 6.1 Two Way Fixed Effects

Recall the linear factor model as a data generating process. The original model assumes that some number of factor loadings and common time factors interact (along with covariates, potentially) to generate outcomes. However, this can be simplified by a lot to what we traditionally know as the two-way fixed effects model.

**Assumption 6.1. Two-Way Fixed Effects Model**

$\forall j \in \mathcal{N}, : t \in \mathcal{T}_1$, pre-treatment outcomes are generated according to

$$y_{jt}(0) = \mu_j + \lambda_t + \varepsilon_{jt},$$

where $\mu_j$ are *unit-specific intercepts* capturing time-invariant heterogeneity, $\lambda_t$ are *time-specific effects* capturing shocks or trends common to all units, and $\varepsilon_{jt}$ are zero-mean *idiosyncratic errors*. This model is a special case of the linear factor model, where there is a single factor with identical loadings across all units:

$$\boldsymbol{\lambda}_t = [1, \lambda_t]^\top, \quad \boldsymbol{\mu}_j = [\mu_j, 1]^\top.$$

In this representation, the first component captures unit-level heterogeneity, while the second captures the shared temporal component.

Intuitively, the TWFE model assumes that all units respond *equally* to the same common time trend $\lambda_t$, differing only in their fixed intercepts $\mu_j$. This simplicity makes it powerful for

many policy applications but also restrictive — if units react differently to aggregate shocks (i.e. have heterogeneous loadings on $\lambda_t$), the model is misspecified, and DID-type estimators may become biased. In that sense, the LFM generalizes TWFE by allowing these sensitivities to vary across units. **?@fig-dag-twfe** represents this DGP graphically.

To make this concrete, consider a toy example with two units over two time periods. Consider two units ($j = 1, 2$) observed over two time periods ($t = 1, 2$). Suppose

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}$$

holds. Let the fixed effects be

$$\mu = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \qquad \lambda = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Ignoring the random errors $\varepsilon_{jt}$, we can represent the predicted outcomes as a matrix where each row corresponds to a time period and each column to a unit:

$$M = \begin{bmatrix} y_{11} & y_{21} \\ y_{12} & y_{22} \end{bmatrix} = \begin{bmatrix} \mu_1 + \lambda_1 & \mu_2 + \lambda_1 \\ \mu_1 + \lambda_2 & \mu_2 + \lambda_2 \end{bmatrix} = \begin{bmatrix} 2+1 & 5+1 \\ 2+3 & 5+3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 5 & 8 \end{bmatrix}.$$

In this example, moving from the first to the second row adds the same amount, +2, to each unit because $\lambda_t$ changes by the same amount for everyone.

**Problem 6.1. Additive Factor Model**

Explain in your own words what the additive factor model

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}$$

represents. What do the terms $\mu_j$ and $\lambda_t$ capture in practical terms?

**Problem 6.2. Unit Effects vs. Time Effects**

Give an example of a real-world scenario where two units might share $\lambda_t$ but differ in $\mu_j$. How would this difference affect naive comparisons of levels?

## 6.2 Demeaning and Difference-in-Differences

A simple way to approximate the counterfactual outcome for a treated unit is to use the mean of the control group. That is, we could take the average outcome of untreated units and use it as an estimate of what the treated unit would have experienced without treatment. However, using the mean alone implicitly assumes that all units start from the same level—that

is, that their $\mu_j$ are identical. In practice, this assumption is rarely justified. Units may differ in income, size, resources, or other characteristics long before any intervention occurs. Ignoring these baseline differences can be misleading: post-treatment differences might simply reflect where units began, not the causal effect of the treatment. This is the basic intuition behind DID. To formalize this idea, we can difference out these time-invariant differences via *demeaning*.

**Lemma 6.1** (Demeaning)**.** *Let the observed outcome for unit $j$ at time $t$ in the pre-treatment periods $\mathcal{T}_1$ follow the additive two-way fixed effects model*

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}, \qquad t \in \mathcal{T}_1,$$

*where $\mu_j$ is a unit-specific baseline, $\lambda_t$ a common time effect, and $\varepsilon_{jt}$ an idiosyncratic error with finite mean. Define the pre-period demeaned outcome*

$$r_{jt} := y_{jt} - \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} y_{js}.$$

*Then for every $j$ and $t \in \mathcal{T}_1$,*

$$r_{jt} = (\lambda_t - \bar{\lambda}) + (\varepsilon_{jt} - \bar{\varepsilon}_j),$$

*where $\bar{\lambda} := \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} \lambda_s$ and $\bar{\varepsilon}_j := \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} \varepsilon_{js}$. In particular, the unit baseline $\mu_j$ cancels exactly, so $r_{jt}$ depends only on deviations of time effects and idiosyncratic errors from their pre-period averages.*

Demeaning is a fundamental idea in econometrics; see, for example, the review by Brabander et al. (2025). Rather than comparing treated and control units at a single moment based on current levels, we examine how their outcomes *evolve* across time. Specifically, we compare the *change* in the treated unit before and after treatment to the *change* observed in untreated units over the same period. By focusing on relative changes, we adjust for baseline differences. The crucial insight is this: what matters is not that two units start at the same level, but that their *pre-treatment trends move similarly*. If trajectories are parallel before treatment, any divergence afterward can be attributed to the treatment itself. This shift from levels to trends is precisely what the Difference-in-Differences estimator formalizes.

**Problem 6.3. Baseline Differences**

Why is it important to adjust for baseline differences between treated and control units when estimating treatment effects? How does DID address this?

## 6.3 Counterfactuals Under DID

Suppose we have a setting where a group of units is treated all at once, while others are never treated. The simplest form of the DID estimator can be obtained by minimizing the distance between the treated unit's outcomes and the average of untreated units, up to an additive

constant $\beta$ that adjusts for baseline differences. The counterfactual for the DID method is simply the mean of the control group plus some constant.

**Definition 6.1** (DID Counterfactual)**.** The post-treatment counterfactual for the treated unit under DID can be expressed as

$$\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*, \quad t > T_0,$$

where

$$\bar{y}_{0t} \coloneqq \frac{1}{N_0} \sum_{j \in \mathcal{N}_0} y_{jt}.$$

Here, $N_0 = |\mathcal{N}_0|$ is the number of untreated units, so that each control unit receives an equal weight of $N_0^{-1}$. $\beta$ is a scalar which is the mean difference between the treated and control group in the pre-treatment period.

A natural extension of this is the idea of DID as a centering estimator.

**Lemma 6.2** (Centered DID)**.** *For any fixed donor pool $\mathcal{N}_0$, the DID estimator for a single treated unit has a closed-form solution*

$$\beta^* = \frac{1}{T_0} \sum_{t=1}^{T_0} \Big( y_{1t} - \bar{y}_{\mathcal{N}_0 t} \Big), \qquad \bar{y}_{\mathcal{N}_0 t} = \frac{1}{|\mathcal{N}_0|} \sum_{j \in \mathcal{N}_0} y_{jt}.$$

*The residual vector after accounting for this optimal intercept is*

$$\mathbf{r} = \mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}, \qquad \mathbf{y}_c = \mathbf{y}_1 - \bar{y}_1 \mathbf{1}_{T_0}, \qquad \bar{\mathbf{y}}_{c,\mathcal{N}_0} = \bar{\mathbf{y}}_{\mathcal{N}_0} - \bar{m}_{\mathcal{N}_0} \mathbf{1}_{T_0}.$$

*Consequently, the pre-treatment sum of squared residuals is*

$$\ell_{DID}(\mathcal{N}_0) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}\|_2^2.$$

Even if the treated and control units hae different levels, the demeaning removes these level differences. DID relies on the Parallel Trends Assumption (PTA), which states that, in expectation, the difference between treated and control units would remain constant over time in the absence of treatment:

**Definition 6.2.**

**Definition 6.1. Parallel Trends Assumption**

In the absence of treatment,

$$\mathbb{E}[y_{1t}(0) - \bar{y}_{0t}(0)] = \mathbb{E}[y_{1t'}(0) - \bar{y}_{0t'}(0)], \qquad \forall t, t' \in \mathcal{T}_1.$$

Under this assumption, subtracting pre-treatment differences correctly removes all systematic level effects, so any post-treatment divergence can be interpreted as a causal effect of treatment. However, we cannot test for the Parallel Trends Assumption globally, as the counterfactual is never observed to the researcher. Thus, econometricians resort to a version of PTA that we may observe,

**Definition 6.3** (Parallel Pre-Trends Assumption)**.**

$$\mathbb{E}\Big[y_{1t}(0) - N_0^{-1} \sum_{j \in \mathcal{N}_0} y_{jt}(0)\Big] = \beta, \quad \forall t \in \mathcal{T}_1.$$

if the expectation holds in the pre-intervention period, we generally think that the DID estimator is unbiased (assuming, additionally, no anticipation and SUTVA hold; see Baker et al. (2025) for more).

**Problem 6.4. Parallel Trends Assumption**

Define the parallel trends assumption formally. How can you assess whether this assumption is plausible in pre-treatment data? You may integrate ideas from the recent staggered DID literature, if you wish.

**Problem 6.5. Counterfactual Construction**

Explain how the DID estimator, as discussed above, constructs the counterfactual for a treated unit. How does using the mean of control units approximate the unobserved $\lambda_t$?

## 6.4 Parallel Trends in Small $N$ Settings

DID is usually framed in the large $N$, small $T$ framework (Baker et al., 2025). In cases of small $N$, the parallel pre-trends assumption can be less likely to hold. To illustrate, consider the canonical paper on SCM by Abadie et al. (2010), which studies the impact of California's Anti-Tobacco Campaign, Proposition 99, on cigarette sales per capita. Enacted in 1989, Prop 99 attempted to reduce smoking via a statewide public health campaign. In the original dataset, the authors had 38 control states to work with from 1970 to 2000. Using the estimator described above, we will use Python to estimate the DID design for the effect of Prop 99 on tobacco consumption. To follow along, you'll need the `dataprep` helper from my mlsynth code, which may be installed like

```
pip install -U git+https://github.com/jgreathouse9/mlsynth.git
```

In the code below, we load the `dataprep` function alongside `numpy` and `pandas`, as well as the Prop 99 dataset.

We then implement the DID estimator as described above, using `numpy` in this case.

The pre-treatment mean difference $\approx -14.359$. So, this is what we subtract the donor mean vector entries by. When we do this, we get **?@fig-did-cali**.

We can see here that the scalar-adjusted control group average does not appear parallel to California. The counterfactual is dissimilar to California for the first five years of the dataset in that it undershoots California's values, and for the rest of the pre-treatment period, it has higher per capita smoking than California. Why might this be? The answer lies in the Law of Large Numbers and the interplay of small $N$ and large $T$. With only a few control units, the donor mean $\bar{y}_{0t}$ is highly sensitive to idiosyncratic deviations and unit-specific effects in each control state. Formally, the variance of the control mean is

$$\text{Var}\Big(\frac{1}{N_0}\sum_{j=1}^{N_0} y_{jt}\Big) = \frac{1}{N_0^2}\sum_{j=1}^{N_0}\text{Var}(y_{jt}),$$

so smaller $N_0$ directly increases the volatility of the average. Over many pre-treatment periods (large $T$), these idiosyncratic shocks accumulate, making the pre-treatment trajectory of the control average unstable and less likely to satisfy the Parallel Trends Assumption (PTA). Put another way, parallel trends assumes any systematic variation over time is common across units (i.e., the common tiem effect of the factor model), and, beyond this common variation, any remaining variation in outcomes is zero-mean and idiosyncratic. That is, the estimator effectively assumes that we can ignore unit-specific effects and focus on trends. This holds only in expectation, and relies on having either a sufficient number of "good" control units in a small sample or a large number of controls so that outliers do not dominate the donor mean. In the small $N$, large $T$ setting, this is less likely to hold. With few control units, idiosyncratic shocks are more influential.

This becomes even clearer when we experiment with different control units. Here, we compare California to Kentucky, NC, and both jointly, using a DID model for each state individually as well as jointly. These results are presented in **?@fig-did-calikent**. For example, Kentucky's cigarette consumption exhibits persistent idiosyncratic dynamics—its high baseline smoking rates and tobacco-based economy create predictable deviations that are correlated with pre-treatment outcomes. Similarly, NC displays systematic differences from California that violate the martingale property. As a result, even after demeaning by the donor mean, the control units' evolution is not mean-independent of their pre-treatment shocks.

Practically, these violations are evident when we compare the ATT estimates. Using all controls, the naive DID yields an ATT of roughly $-27$. Using Kentucky alone yields $-29$, NC alone yields $-34$, and the Kentucky + NC combination gives $-32.36$. Small $N$ exacerbates the issue because the law of large numbers cannot stabilize the average across control units. This provides a concrete illustration of why PTA may not hold in real-world, small-$N$ applications like Proposition 99.

It is important to emphasize that this is not a fault of DID per se. The DID method performs as intended when its key assumptions hold: SUTVA (no interference between units) and parallel trends. In the chapters that follow, we will explore other forms of looking at this problem, which address these issues by constructing a weighted combination of controls to better match the treated unit's pre-treatment trajectory, mitigating the impact of non-parallel trends and small $N$ challenges.

**Problem 6.6. Small N and Pre-Treatment Variation**

In settings with a small number of control units, the pre-treatment average of the controls can fluctuate due to idiosyncratic shocks in individual units.

a) Explain why this makes the pre-treatment trajectory of the donor mean potentially unstable.

b) How does this instability affect the accuracy of the DID counterfactual for the treated unit?

c) What role does the number of control units ($N$) play in mitigating or exacerbating this problem?

**Problem 6.7. Observed vs. Counterfactual Trajectories**

Suppose California's pre-treatment trajectory is steadily decreasing while Kentucky's is flat. Sketch and describe what the DID counterfactual using Kentucky would look like. Why might this estimate be biased?

**Problem 6.8. Multiple Donors**

Conceptually, why might using multiple control units (e.g., Kentucky + NC) improve the counterfactual estimate? Under what circumstances might it still fail?

**Problem 6.9. Interpretation of ATT**

If a DID analysis gives an ATT of –32, what does this mean in terms of California's cigarette consumption post–Proposition 99?

**Problem 6.10. Limitations of DID**

Discuss at least two situations in which DID may give biased estimates, even if implemented correctly. Relate your answer to the additive factor model.

**Problem 6.11. Arithmetic Mean as a DID Counterfactual**

In previous chapters, we defined the arithmetic average as a weighted sum where each member's weight is proportional to its size in the sequence. In the context of using the arithmetic mean of control units to construct a DID counterfactual:

1. Explain what assumptions are being made implicitly about the control units when we take a simple arithmetic mean. Consider both their contribution to the counterfactual and their relative similarity to the treated unit.

2. Discuss potential limitations of this assumption. Under what circumstances might the arithmetic mean of controls misrepresent the "true" counterfactual trajectory?

*Proof.* By definition,

$$r_{jt} = y_{jt} - \frac{1}{|\mathcal{T}_1|}\sum_{s\in\mathcal{T}_1} y_{js} = (\mu_j + \lambda_t + \varepsilon_{jt}) - \frac{1}{|\mathcal{T}_1|}\sum_{s\in\mathcal{T}_1}(\mu_j + \lambda_s + \varepsilon_{js}).$$

Because summation is linear,

$$\frac{1}{|\mathcal{T}_1|}\sum_{s\in\mathcal{T}_1}(\mu_j + \lambda_s + \varepsilon_{js}) = \mu_j + \bar{\lambda} + \bar{\varepsilon}_j.$$

Subtracting this average from $y_{jt}$ gives

$$r_{jt} = (\mu_j + \lambda_t + \varepsilon_{jt}) - (\mu_j + \bar{\lambda} + \bar{\varepsilon}_j) = (\lambda_t - \bar{\lambda}) + (\varepsilon_{jt} - \bar{\varepsilon}_j),$$

which proves the claim. □

*Proof.* Expanding the squared Euclidean norm gives

$$f(\beta) = \sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{0t} - \beta\Big)^2.$$

For clarity, define the inner function

$$u_t(\beta) = y_{1t} - \bar{y}_{0t} - \beta,$$

so that

$$f(\beta) = \sum_{t=1}^{T_0} u_t(\beta)^2.$$

To differentiate $u_t(\beta)^2$ with respect to $\beta$, we apply the chain rule. Here, the outer function is $g(u) = u^2$ with derivative $g'(u) = 2u$, and the inner function is $h(\beta) = u_t(\beta) = y_{1t} - \bar{y}_{0t} - \beta$ with

$$\frac{\mathrm{d}h}{\mathrm{d}\beta} = -1,$$

since $y_{1t}$ and $\bar{y}_{0t}$ are constants with respect to $\beta$. Applying the chain rule, we find the derivative of each term:

$$\frac{\mathrm{d}}{\mathrm{d}\beta}u_t(\beta)^2 = 2u_t(\beta)\cdot(-1) = -2\Big(y_{1t} - \bar{y}_{0t} - \beta\Big).$$

Summing over all $T_0$ pre-treatment periods, by linearity of the derivative, gives

$$\frac{\mathrm{d}f}{\mathrm{d}\beta} = -2\sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{0t} - \beta\Big).$$

Setting this derivative equal to zero, which is our first-order condition (FOC), we have

$$-2\sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{0t} - \beta\Big) = 0.$$

Dividing both sides by $-2$ yields

$$\sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{0t} - \beta\Big) = 0.$$

Distributing the summation and noting that $\beta$ is constant across $t$:

$$\sum_{t=1}^{T_0}(y_{1t} - \bar{y}_{0t}) - \sum_{t=1}^{T_0}\beta = \sum_{t=1}^{T_0}(y_{1t} - \bar{y}_{0t}) - T_0\beta = 0.$$

Now we solve for beta. To do this we add $T_0\beta$ to the RHS. That gives us this result:

$$\sum_{t=1}^{T_0}(y_{1t} - \bar{y}_{0t}) = T_0\beta$$

Now to isolate beta, we divide both sides by $T_0$, noting that division by a scalar $T_0$ is the same as multiplying by the reciprocal of that scalar:

$$\beta^* = \frac{1}{T_0}\sum_{t=1}^{T_0}(y_{1t} - \bar{y}_{0t}).$$

This shows that $\beta^*$ is simply the average pre-treatment difference between the treated unit and the control group. Under parallel trends, this difference would hold in expectation, even absent treatment. Finally, the post-treatment counterfactual for the treated unit is simply the control group mean adjusted by this optimal $\beta^*$:

$$\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*, \quad \forall\, t \in \mathcal{T}_2$$

□

*Proof.* For any fixed donor pool $\mathcal{N}_0$, we know from this proof

$$\beta^* = \frac{1}{T_0}\sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{\mathcal{N}_0,t}\Big) \qquad \text{where} \qquad \bar{y}_{\mathcal{N}_0,t} = \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}.$$

Here is the residual:

$$r_t = y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \beta^* \qquad \text{where} \qquad \beta^* = \frac{1}{T_0}\sum_{t=1}^{T_0}\left(y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right).$$

Substitute $\beta^*$ directly into the residual:

$$\begin{aligned} r_t &= y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \frac{1}{T_0}\sum_{t=1}^{T_0}\left(y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right) \\ &= y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \frac{1}{T_0}\sum_{t=1}^{T_0} y_{1t} + \frac{1}{T_0}\sum_{t=1}^{T_0}\left(\frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right). \end{aligned}$$

Now recognize the averages:

$$\frac{1}{T_0}\sum_{t=1}^{T_0} y_{1t} = \bar{y}_1$$

and

$$\frac{1}{T_0}\sum_{t=1}^{T_0}\left(\frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right) = \frac{1}{T_0}\sum_{t=1}^{T_0}\bar{y}_{\mathcal{N}_0,t} = \bar{m}_{\mathcal{N}_0}.$$

So

$$r_t = y_{1t} - \bar{y}_1 - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} + \bar{m}_{\mathcal{N}_0}$$

becomes

$$(y_{1t} - \bar{y}_1) - \left(\frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \bar{m}_{\mathcal{N}_0}\right),$$

which simplifies to

$$(y_{1t} - \bar{y}_1) - (\bar{y}_{\mathcal{N}_0,t} - \bar{m}_{\mathcal{N}_0}).$$

In vector form, this is:

$$\boxed{\mathbf{r} = \mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0} \qquad \Rightarrow \qquad \ell_{\text{DID}}(\mathcal{N}_0) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}\|_2^2}$$

where

$$\mathbf{y}_c = \mathbf{y}_1 - \bar{y}_1 \mathbf{1}_{T_0}, \qquad \bar{\mathbf{y}}_{c,\mathcal{N}_0} = \bar{\mathbf{y}}_{\mathcal{N}_0} - \bar{m}_{\mathcal{N}_0} \mathbf{1}_{T_0}.$$

□

# Chapter 7

# Using Least Squares

> *"Hey, baby, I really do, really do j'adore you... But you're so complicated."*
> — *Neo Jessica Joshua*
>
> *"You're the worst, you know what you've done to me."*
> — *Jhené Aiko Efuru Chilombo*

— *OLS: beautiful in theory, complicated in interpretation, and sometimes literally the worst choice for generating synthetic controls.*

After reviewing DID as a method to generate counterfactuals, we saw that it implicitly assigns equal weight to all control units. In other words, DID assumes that each control unit should contribute identically to the counterfactual outcome of the treated unit, and that the baseline differences are enough to account for differences in trends. While this assumption simplifies estimation, it is often unrealistic. Control units can differ widely in how closely they resemble the treated unit before treatment, so treating them all as equally informative may distort the counterfactual.

To address this limitation, we need a method for weighting the donor pool $j \in \mathcal{N}_0$ in a way that reflects each unit's similarity to the treated unit. One natural idea is to use ordinary least squares (OLS): choose weights that minimize the squared difference between the treated unit's pre-treatment outcomes and a weighted combination of the donors. This formulation is exactly what synthetic control builds upon. In the next chapters, we'll see how SCM modifies this OLS problem by adding constraints to the weights—turning this unconstrained regression into a more interpretable and robust estimator.

## 7.1 The Model

Formally, if $\mathbf{y}_1$ is the vector of pre-treatment outcomes for the treated unit and $\mathbf{Y}_0$ is the matrix of pre-treatment outcomes for the donor units, we can define the OLS problem as:

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

Here, the resulting weights $\mathbf{w}^*$ can then be used to construct a synthetic control, both pre- and post-treatment, by taking a weighted sum of the donor outcomes. In essence, OLS assigns different weights to control units according to their similarity to the treated unit. We can examine the classic example of the Basque Country in Spain, which experienced a surge in terrorism in the 1970s due to the terrorist organization ETA murdering politicians and doing kidnappings. Suppose we want to estimate what Basque GDP per capita would have been in the absence of terrorism. Using OLS, we can easily construct a synthetic Basque by finding the combination of other Spanish regions whose weighted average best reproduces the Basque GDP path in the pre-1975 period. When we solve the OLS problem, each donor region receives a weight that minimizes the squared difference between its weighted combination and the Basque trajectory before the intervention. Intuitively, regions whose economic paths closely track the Basque Country are given higher weights, while less similar regions receive lower weights.

> **i** Programming Note
>
> This is the first chapter in which we will cease doing the math by hand. Although OLS has a closed-form solution, the problem we consider here has about 20 observations and 16 predictors. You may do this by hand if you wish, but that will likely take much longer than you'd want to spend. So, from here on out, `cvxpy` and `numpy` will be your best friends here.

## 7.2 Counterfactuals Generated by OLS

```
import cvxpy as cp
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from mlsynth.utils.datautils import dataprep
from IPython.display import display, Markdown
from scmbook.style import set_book_style
set_book_style()

url = ("https://raw.githubusercontent.com/"
       "jgreathouse9/mlsynth/"
       "refs/heads/main/basedata/"
       "basque_data.csv")


data = pd.read_csv(url)

prepped_data = dataprep(data,
"regionname",
"year",
```

```
"gdpcap",
"Terrorism")

y, Y0, T0, time = prepped_data["y"], prepped_data["donor_matrix"], prepped_data["pre_per

# Extract Ywide
Ywide = prepped_data["Ywide"]

subset = Ywide.iloc[:10, :4]

# Round to 2 decimal places
subset_rounded = subset.round(2)

# Convert to markdown
md_table = subset_rounded.to_markdown(index=True)

# Display
display(Markdown(md_table))
```

When we use OLS to construct a synthetic control, nothing mysterious is happening. We are literally just running a linear regression of the treated unit on the donor units, using only pre-treatment data. In this setup, the regression coefficients – the betas – are exactly the weights $\mathbf{w}$ that tell us how much each donor contributes to the synthetic control.

Formally, if $\mathbf{y}_1$ is a $T_0 \times 1$ vector of the treated unit's outcomes in the pre-treatment period and $\mathbf{Y}_0$ is a $T_0 \times J$ matrix of donor outcomes, the OLS weights $\mathbf{w}^*$ are the solution to the normal equations $\mathbf{Y}_0^\top \mathbf{Y}_0 \mathbf{w}^* = \mathbf{Y}_0^\top \mathbf{y}_1$. When we solve, we get: $\mathbf{w}^* = (\mathbf{Y}_0^\top \mathbf{Y}_0)^{-1} \mathbf{Y}_0^\top \mathbf{y}_1$. This is just the standard formula for OLS regression coefficients. The solution minimizes the sum of squared differences between the treated unit and the weighted combination of donors over the pre-treatment period. We can compute the OLS weights directly in Python using `numpy` or `cvxpy`:

See? Same weights! No tricks, no fancy adjustments, just plain old least squares. Let's look at who gets selected:

Much of this does not really square very nicely with intuition. Cataluna, which is very close to the Basque Country (both literally in terms of GDP and being a French border region in Northern Spain) gets negative weight from OLS. The units which are much farther away from the Basque Country (on top of having much different political/economic histories) get much more weight. The reason for this is simple: OLS cares about minimizing errors first and foremost, and will assign any kind of weight it needs to pursuant to that end. This is the unconstrained minimizer, subject to no constraints upon the weights other than them being on the real line. Either way, we can still compute a counterfactual. Let's recall our chapter on averaging: all a weighted average is, is where the weight assigned per donor/element may vary. In the DID estimator we do

$$\beta^* = \underset{\beta \in \mathbb{R}}{\operatorname{argmin}} \left\| \underbrace{\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}}_{\text{Fit}} - \underbrace{\beta}_{\text{Mean difference}} \right\|_2^2 \quad \text{s.t.} \quad w_j = \frac{1}{N_0}, \quad \forall t \in \mathcal{T}_1, \, \forall j \in \mathcal{N}_0$$

where the arithmetic mean wins the day. Here, by contrast, the weights are assigned entirely through least squares (without an intercept term).

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \, \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

And, as per the definition of a weighted sum, here's what happens next: we take the weight for Andalucia and multiply it by all the values of the Andalucia vector. Then we do the same for Aragon, and the rest of the donors, scaling them by the weight OLS assigns. Finally, we sum these weighted vectors to obtain our synthetic counterfactual trajectory, or $\hat{y}^{\text{LS}} = \mathbf{Y}_0 \mathbf{w}^*$. Let's do that now.

Here is our prediction using OLS. Unfortunately, while OLS fits very well to the Basque Country before terrorism happened, the out-of-sample/post-1975 predicions exhibits a lot more variance than any control unit in the pre-treatment period did. OLS interpolates perfectly within the span of donor trajectories in-sample, but because the coefficients are unconstrained, extrapolation outside the pre-treatment window becomes unstable. The coefficients for the in-sample model swing wildly when applied to new data. We can see that the prediction line for the Basque country suggests that its economy... would have fallen off a cliff... had terrorism not happened? In other words, if the OLS estmate is to be taken seriously, the onset of terrorism saved the Basque economy... This does not seem like a sensible finding historically, or a very reasonable prediction statistically.

**Problem 7.1. DID vs OLS**

Explain in your own words why DID assigns equal weight to all control units, and why this assumption might be unrealistic when constructing counterfactuals.

**Problem 7.2. Understanding OLS Weights**

Describe the role of OLS weights in constructing a synthetic control. How does OLS decide which donor units get larger weights?

**Problem 7.3. Negative Weights**

1. Explain why OLS can produce negative weights.

2. What are the implications of a negative weight when creating a counterfactual?

**Problem 7.4. Coding Pre-Treatment Fit**

Using the Basque Country dataset:

```
import numpy as np
import pandas as pd
from mlsynth.utils.datautils import dataprep

# Load data
url = "https://raw.githubusercontent.com/jgreathouse9/mlsynth/refs/heads/main/basedata/b
data = pd.read_csv(url)

# Prepare pre-treatment data
prepped_data = dataprep(data, "regionname", "year", "gdpcap", "Terrorism")
y, Y0, T0, time = prepped_data["y"], prepped_data["donor_matrix"], prepped_data["pre_per

# TODO: compute OLS weights and plot pre-treatment fit
```

1. Compute OLS weights using either `numpy` or `cvxpy`.
2. Plot the treated unit against the OLS synthetic control in the pre-treatment period.
3. Comment on which donors dominate the pre-treatment fit. What is the Root Mean Squared Error?

**Problem 7.5. Post-Treatment Predictions**

1. Extend your synthetic control to the post-treatment period.
2. Plot the post-treatment predictions and the actual outcomes.
3. Discuss any issues you observe with the OLS predictions after the intervention.

**Problem 7.6. Overfitting Concept**

1. Define overfitting in the context of OLS synthetic controls.
2. Explain why overfitting occurs more readily when there are many donor units relative to pre-treatment periods.

**Problem 7.7. Simulated Overfitting**

```
# Simulate data
np.random.seed(42)
n, p = 10, 5
X = np.random.randn(n, p)
true_w = np.array([1, 0.5, 0, 0, 2])
y = X @ true_w + np.random.randn(n) * 0.5

# TODO: fit unconstrained OLS and plot predicted vs true values
```

1. Fit an unconstrained OLS model to the data.
2. Plot the predicted values vs the true values.
3. Comment on the in-sample fit and potential issues if you were to predict new outcomes.

## 7.3 The Problem with OLS

The OLS approach to constructing synthetic controls, while straightforward, often produces unreliable counterfactuals, as seen in the Basque Country example. The erratic post-1975 predictions, which suggest an implausible economic collapse absent terrorism, highlight a fundamental issue: OLS prioritizes in-sample fit at the expense of out-of-sample validity. This section explores why OLS fails in this context and the statistical mechanisms driving these shortcomings. OLS is designed to minimize the sum of squared differences between the treated unit's pre-treatment outcomes ($\mathbf{y}_1$) and the weighted combination of donor outcomes ($\mathbf{Y}_0\mathbf{w}$):

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

The closed-form solution, $\mathbf{w}^*_{\text{OLS}} = (\mathbf{Y}_0^\top\mathbf{Y}_0)^{-1}\mathbf{Y}_0^\top\mathbf{y}_1$, achieves this by assigning weights that best fit the pre-treatment data. However, this unconstrained optimization leads to overfitting to noise, where OLS captures both systematic patterns and idiosyncratic noise ($\varepsilon$) in the treated unit's pre-treatment outcomes, producing a synthetic control that closely tracks the pre-treatment period but fails to generalize post-treatment. This explains the volatile "zig-zag" counterfactual in the Basque case, which lacks the stability of any individual donor unit.

Without restrictions, OLS can also assign positive, negative, or arbitrarily large weights to donor units, leading to counterintuitive results. In the Basque example, Cataluna—a region economically and geographically similar to the Basque Country—received a negative weight, while distant regions like Castilla y León were assigned large positive weights. These weights prioritize in-sample accuracy over economic or historical plausibility, undermining interpretability. Furthermore, the model $\mathbf{y}_1 = \mathbf{Y}_0\mathbf{w} + \varepsilon$ is only an approximation of the true data-generating process. By fitting the pre-treatment data too closely, OLS produces a synthetic control that lacks predictive power outside the pre-treatment/in-sample period, resulting in absurd post-treatment predictions, such as the Basque economy being helped by terrorism.

In essence the lesson of this chapter is very simple: OLS's flexibility—its ability to select any weights in $\mathbb{R}^{N_0}$—is also its downfall. Without constraints ensuring some form of regularization, OLS sacrifices interpretability and robustness for an overly precise in-sample fit.

## 7.4 Where OLS Leaves Off

Even though we see that it produces complicated and in this case uninterpretable results, OLS is still the most natural starting point for counterfactual construction in the panel data setting: it learns weights that make the treated unit's pre-treatment outcomes as close as possible to a combination of donors. But as we've seen, this freedom comes at a price. By allowing weights to take any real value, OLS can perfectly interpolate the pre-treatment

data but at the cost of producing implausible, unstable counterfactuals. In this sense, OLS represents the *purest* form of fitting: a method with no guardrails to contain its predictions.

The next chapter introduces *penalized least squares*, where we rein in OLS by attaching a penalty to the size (or shape) of the weight vector. This penalty forces the model to balance two competing goals:

1. **Fit:** stay close to the treated unit's pre-treatment outcomes.

2. **Simplicity:** avoid overly large or erratic weights.

When we add these penalties, new geometries emerge. The $\ell_2$ penalty (ridge) keeps weights small but allows them all to stay active; the $\ell_1$ penalty (lasso) forces some weights to zero, yielding sparse synthetic controls. And much later on, when we go one step further—replacing soft penalties with hard constraints—we arrive at the synthetic control method. OLS, then, is the first stop in a larger journey: from *fitting everything* to *fitting responsibly*.

# Chapter 8

# Penalized Synthetic Controls

> "I'll keep it simple, baby... I'ma keep it simple with you, baby. You know I don't ever play no games. You know I don't ever complicate it."
> — *Jhené Aiko Efuru Chilombo*

In the last chapter we saw how OLS can fit the pre-treatment data almost perfectly, yet fail catastrophically out of sample. This behavior is a textbook example of *overfitting*. Overfitting occurs when a model captures not only the systematic patterns in the data but also the random noise specific to the sample at hand. An overfit model performs tends to perform extremely well on the data it was trained on but generalizes poorly to new or unseen data. In our case, the OLS synthetic control overfits the pre-treatment outcomes of the Basque Country by adjusting the donor weights so precisely that it reproduces every wiggle in the historical GDP series, including idiosyncratic shocks that have nothing to do with long-run economic structure. Formally, overfitting is a manifestation of the bias–variance tradeoff. A highly flexible model has low bias—it can approximate the training data closely—but high variance, meaning that small perturbations in the data can cause large swings in the estimated parameters. Bias, by extension, is about the difference between the model prediction and the true values; the OLS synthetic control has very low bias in the pre-treatment period, but has extremely high variance in the post-intervention period. That is, the close fit OLS obtains comes at the cost of having high post-intervention variance in the out of sample predictions.

To make OLS behave more responsibly, we need to give it boundaries — some way to tell the model that a perfect fit is not always desirable if it means using bizarre or extreme weights. Statistically, this is called **regularization**. In optimization terms, it means we trade off fidelity for discipline: we still want the in-sample (pre-treatment) prediction to fit well, but not so well that it memorizes noise or violates interpretability. Regularization is a method for tempering this variance by constraining or penalizing the size of the coefficient vector. In this setting, we accept a small amount of bias in exchange for substantial reductions in variance and improved stability of the predictions. Regularization modifies the least squares objective by adding a penalty term that discourages overly complex or extreme coefficients. When we say penalty term, we usually talk about adding a norm to the regression problem. In geometric terms, regularization shrinks the solution space from an open field to a smaller

playground—the model can still move, but not run off into absurd directions. This trades off fit (the first term) against simplicity (the second). The tuning parameter $\lambda$ controls how much we're willing to sacrifice accuracy to gain stability. The general form of a regularized least squares problem is

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_p^p,$$

where $\lambda \geq 0$ controls the strength of the penalty and $p$ determines its type.

To develop intuition, consider an analogy from the music industry—one that reflects how access, selectivity, and diversity shape outcomes.

### 8.0.1 The Sparse World: Motown and the $\ell_1$ Norm

Back in the 1960s and 70s, the music world was dominated by a handful of major record labels and A&R gatekeepers. Recording equipment was expensive, and access to distribution channels was limited. Only a select few artists ever made it to the airwaves. This was the era of Motown, Prince, and the Jackson 5—a small, elite set of artists who carried enormous weight in defining popular music in the United States and globally.

This is the world of the $\ell_1$ norm. Under $\ell_1$ regularization, each coefficient pays a *flat fee* simply for existing—just as every artist must pass through an expensive and highly selective gatekeeping process. As a result, only a few coefficients "make it big." The rest are driven exactly to zero, left unsigned and unheard. The resulting model is *sparse*: only the most essential features or donor units remain active. Mathematically, if our unpenalized mean squared error (MSE) is 12 for weights $(0.03, 0.9, 12)$ and we apply an $\ell_1$ penalty with $\lambda = 1$, the total loss becomes

$$\text{Loss}_1(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1,$$

where $\|\mathbf{w}\|_1 = \sum |w_i|$ penalizes the sum of the absolute values of the coefficients. For example, with weights $\mathbf{w} = (0.03, 0.9, 12)$, an unpenalized MSE of 12, and $\lambda = 1$, the total loss is:

$$12 + (|0.03| + |0.9| + |12|) = 12 + 0.03 + 0.9 + 12 = 24.93.$$

The large coefficient (12) incurs a hefty penalty, so the optimizer may shrink it or set it to zero unless it significantly improves the fit. Unless it drastically improves the fit, the optimizer will prefer to zero it out entirely—just as only a handful of artists survive the Motown audition process. The $\ell_1$ norm thus rewards exclusivity in variable selection: few stars, high impact.

Consider weights $\mathbf{w} = (0.03, 0.9, 12)$ with an unpenalized MSE of 12. Let's explore two scenarios with different $\lambda$ values. A small $\lambda$ is like a lenient A&R manager, allowing more artists to get a deal. The $\ell_1$ penalty is:

$$\|\mathbf{w}\|_1 = |0.03| + |0.9| + |12| = 0.03 + 0.9 + 12 = 12.93,$$

and the total loss is:

$$\text{Loss}_1(\mathbf{w}) = 12 + 0.1 \cdot 12.93 = 12 + 1.293 = 13.293.$$

The penalty is small, so the optimizer may keep all coefficients, as the cost of including even the large weight (12) is modest. The model remains close to the unpenalized OLS solution, prioritizing fit over sparsity.

A large $\lambda$ is like a ruthless gatekeeper, signing only the biggest stars. The penalty becomes:

$$\|\mathbf{w}\|_1 = 12.93,$$

and the total loss is:

$$\text{Loss}_1(\mathbf{w}) = 12 + 10 \cdot 12.93 = 12 + 129.3 = 141.3.$$

The large penalty, especially for the weight 12, pressures the optimizer to shrink or eliminate coefficients. The optimizer is likely to set smaller weights (e.g., 0.03 and 0.9) to zero and reduce the large weight, favoring a sparse model with only the most essential donor units.

As $\lambda$ becomes very large, the $\ell_1$ penalty term $\lambda\|\mathbf{w}\|_1$ dominates the objective function. To keep the total loss finite, the optimizer must minimize $\|\mathbf{w}\|_1$, driving all coefficients to zero ($\mathbf{w} = \mathbf{0}$). In this limit, the loss becomes:

$$\text{Loss}_1(\mathbf{w}) \approx \|\mathbf{y}_1\|_2^2,$$

since $\mathbf{Y}_0\mathbf{w} = \mathbf{0}$, and the penalty term vanishes ($\|\mathbf{w}\|_1 = 0$). The model predicts $\mathbf{y}_1 \approx \mathbf{0}$, ignoring the data entirely, resulting in high bias but zero variance. In the Motown analogy, this is like an impossibly high entry cost—no artists get signed, and the label produces no music. Such a model is useless for prediction but illustrates how $\ell_1$ enforces extreme sparsity as $\lambda$ grows. The $\ell_1$ norm thus rewards exclusivity: only a few coefficients—our "star artists"—remain non-zero, creating a simpler, more interpretable model that avoids overfitting by focusing on the most essential donor units.

**Problem 8.1. Computing the l1 Norm**

Compute the $\ell_1$ norm of the weight vector $\mathbf{w} = (0.5, -1.2, 3.4, 0)$.

**Problem 8.2. Total Loss with Small lambda**

Suppose the unpenalized MSE is 5 and $\mathbf{w} = (0.5, -1.2, 3.4, 0)$. Compute the total $\ell_1$-penalized loss for $\lambda = 0.1$.

**Problem 8.3. Total Loss with Large lambda**

Using the same weights and unpenalized MSE as in Problem 2, compute the total loss for $\lambda = 10$.

**Problem 8.4. Effect of Increasing lambda**

Explain in one sentence what happens to small coefficients when $\lambda$ increases in $\ell_1$ penalization. Then compute the total loss if $\mathbf{w}$ is shrunk to $(0, 0, 2, 0)$ with $\lambda = 5$ and unpenalized MSE 5.

**Problem 8.5. Extreme Sparsity Limit**

If $\lambda$ becomes extremely large, the optimizer drives all weights to zero. Compute the approximate total loss if $\mathbf{y}_1$ has norm $\|\mathbf{y}_1\|_2^2 = 20$ and $\mathbf{w} = \mathbf{0}$.

### 8.0.2 The Dense World: TikTok and the $\ell_2$ Norm

Now, fast forward to today. In today's music industry, digital production, social media, and streaming platforms have democratized access. Artists from Lagos, Medellín, or London (think Burna Boy and Tems, Karol G, and Ego Ella May) can all reach a global audience in ways that would be impossible even in the 1990s. Distribution of music is *relatively* cheap. Beyond, methods of discovery (like Instagram, SoundCloud, or Spotify) are close to free, open to countless creators. The practical result of this is that we have a dense, inclusive ecosystem where many artists share the spotlight/different sugments of the spotlight. This is the world of the $\ell_2$ norm in regularization. The $\ell_2$ penalty adds a cost to the loss function based on the squared magnitudes of the coefficients:

$$\text{Loss}_2(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2,$$

where $\|\mathbf{w}\|_2^2 = \sum w_i^2$. For example, with weights $\mathbf{w} = (0.03, 0.9, 12)$, an unpenalized MSE of 12, and $\lambda = 1$, the total loss is:

$$12 + (0.03^2 + 0.9^2 + 12^2) = 12 + (0.0009 + 0.81 + 144) = 156.8109.$$

The quadratic penalty scales sharply: small coefficients, like 0.03, contribute almost nothing (0.0009), while large ones, like 12, are heavily penalized (144). This encourages the optimizer to shrink all coefficients rather than eliminate them, keeping every donor unit in the model but reducing their individual impact. The result is a smooth, balanced ensemble—no single unit dominates, mirroring the diverse, inclusive world of modern music streaming.

As above, we can think about what happens as the penality term $\lambda$ increases or decreases. Consider weights $\mathbf{w} = (0.03, 0.9, 12)$ with an unpenalized MSE of 12. Let's examine two scenarios. A small $\lambda$ is like a low streaming platform fee, allowing artists to thrive with minimal restriction. The $\ell_2$ penalty is:

$$\|\mathbf{w}\|_2^2 = 0.03^2 + 0.9^2 + 12^2 = 0.0009 + 0.81 + 144 = 144.8109,$$

and the total loss is:

$$\text{Loss}_2(\mathbf{w}) = 12 + 0.1 \cdot 144.8109 = 12 + 14.48109 = 26.48109.$$

The penalty is moderate, so the optimizer slightly shrinks the coefficients, especially the large one (12), but keeps all non-zero. The model remains close to the OLS solution, prioritizing fit over aggressive shrinkage.

A large $\lambda$ is like a steep tax on popularity, heavily penalizing dominant artists. The penalty remains:

$$\|\mathbf{w}\|_2^2 = 144.8109,$$

but the total loss is:

$$\text{Loss}_2(\mathbf{w}) = 12 + 10 \cdot 144.8109 = 12 + 1448.109 = 1460.109.$$

The large penalty, driven by the $12^2 = 144$ term, pushes the optimizer to significantly shrink all coefficients, especially the largest. The model becomes smoother, with no single donor unit dominating, resembling a diverse playlist where many artists contribute modestly. As $\lambda$ grows very large, the $\ell_2$ penalty term $\lambda\|\mathbf{w}\|_2^2$ dominates the objective. To keep the loss finite, the optimizer minimizes $\|\mathbf{w}\|_2^2$, driving all coefficients toward zero ($\mathbf{w} \to \mathbf{0}$). The loss approaches:

$$\text{Loss}_2(\mathbf{w}) \approx \|\mathbf{y}_1\|_2^2,$$

since $\mathbf{Y}_0\mathbf{w} \to \mathbf{0}$ and the penalty term vanishes ($\|\mathbf{w}\|_2^2 \to 0$). The model predicts $\mathbf{y}_1 \approx \mathbf{0}$, sacrificing fit for extreme simplicity, resulting in high bias but low variance. In the streaming analogy, this is like a platform charging such high fees that no artist can afford to stand out—everyone's presence shrinks to nearly nothing. Unlike $\ell_1$, $\ell_2$ doesn't produce sparsity; coefficients approach zero smoothly, maintaining a balanced but minimal model. The $\ell_2$ norm thus fosters inclusivity: all coefficients—our "global artists"—remain in the model, but their contributions are shrunk, creating a smooth, balanced ensemble that avoids overfitting by distributing influence evenly.

### 8.0.3 The Tradeoff

The contrast between these two regimes mirrors the evolution of music. The $\ell_1$ norm enforces scarcity, pushing only the most valuable features into the spotlight. The $\ell_2$ norm promotes inclusivity, blending many small contributions into a harmonious whole. Both approaches combat overfitting by introducing discipline—one through *selectivity*, the other through

*shrinkage*. In econometric terms, regularization constrains the solution space of $\mathbf{w}$ and prevents the unconstrained OLS estimator from taking extreme values just to eliminate small residuals. The difference lies in how each penalty shapes this constraint: $\ell_2$ encourages smooth shrinkage across all donors, while $\ell_1$ encourages sparsity by zeroing out many coefficients entirely. Both serve the same purpose—to teach our model not to memorize noise, but to focus on structure that generalizes.

**Problem 8.6. Computing the $\ell_2$ Norm Squared**

Compute the squared $\ell_2$ norm of the weight vector $\mathbf{w} = (0.5, -1.2, 3.4, 0)$.

**Problem 8.7. Total Loss with Small $\lambda$**

Suppose the unpenalized MSE is 5 and $\mathbf{w} = (0.5, -1.2, 3.4, 0)$. Compute the total $\ell_2$-penalized loss for $\lambda = 0.1$.

**Problem 8.8. Total Loss with Large $\lambda$**

Using the same weights and unpenalized MSE as in Problem 2, compute the total loss for $\lambda = 10$.

**Problem 8.9. Effect of Increasing $\lambda$**

Explain in one sentence how increasing $\lambda$ affects large coefficients in $\ell_2$ penalization. Then compute the total loss if $\mathbf{w}$ is shrunk to $(0.1, -0.2, 0.5, 0)$ with $\lambda = 5$ and unpenalized MSE 5.

**Problem 8.10. Extreme Shrinkage Limit**

If $\lambda$ becomes extremely large, the optimizer drives all weights toward zero. Compute the approximate total loss if $\|\mathbf{y}_1\|_2^2 = 20$ and $\mathbf{w} \approx \mathbf{0}$.

## 8.1 Cross Validation

Whether we live in the sparse Motown world of $\ell_1$ or the crowded TikTok ecosystem of $\ell_2$, we still face one final act of judgment: how much selectivity is too much? If the label signs too few artists, we miss out on talent; if it signs everyone, we drown in noise. In statistical terms, this question is about choosing the right value of $\lambda$. Recall that the basic penalized model was

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_p^p,$$

where we have a lambda term which chooses the strength of the norm that we select. Thus, a crucial aspect of penalized synthetic control methods is the choice of the regularization

parameter, lambda. For the $\ell_1$ norm, if lambda is too small, the method behaves much like OLS, producing many non-zero weights and potentially overfitting the pre-treatment period. If lambda is too large, most weights are shrunk toward zero, resulting in an overly sparse synthetic control that may underfit the treated unit. For the $\ell_2$, a small lambda allows weights to take extreme values, again risking overfitting, while a large lambda shrinks all weights toward zero, distributing influence more evenly but potentially introducing bias in the post-treatment counterfactual. To do this, we employ cross validation. Cross validation is used to select hyperparameters for machine-learning models.

**Definition 8.1** (Cross Validation)**.** Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a dataset of $n$ labeled observations and let $\mathcal{A}$ be a learning algorithm producing a model $\hat{f}$ given training data. Cross validation is a procedure to estimate the out-of-sample prediction error of $\hat{f}$ by partitioning $\mathcal{D}$ into $K$ disjoint folds $\mathcal{D}_1, \ldots, \mathcal{D}_K$. For each fold $k = 1, \ldots, K$, we train the model on the complement $\mathcal{D} \setminus \mathcal{D}_k$ and evaluate it on the holdout fold $\mathcal{D}_k$, producing a validation error $\mathrm{Err}^{(k)}$. The cross-validated error is the arithmetic average over the folds:

$$\mathrm{CV}(\mathcal{A}, \mathcal{D}) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{Err}^{(k)}.$$

In our case we wish to select $\lambda$. The idea is simple: for a given section of the pre-treatment period and a candidate lambda, we fit the penalized least-squares model and compute the prediction error on held-out pre-treatment data. By repeating this process across multiple folds or rolling windows and averaging the validation errors, we identify the lambda that minimizes out-of-sample pre-treatment error, balancing the bias-variance tradeoff for more reliable post-treatment counterfactual predictions. Let $\mathbf{y}_1 \in \mathbb{R}^{T_0}$ be the vector of pre-treatment outcomes for the treated unit, and $\mathbf{Y}_0 \in \mathbb{R}^{T_0 \times N_0}$ the corresponding donor matrix. Define the rolling-origin splits of the pre-treatment period. For each window $k = 1, \ldots, K$, let

$$\mathbf{y}_{1,\mathrm{train}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{train}}^{(k)}|}, \quad \mathbf{Y}_{0,\mathrm{train}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{train}}^{(k)}| \times N_0}$$

$$\mathbf{y}_{1,\mathrm{val}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{val}}^{(k)}|}, \quad \mathbf{Y}_{0,\mathrm{val}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{val}}^{(k)}| \times N_0}$$

be the training and validation vectors/matrices for that window. For a given norm $p \in \{1, 2\}$ and regularization $\lambda$, the penalized weights for the $k$-th training window are

$$\mathbf{w}^{(k)}(\lambda) = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_{1,\mathrm{train}}^{(k)} - \mathbf{Y}_{0,\mathrm{train}}^{(k)} \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_p^q$$

where $q = 1$ for LASSO and $q = 2$ for Ridge. The validation MSE for this window is

$$\mathrm{MSE}^{(k)}(\lambda) = \frac{1}{|\mathcal{T}_{\mathrm{val}}^{(k)}|} \|\mathbf{y}_{1,\mathrm{val}}^{(k)} - \mathbf{Y}_{0,\mathrm{val}}^{(k)} \mathbf{w}^{(k)}(\lambda)\|_2^2$$

The cross-validated MSE for each $\lambda$ is the average across all rolling windows:

$$\overline{\text{MSE}}(\lambda) = \frac{1}{K}\sum_{k=1}^{K} \text{MSE}^{(k)}(\lambda)$$

The optimal regularization parameter is

$$\lambda^* = \underset{\lambda \in \{\lambda_1,\ldots,\lambda_L\}}{\text{argmin}} \quad \overline{\text{MSE}}(\lambda)$$

Finally, the weights for the full pre-treatment period using $\lambda^*$ are

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\text{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda^* \|\mathbf{w}\|_p^q$$

We can even visualize this. For example, suppose we had pre-treatement data from 1900 to 1975. **?@fig-rocvex** plots the folds of "training" and validation.

Many forms of cross validation exist, and users can choose the ones they feel are most important for their purposes.

**Problem 8.11. Computing Pre-Treatment RMSE**

Given `y_pre = [2, 3, 5]` and `y_hat_pre = [2.1, 1.9, 4.8]`, compute the pre-treatment RMSE.

**Problem 8.12. Comparing LASSO and Ridge Effects**

Explain in one sentence why LASSO produces sparse weights while Ridge produces more evenly distributed weights. Then, for weights $\mathbf{w} = [0.5, 0.5, 0.5, 0.5]$ and $\lambda = 0.2$, compute the $\ell_1$ and $\ell_2$ penalties.

**Problem 8.13. Conceptual Understanding of Lambda**

Explain in your own words why choosing a very small $\lambda$ or a very large $\lambda$ can lead to poor out-of-sample performance for LASSO and Ridge. Use the bias-variance tradeoff in your explanation.

**Problem 8.14. Rolling-Origin Window Creation**

For pre-treatment years 1900–1910, create three rolling-origin windows with `initial_train = 5` and `step = 2`. List the training and validation indices for each fold.

**Problem 8.15. Custom Cross Validation Routine**

Write a short Python function that takes a donor matrix $Y_0$, treated vector $y$, and a list of candidate $\lambda$ values, and performs $K$-fold cross validation to select the best $\lambda$ for LASSO. You may use `cvxpy` for optimization.

**Problem 8.16. Interpreting CV Results**

After cross validation, suppose $\lambda^* = 0.3$ for LASSO and $\lambda^* = 0.1$ for Ridge. Sketch or describe how you expect the post-treatment counterfactual predictions to differ between these two models. Which would be sparser and why?

## 8.2 Implementing Penalized OLS Models

Okay, now that we have a good sense of what these models do conceptually, let's work on implementing this in code. At this point, we will develop more structured forms of script to accomplish our tasks. Consider the function below:

```
l2results = Opt.SCopt(
    num_control_units=Y0.shape[1],
    target_outcomes_pre_treatment=y[:T0],
    num_pre_treatment_periods=T0,
    donor_outcomes_pre_treatment=Y0,
    scm_model_type="OLS",
    donor_names=donor_names,
    lambda_penalty=0.2,
    p=2,
    q=2,
)
```

This is the `Opt.SCopt` backend. It is buried within the lightless depths of the `mlsynth` API. This class provides a unified way to estimate penalized regression models using unconstrained OLS and penalized least-squared models such as the Lasso and Ridge penalties. producing the donor weights, the predicted counterfactual trajectory for the treated unit, and additional information such as pre-treatment RMSE and a mapping of donor names to weights. The key inputs include the donor matrix `Y0` (columns correspond to control units, rows to time periods), the treated series `y` to construct a counterfactual for, and `T0`, the number of pre-treatment periods. Optional inputs such as `donor_names` make the resulting weights easier to interpret, while `lambda_penalty` sets the strength of the penalty, and `p` and `q` determine the type of regularization: `p=1` for Lasso, `p=2` for Ridge. The function outputs a structured object containing the estimated numeric weights, predicted synthetic series, a dictionary mapping donor names to weights, and the pre-treatment RMSE. Using this approach, we can directly compare OLS and penalized synthetic control estimates in a consistent framework, while ensuring that each donor's contribution is appropriately weighted. Given that we have the ability to fit penalized synthetic controls with `Opt.SCopt`, we can choose the optimal penalty parameter $\lambda$ using cross-validation on the pre-treatment period. The `fit_scopt_cv` function automates this process.

```
results = fit_scopt_cv(Y0, y, T0, donor_names=donor_names, p_list=[1,2])
```

The function takes the full donor matrix `Y0`, the treated series `y`, and `T0`, the number of pre-treatment periods. Optional inputs include `donor_names` for easier interpretation, and `p_list` to indicate which penalization norms to fit: 1 for Lasso and 2 for Ridge. For each penalized model, the function tests a grid of $\lambda$ values and evaluates performance on held-out pre-treatment periods. The $\lambda$ that minimizes the average root mean squared error (RMSE) is selected as optimal. After choosing the best $\lambda$, the function fits the full synthetic control series for the treated unit, producing the numeric weights, the counterfactual predictions, a mapping of donor names to weights, and the pre-treatment RMSE for each method. OLS can also be included alongside penalized models for comparison.

The pre-treatment RMSE for the LASSO with $\lambda = 0.2$ is 0.083, for Ridge with $\lambda = 0.2$ is 0.065, and for OLS is 0.002. For the LASSO, the weights are very sparse. Cataluna contributes the most with 0.757, followed by Madrid at 0.190 and Principado de Asturias at 0.066. All other donors have weights effectively zero, reflecting the L1 penalty's tendency to select only a few contributors. For Ridge, the weights are more evenly distributed across donors. The largest contributions come from Madrid at 0.299, Cataluna at 0.239, Principado de Asturias at 0.247, Rioja at 0.202, and Cantabria at 0.172. Other donors have moderate positive or negative weights, showing the L2 penalty spreads influence across many donors rather than selecting only a few. OLS produces more extreme weights, with Castilla Y Leon at 6.093, Andalucia at 4.177, and Castilla-La Mancha at -2.655, while most other donors have smaller positive or negative contributions. From these results, we can clearly see that OLS happens to fit the best out of all of these, but the weights are much different when we look across them. This is most obvious when we plot the counterfactual as we do in **?@fig-penlsest**.

The out of sample estimates are much smoother than the OLS predictions which have much higher variance.

Unlike the positive effect suggested by OLS, the penalized estimators tell a very different story. Both LASSO and Ridge produce negative ATT estimates: LASSO at -671 and Ridge at -758. This indicates that once we shrink or regularize the donor weights, the synthetic control tracks the treated unit more cautiously, and the apparent "boost" disappears, even turning negative. In machine learning terms, this reflects the bias-variance tradeoff: we accept slightly more bias in the pre-treatment fit if it reduces the variance of out-of-sample predictions, leading to more reliable post-treatment estimates.

## 8.3 Principal Components Regression

In the previous section, we used penalized regression to shrink or select donor weights. An alternative approach to control overfitting is Principal Component Regression (PCR) introduced by Amjad et al. (2018) and Agarwal et al. (2021). Principal Component Analysis is based on the idea that large datasets may be well approximated by only a few key dominant patterns. Now, imagine the outcome matrix of controls as a literal cloud of points whose

shape we wish to explain using as few dimensions as possible, capturing the essence of the data without reproducing every detail.

We already know the idea of fitting a line of best fit in two dimensions: it's just a line that summarizes the linear relationship among the points. PCA is the natural extension of that idea to higher dimensions. Instead of one line, we now look for a few directions (principal components) through this cloud of points that capture the most variance. We have as many principal components as we have donor units. Each principal component is like a stiff line through the data that explains as much of the spread as possible. Once we've identified the top few directions, we can project the donor points onto these lines. This gives us a low-dimensional summary that preserves most of the original structure. Reconstructing the donor series from these projections is just like asking: "If I only knew the points' positions along these main lines, what would the original points look like?" The reconstruction is never perfect, but with enough top components, it usually captures the bulk of the pattern, filtering out minor fluctuations or noise.

### 8.3.1 Singular Value Decomposition

We can formalize the idea of projecting onto dominant patterns using Singular Value Decomposition (SVD). SVD decomposes the centered matrix $\mathbf{Y}_0 - \bar{\mathbf{Y}}_0$ as

$$\mathbf{Y}_0 - \bar{\mathbf{Y}}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U} \in \mathbb{R}^{T_0 \times T_0}$ and $\mathbf{V} \in \mathbb{R}^{N_0 \times N_0}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{T_0 \times N_0}$ is diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Centering ensures PCA focuses on the variation around the mean, not the absolute values. The columns of $\mathbf{V}$ define the principal components, i.e., directions through the "cloud of donor points" that explain maximal variance. Projecting the donor series onto the top $K$ components,

$$\mathbf{Z}_K = (\mathbf{Y}_0 - \bar{\mathbf{Y}}_0)\mathbf{V}_K, \quad \mathbf{V}_K \in \mathbb{R}^{N_0 \times K},$$

provides a low-dimensional summary of the donor behavior. Each row of $\mathbf{Z}_K$ gives the coordinates of a donor in this $K$-dimensional subspace—analogous to the projected coordinates along a line of best fit in 2D. The original donor matrix can then be approximately reconstructed as

$$\hat{\mathbf{Y}}_0^{(K)} = \mathbf{Z}_K \mathbf{V}_K^\top + \bar{\mathbf{Y}}_0,$$

where $\hat{\mathbf{Y}}_0^{(K)}$ captures the bulk of the variation in the donors while filtering out minor fluctuations or noise. This is entirely analogous to fitting a line of best fit in two dimensions: here, instead of one line, we have a few stiff directions through the high-dimensional cloud of points.

To visualize this in 2D, we load the Basque data and slice the donor matrix off by its pre-intervention period (before 1975).

Next, we fit the principal components. I do this both in SVD and PCA, but either way is correct:

We can see that while we have 16 donor units at hand, we only need 4 or 5 components to explain 99.23 percent of the variation in the donors. The colorful lines are the values taken by the reconstructed donor matrix, and this is what we use to fit the target series that is treated.

### 8.3.2 Singular Value Thresholding

To select K, we use procedure called Universal Singular Value Thresholding developed by Chatterjee (2015). USVT works by analyzing the singular values from the SVD of the centered donor matrix. Recall that our singular values measure the importance of each principal component, with larger values indicating directions that capture more variance. USVT selects $K$ by setting a threshold to distinguish significant singular values (signal) from smaller ones likely representing noise. To choose $K$ with USVT, we first calculate the matrix's aspect ratio, defined as the ratio of its smaller dimension to its larger dimension, i.e.,

$$r = \frac{\min(T_0, N_0)}{\max(T_0, N_0)},$$

where $T_0$ is the number of pre-treatment periods and $N_0$ is the number of donor units. For the Basque data, the pre-treatment donor matrix has shape $20 \times 16$ (20 time periods, 16 donors), so the aspect ratio is $16/20 = 0.8$. Next, USVT computes a multiplier, called the omega factor, based on the aspect ratio using the formula:

$$\omega = 0.56 \cdot r^3 - 0.95 \cdot r^2 + 1.82 \cdot r + 1.43$$

Essentially, we adjust the threshold to account for how the matrix's shape affects the distribution of singular values. For our Basque data with an aspect ratio of 0.8, we calculate:

$$\omega = 0.56 \cdot (0.8)^3 - 0.95 \cdot (0.8)^2 + 1.82 \cdot 0.8 + 1.43 \approx 2.584$$

The omega factor ensures the threshold is appropriately scaled for matrices of different shapes (e.g., tall and thin vs. wide and short). The threshold for retaining singular values is then set as:

$$\text{threshold} = \omega \cdot \text{median}(\sigma_1, \sigma_2, \ldots, \sigma_{\min(T_0, N_0)})$$

The median singular value is used because it provides a robust measure of the typical scale of singular values, less sensitive to outliers than the mean. Singular values above this threshold are considered significant, capturing the main patterns in the data, while those below are treated as noise. The number of principal components, $K$, is the count of singular values greater than the threshold:

$$K = \text{number of } \sigma_i \text{ where } \sigma_i > \text{threshold}$$

Once the rank $K$ has been selected, we rebuild the donor matrix using only the top $K$ singular components. Recall that the centered donor matrix can be decomposed by Singular Value Decomposition (SVD) as

$$\mathbf{Y}_0 - \bar{\mathbf{Y}}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U} \in \mathbb{R}^{T_0 \times T_0}$ and $\mathbf{V} \in \mathbb{R}^{N_0 \times N_0}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{T_0 \times N_0}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. To obtain a low-rank approximation that captures the main structure of $\mathbf{Y}_0$, we retain only the first $K$ singular values and their corresponding singular vectors:

$$\mathbf{U}_K = [\mathbf{u}_1, \ldots, \mathbf{u}_K], \quad \mathbf{\Sigma}_K = \text{diag}(\sigma_1, \ldots, \sigma_K), \quad \mathbf{V}_K = [\mathbf{v}_1, \ldots, \mathbf{v}_K].$$

The low-rank reconstruction of the donor matrix is then given by

$$\mathbf{Y}_0^{(K)} = \mathbf{U}_K \mathbf{\Sigma}_K \mathbf{V}_K^\top + \bar{\mathbf{Y}}_0.$$

This reconstruction, $\mathbf{Y}_0^{(K)}$, preserves the dominant co-movements across donors while filtering out minor, idiosyncratic variations. In practice, this means that each donor's trajectory is projected into a $K$-dimensional subspace that captures the most important temporal and cross-sectional patterns.

Now, we can perform linear regression. Like the above, our target vector is the pre-treatment outcome of the treated unit. However, instead of the raw donor matrix as predictors, we use the denoised version of the donor matrix:

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\text{argmin}} \, \|\mathbf{y}_1 - \mathbf{Y}_0^{(K)}\mathbf{w}\|_2^2 + \lambda \, \|\mathbf{w}\|_p^q.$$

Here the target vector is the outcomes of the treated unit and $\mathbf{Y}_0^{(K)}$ is the denoised donor matrix. Because this is just a form of OLS, we can also impose the standard penalties we have discussed above to the PCR setting. To implement this, we use the `CLUSTERSC` class from `mlsynth`. This method calls on the `pcr` method, which itself calls on the `Opt.SCopt` class. Below, I run unregularized PCR (which I just refer to as OLS) and PCR with an $\ell_1$ and $\ell_2$ penalty on the weights. I use a lambda of 0.05 for both. Below, here are the weights.

```
from mlsynth import CLUSTERSC
import pandas as pd

# Load Basque dataset
url = (
```

```
    "https://raw.githubusercontent.com/"
    "jgreathouse9/mlsynth/"
    "refs/heads/main/basedata/"
    "basque_data.csv"
)
data = pd.read_csv(url)

# Base configuration
base_config = {
    "df": data,
    "outcome": data.columns[2],
    "treat": data.columns[-1],
    "unitid": data.columns[0],
    "time": data.columns[1],
    "display_graphs": False,
    "save": False,
    "counterfactual_color": ["red"],
    "cluster": False,
    "method": "PCR"
}

# Define model setups
model_setups = {
    "OLS": {"lambda_penalty": 0, "p": 1, "q": 1},
    "Lasso": {"lambda_penalty": 0.05, "p": 1, "q": 1},
    "Ridge": {"lambda_penalty": 0.05, "p": 2, "q": 2}
}

# Containers for results
weight_dict = {}
counterfactual_dict = {}
att_dict = {}
rmse_dict = {}

# Fit each model
for name, params in model_setups.items():
    config = base_config.copy()
    config.update(params)

    model = CLUSTERSC(config)
    arco = model.fit()

    weights = arco.sub_method_results["PCR"].weights.donor_weights
    counterfactual = arco.sub_method_results["PCR"].time_series.counterfactual_outcome
```

```
    weight_dict[name] = weights
    counterfactual_dict[name] = counterfactual

    att_dict[name] = arco.sub_method_results["PCR"].effects.additional_effects.get("ATT"
    rmse_dict[name] = arco.sub_method_results["PCR"].fit_diagnostics.additional_metrics.


# Convert each donor_weights dict to a Series and round to 3 decimals
weights_df = pd.DataFrame({
    name: pd.Series({k: round(v, 3) for k, v in weights.items()})
    for name, weights in weight_dict.items()
})


display(Markdown(weights_df.to_markdown()))
```

These weights return these predictions:

along with these ATTs and RMSEs for each penalized SC method.

We can see that the PCR approach effectively regularizes the OLS fit compared to the non-PCR OLS fit. Although OLS in theory allows weights to freely adapt to noise, PCR implicitly regularizes the solution by expressing $\mathbf{Y}_0$ only in terms of its dominant components. This effectively smooths the fit, as high-variance, low-signal directions are penalized. This has the effect of distributing weight across donors that explain the largest share of GDP variation in the pre-treatment period, while downweighting minor, noisy fluctuations. As a result, the synthetic Basque GDP is smoother, more stable, and driven by the main patterns in the donor pool rather than overfitting to idiosyncratic deviations.

Looking at the actual donor weights, we see that OLS assigns weight to multiple regions, including Cataluna (0.264) and Madrid (0.277), but also some negative weights such as Canarias (-0.218). Ridge regularization keeps a similar distribution but slightly shrinks extreme values toward zero, producing a smoother set of weights. Lasso, on the other hand, sparsifies the solution dramatically, assigning nearly all the weight to Cataluna (0.867) and Madrid (0.134) while zeroing out contributions from most other donors.

These weight patterns are reflected in the treatment effect estimates and pre-treatment RMSE. OLS produces an ATT of -0.849 with an RMSE of 0.077, indicating tight pre-treatment fit. The regularized solutions give very similar results, with slight differences in fit and ATT. Ridge gives a slightly smaller ATT compared to OLS (-0.826) with effectively the same RMSE, maintaining stability while smoothing extreme weights. Lasso, by concentrating weight on a few donors, produces a smaller ATT in magnitude (-0.732) and slightly higher pre-treatment RMSE (0.083), reflecting a trade-off between sparsity and fit. Overall, regularization allows the PCR approach to balance fidelity to the pre-treatment data with stability and interpretability of the donor weights.

## 8.4 Downsides of Penalized Least Squares

Implementing penalized least squares requires several practical tuning decisions. We must choose the size of the candidate $\lambda$ grid. A coarse grid may miss the optimal $\lambda$, while an overly fine grid increases computational cost without guaranteed improvement. The grid spacing is also critical. Analysts can linearly, logarithmically, or geometrically space lambda values. This choice also matters. A logarithmic scale can capture behavior when small changes in $\lambda$ have large effects, whereas a linear scale may suffice when sensitivity is low.

Beyond the choices, another factor is how these methods influence the interpretation of the decisions. LASSO's sparsity can simplify interpretation by selecting a few donors, but the choice of which donors remain can be sensitive to small changes in the data. Ridge distributes weights more evenly, making the model stable, but at the cost of obscuring which units are most influential. In practice, both penalties can shift the relative importance of donors in ways that might not align with substantive knowledge. Penalized models also rely on pre-treatment fit to guide the penalty. If the pre-treatment period is short, noisy, or unrepresentative, the regularization can either overcorrect or undercorrect, affecting post-treatment predictions.

These challenges do not go away for PCR either. PCR requires choosing how many top singular values to retain. USVT is one way of doing this, but other methods have been proposed in the literature as well. For example, Gavish & Donoho (2014) find the optimal threshold to be $\frac{4}{\sqrt{3}}$ (also see Donoho et al., 2023). The practical consequences of this are simple: keeping too few components may miss important variation in the donor pool, underfitting the treated unit. Keeping too many may reintroduce noise, overfitting minor idiosyncrasies. There is no universally correct rule, and analysts may experiment according to different rules as per their context.

## 8.5 Onward to Convex SC

Regularization teaches OLS restraint. By adding a penalty term, we give the model a sense of proportion: fit matters, but so does humility. The $\ell_1$ and $\ell_2$ norms are two ways of communicating that discipline—one selective, the other inclusive. Both make OLS behave less like an overzealous mimic and more like a thoughtful imitator. Methods such as PCR allow us to exploit data-compression techniques in order to do counterfactual prediction. Yet, penalized methods still leave the model free to misbehave, for the reasons I mentioned above. To address this, we may wish to move beyond soft constraints upon the weights to harsher constraints with stricter rules to follow. This is the subject for the next chapter. Geometrically, we go from a soft, round penalty landscape to a sharp, polygonal constraint space. Conceptually, we shift from telling OLS to "behave" to explicitly defining what "behavior" is allowed.

**Problem 8.17. Analogy**

Come up with your own analogy to explain the $\ell_1$ and $\ell_2$ norms, to accompany my music analogy.

**Problem 8.18. Understanding Overfitting**

1. Define overfitting in the context of synthetic control.
2. Explain why OLS synthetic control tends to overfit the pre-treatment period in the Basque Country example.
3. How does overfitting affect out-of-sample (post-treatment) predictions?

**Problem 8.19. L1 vs L2 Regularization**

1. For the same dataset, fit both LASSO (L1) and Ridge (L2) penalized regressions.
2. Compare the donor weights obtained from each method. Which donors are selected or shrunk more under each norm?
3. Explain how the choice of $\lambda$ affects the sparsity of the LASSO weights and the smoothness of Ridge weights.

**Problem 8.20. Bias-Variance Tradeoff**

1. Explain the bias-variance tradeoff in your own words.
2. Using your OLS, LASSO, and Ridge results, describe which models have higher bias or higher variance.
3. How does regularization help balance this tradeoff in synthetic control?

**Problem 8.21. Implementing Penalized OLS**

1. Using the `fit_penalized` function, compute penalized weights for the Basque dataset with $\lambda = 0.1, 0.5, 1.0$.
2. Plot the predicted counterfactuals for each $\lambda$.
3. Discuss how increasing $\lambda$ affects the pre-treatment RMSE and the donor weight distribution.

**Problem 8.22. Cross-Validation Setup**

1. Explain the purpose of rolling-origin cross-validation in the context of synthetic control.

2. Define training and validation periods for a rolling window with initial train size 10 and step 5.

3. Why might a logarithmic grid of $\lambda$ values be preferable to a linear grid in CV?

**Problem 8.23. Selecting Optimal $\lambda$**

1. Perform rolling-origin cross-validation to select the optimal $\lambda$ for LASSO and Ridge.

2. Plot the cross-validated MSE against $\lambda$ for each norm.

3. Report the $\lambda$ that minimizes CV error and explain why it represents a balance between bias and variance.

**Problem 8.24. ATT Estimation**

1. Compute the average post-treatment treatment effect (ATT) for OLS, LASSO, and Ridge synthetic controls.

2. Compare the magnitude and sign of the ATT estimates across models.

3. Interpret the differences in the context of penalization and overfitting.

**Problem 8.25. Hyperparameter Sensitivity**

1. Explore how changing the size of the $\lambda$ grid affects the CV-selected $\lambda$.

2. Try linear, logarithmic, and geometric grids.
3. Discuss how grid choice could influence your synthetic control results.

# Chapter 9

# Convex Synthetic Controls

> "People don't need to wear the same shoes; they should find what suit their feet."
> — Wei Yuan

Having discussed the pitfalls of DID in the small-$N$ setting, we saw its validity hinges on the Parallel Pre-Trends Assumption: the expected difference between treated and control units must remain constant before treatment. When this assumption fails, DID can produce biased estimates. One alternative is OLS, which constructs counterfactuals by flexibly matching pre-treatment outcomes. While OLS can achieve near-perfect pre-intervention fit, it allows unconstrained weights that may extrapolate far outside the support of the donor pool, producing counterfactuals with implausibly high variance. Regularization using $\ell_1$ and $\ell_2$ norms or PCR weights can shrink extreme weights, but we might prefer a structural constraint that ensures the counterfactual remains within the range of the donor units. This insight motivates the SCM introduced by Abadie et al. (2010), which constructs a weighted combination of donor units that closely approximates the treated unit's pre-treatment trajectory while respecting convexity constraints. This chapter presents the canonical SCM, the one taught in graduate school for econ and public policy students.

## 9.1 Convex Least Squares

For the classic SCM, we return to a familiar setting where we yet again solve another least squares problem. However, this problem is different from all of the other ones that we have seen before:

$$\mathbf{w}^{\text{SCM}} = \underset{\mathbf{w}^{\text{SCM}} \in \mathcal{W}_{\text{conv}}}{\operatorname{argmin}} \left\| \mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}^{\text{SCM}} \right\|_2^2 \quad \forall\, t \in \mathcal{T}_1, \tag{9.1}$$

$$\mathcal{W}_{\text{conv}} \in \left\{ \mathbf{w}^{\text{SCM}} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \right\}. \tag{9.2}$$

At first glance, this looks like yet another least squares problem — and in a purely algebraic sense, it is. The difference lies not in the objective but in the *constraints*. In OLS and

penalized variants, the weights are completely unrestricted: they can theoretically take any real value, positive or negative. This freedom allows the model to "extrapolate" — to combine donor units in ways that generate counterfactuals lying *outside* the convex hull of observed outcomes. In contrast, the convex synthetic control imposes two structural restrictions: $w_j \geq 0$ and $\sum_j w_j = 1$. No donor is allowed to "subtract" from the synthetic control — we can only form additive mixtures of actual units, and the weights must form a convex combination, ensuring that the synthetic unit lies within the convex hull of the donor pool.

These two constraints completely change the character of the problem. Rather than searching the entire vector space of possible weights, SCM confines the solution to a simplex/convex hull — a compact, convex region of feasible combinations. Because $\mathcal{W}_{\text{conv}}$ is a convex set, the solution $\mathbf{w}^{\text{SCM}}$ is a convex combination of the donor units. This implies that the SCM prediction

$$\hat{\mathbf{y}}_1^{\text{SCM}} = \mathbf{Y}_0 \mathbf{w}^{\text{SCM}}$$

must lie within the convex hull of the donor outcomes. Consider the donor outcome vectors $(\mathbf{y}_j)_{j \in \mathcal{N}_0} \in \mathbb{R}^{T_0}$. What this means is that the prediction (or, the weighted average) lies inside the convex hull of the donor outcomes. Geometrically, this means that if we imagine the donor outcomes as points in a space, the synthetic control must live inside the polygon (or polyhedron, in higher dimensions) formed by those points. A simple way to see this is to start with a one-dimensional example: if we have two donors with pre-treatment averages of 3 and 7, then any convex combination of them—say, $0.25 \times 3 + 0.75\ddot{O}7 = 6$, will fall somewhere on the line segment connecting 3 and 7.

Consider any points within this line segment: there is no way to get outside that line segment unless we allow negative weights or weights that sum to more than one. We can do this in two dimensions too:

We can see that the space of predictions is a combination of the control units. Here, there are an infinte number of solutions that we can use for the donors to solve for the target weight vector. As we move to higher dimensions, this idea generalizes: the convex hull is simply the set of all mixtures of donors where the weights are nonnegative and sum to one. We can understand the convex hull idea better by projecting them onto a *two-dimensional plane*, per **?@fig-donor-span**.

This plot shows the Basque Country within the convex hull of donor units. If we were to assign any donor a weight of 1 and the rest 0, the predicted synthetic control would just be that control unit. But because one control unit will likely be insufficient to minimize the pre-treatment residual, we oftentimes rely on more donor units. One of the things I struggled with while learning SCM is the reasons why the convex hull constraint mattered. I did not know what it did or why, I just heard it was important for identification. So, to properly illustrate this, below we will derive the SCM weights for a corner solution.

**Problem 9.1. Parallel Trends and SCM**

Explain why the standard Difference-in-Differences (DID) assumption of parallel pre-Assumption is problematic in small-$N$ settings, and how SCM addresses this issue. In particular, discuss their weighting schemes and what this implies.

**Problem 9.2. Convex Hull**

Define the convex hull of the donor pool outcomes in your own words. Why might constraining the synthetic control weights to this hull be important?

**Problem 9.3. Weight Constraints**

Consider the SCM weight set

$$\mathcal{W}_{\text{conv}} \in \left\{ \mathbf{w}^{\text{SCM}} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \right\}.$$

Explain the interpretation of the constraint $\sum_j w_j = 1$ and $w_j \geq 0$. What does the summation to 1 suggest? The nonnegativity?

**Problem 9.4. OLS vs SCM**

Describe a scenario where unconstrained OLS would assign negative or extreme weights to donor units. How does SCM prevent this?

### 9.1.1 A Simple Synthetic Control

Above we say how in very simple settings when we have two time periods and one donor that lies within the span of the donor outcomes, we will have an infinite number of solutions. But, what if we could only have one possible solution? Suppose we have a treated unit and two donor units in 2D:

$$\mathbf{y} = \begin{bmatrix} 50 \\ 60 \end{bmatrix}, \qquad \mathbf{Y}_0 = [\mathbf{y}_1 \; \mathbf{y}_2] = \begin{bmatrix} 20 & 48 \\ 22 & 50 \end{bmatrix}.$$

The first of these units is treated, and the latter two are the control units. Graphically, the plot of their trajectories looks like

We aim to find a synthetic control as a convex combination of the donor units that matches the target vector as closely as possible. Again, because of the fact that the feasible weight set is a convex set, this means a tradeoff is at work: where we increase one weight for one donor, this necessarily reduces the weight the other one gets. Exploiting this fact, this allows us to define one weight *in terms of* the other one. To do this, we need the difference vector:

$$\mathbf{y}_1 - \mathbf{y}_2 = \begin{bmatrix} 20 \\ 22 \end{bmatrix} - \begin{bmatrix} 48 \\ 50 \end{bmatrix}.$$

Perform the subtraction componentwise:

$$(\mathbf{y}_1 - \mathbf{y}_2)_1 = 20 - 48 = -28, \tag{9.3}$$
$$(\mathbf{y}_1 - \mathbf{y}_2)_2 = 22 - 50 = -28. \tag{9.4}$$

Using this, we can write the synthetic prediction as a function of $w_1$ (the weight on donor 1):

$$\hat{\mathbf{y}}(w_1) = \mathbf{y}_2 + w_1(\mathbf{y}_1 - \mathbf{y}_2) = \begin{bmatrix}48\\50\end{bmatrix} + w_1 \begin{bmatrix}-28\\-28\end{bmatrix} = \begin{bmatrix}48 - 28w_1\\50 - 28w_1\end{bmatrix}.$$

Finally, the residual vector, or the difference between the treated unit and the in-sample prediction, is

$$\mathbf{r}(w_1) = \mathbf{y} - \hat{\mathbf{y}}(w_1) = \begin{bmatrix}50\\60\end{bmatrix} - \begin{bmatrix}48 - 28w_1\\50 - 28w_1\end{bmatrix} = \begin{bmatrix}2 + 28w_1\\10 + 28w_1\end{bmatrix}.$$

This sets up the next step: minimizing the squared norm of the residual vector to find the optimal weight $w_1$. Minimizing the $\ell_2$ norm gives us the following optimization problem:

$$w_1^* = \underset{w_1 \in \mathbb{R}}{\operatorname{argmin}} \, \|\mathbf{r}(w_1)\|_2^2 \equiv \underset{w_1 \in \mathbb{R}}{\operatorname{argmin}} \, \Big[(2 + 28w_1)^2 + (10 + 28w_1)^2\Big].$$

To find the minimizing weight of this convex function, we differentiate $f(w_1)$ with respect to $w_1$. Since $f(w_1)$ is a sum of two terms, the sum rule allows us to differentiate each term separately and then sum the results:

$$f(w_1) = (2 + 28w_1)^2 + (10 + 28w_1)^2$$

Let's begin with the first term, $(2 + 28w_1)^2$. This is a composition of two functions: a linear inner function and a quadratic outer function. To differentiate, we apply the chain rule. Let

$$h_1(w_1) = 2 + 28w_1 \quad \text{(inner function)}, \qquad g_1(u) = u^2 \quad \text{(outer function)}.$$

The derivatives are

$$\frac{\mathrm{d}g_1}{\mathrm{d}u} = 2u, \quad \frac{\mathrm{d}h_1}{\mathrm{d}w_1} = 28.$$

Applying the chain rule gives

$$\frac{\mathrm{d}}{\mathrm{d}w_1}(2 + 28w_1)^2 = \left.\frac{\mathrm{d}g_1}{\mathrm{d}u}\right|_{u=h_1(w_1)} \cdot \frac{\mathrm{d}h_1}{\mathrm{d}w_1} = 2(2 + 28w_1) \cdot 28.$$

Next, consider the second term, $(10 + 28w_1)^2$. Let

$$h_2(w_1) = 10 + 28w_1 \quad \text{(inner function)}, \qquad g_2(v) = v^2 \quad \text{(outer function)}.$$

The derivatives are

$$\frac{\mathrm{d}g_2}{\mathrm{d}v} = 2v, \quad \frac{\mathrm{d}h_2}{\mathrm{d}w_1} = 28.$$

Applying the chain rule gives the derivative of the second term:

$$\frac{\mathrm{d}}{\mathrm{d}w_1}(10 + 28w_1)^2 = 2(10 + 28w_1) \cdot 28.$$

By the sum rule we have:

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 2(2 + 28w_1) \cdot 28 + 2(10 + 28w_1) \cdot 28.$$

which just simplifies to

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56(2 + 28w_1) + 56(10 + 28w_1).$$

Now we factor out the 56

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56\Big((2 + 28w_1) + (10 + 28w_1)\Big)$$

and then combine like terms inside of the parentheses

$$(2 + 28w_1) + (10 + 28w_1) = 12 + 56w_1,$$

and that gives us this result:

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56(12 + 56w_1).$$

We can now distribute the 56 $56 \cdot 12 + 56 \cdot 56w_1 = 0$, which gives us this result: $672 + 3136w_1 = 0$. Now we isolate the $w_1$ by subtracting 672 from the LHS

$$3136w_1 = -672$$

and divide by 3136

$$w_1 = \frac{-672}{3136}.$$

Now we find the greatest common divisor. I was too lazy to do this when I was writing this out so I used Python.

Since this value lies outside the feasible interval $[0, 1]$, the SCM solution occurs at the nearest boundary, which is $w_1 = 0$ and $w_2 = 1$. This gives us the synthetic control

$$\hat{\mathbf{y}} = \mathbf{y}_2 = \begin{bmatrix} 48 \\ 50 \end{bmatrix},$$

which is simply the second donor vector copied over. Now, we can verify with `cvxpy`:

Okay, so without question, this is the toughest derivation we've done so far. But I want to emphasize why I chose to do it this way. Even in a corner solution like the one we just saw, there is a powerful lesson to be learned: if your treated unit is extreme with respect to your donor pool, the weight assigned to the donors will increasingly hinge on the nearest neighbors, because that is the best the convex hull condition allows. In other words, the SCM is responding to the geometry of the data. The treated unit lies outside the convex hull/span of the donors, so the optimization goes to the nearest boundary, assigning all weight to the donor closest to the treated unit.

This has a clear practical takeaway: analysts should always check whether the treated unit is extreme relative to the donor set. If it is, the synthetic control may rely heavily on a few donors, and the resulting fit may reflect geometric constraints rather than donor suitability. Of course, there are ways to circumvent this problem, but understanding that it is a problem in the first place is key. Being comfortable reasoning about the boundary behavior of one's is crucial for interpreting SCM results correctly and for making informed decisions about donor selection, as well as diagnosing whether the standard solver needs adjustments.

**Problem 9.5. Corner Solutions**

What does it mean for a treated unit to lie "outside the convex hull" of the donor pool? How does this affect the synthetic control weights?

**Problem 9.6. Simple 2D SCM**

Given a treated unit $\mathbf{y} = [50, 60]^\top$ and donor units $\mathbf{y}_1 = [20, 22]^\top$, $\mathbf{y}_2 = [48, 50]^\top$, derive the optimal SCM weight $w_1$ analytically, and identify the resulting synthetic control.

**Problem 9.7. Residual Norm Minimization**

Let $\mathbf{r}(w) = \mathbf{y} - \hat{\mathbf{y}}(w)$, where $\hat{\mathbf{y}}(w) = \sum_j w_j \mathbf{y}_j$. Show that minimizing $\|\mathbf{r}(w)\|_2^2$ is equivalent to the standard least squares problem for SCM.

## 9.2 Returning to the Basque Example

Per our discussion and **?@fig-donor-span**, we can see that the Basque Country is not extreme with respect to its donor set. The Basque Country is an outlier in other ways, such as industrialization and education and other covariates, but it is relatively well within the span of the outcomes of the donor pool. This suggests that we can at least try and use the standard SCM to obtain our weights for the Basque Country. For the purposes of distinguishing these from one another, I will compare the Convex Basque to the OLS Basque as well as the penalized approaches from the previous chapter.

The OLS weights are highly variable, with some donors receiving extremely large positive or negative values, reflecting the unconstrained nature of the estimator and its tendency to overfit the pre-treatment data. Ridge (L2) penalization shrinks these extreme weights toward zero, producing smaller positive and negative values, but still allowing negative contributions. LASSO (L1) introduces sparsity, assigning nonzero weights to only a few donors, notably Cataluna, Madrid, and Principado de Asturias, while setting most others to zero. The SIMPLEX estimator, by contrast, produces a sparse, interpretable set of weights that are strictly non-negative and sum to one, with Cataluna contributing the majority of the weight, followed by moderate contributions from Madrid and a minimal contribution from Principado de Asturias. Compared to LASSO, SIMPLEX is cleaner and more interpretable, as it enforces the convex combination constraint while automatically selecting the most relevant donors for constructing the synthetic Basque counterfactual without needing to tune any lambda parameters. Now we can plot the predictions of both models, depicted in **?@fig-scm-basque**.

We can see here that aside from having more sensible weights, the convex SCM has a more sensible out of sample prediction too, certainly compared to OLS. This is because of the convex hull restriction, meaning that the weights will never allow the post-intervention prediction to extrapolate. In other words, even though SCM does not fit the Basque Country as well as least-squares does, the out-of-sample predictions are more realistic because of the geometric regularization that SCM provides.

The pre-treatment RMSEs indicate that OLS achieves the closest fit to the Basque series before treatment, with a near-zero RMSE of 0.002, reflecting its unconstrained ability to overfit the pre-treatment data. Ridge, LASSO, and SIMPLEX have higher pre-treatment RMSEs (0.065, 0.083, and 0.084, respectively) due to penalization or convexity constraints, which limit overfitting but produce a slightly less precise fit. In terms of the ATT, OLS predicts a large positive post-treatment effect (1.522), likely exaggerated by its overfitting, while Ridge, LASSO, and SIMPLEX yield negative ATTs (-0.758, -0.671, and -0.692, respectively), suggesting that the Basque outcome was slightly lower than the synthetic counterfactual post-treatment. Overall, these results illustrate a trade-off: OLS achieves perfect pre-treatment fit but unrealistic post-treatment effects, whereas Ridge, LASSO, and SIMPLEX provide more regularized, interpretable estimates at the cost of slightly higher pre-treatment error.

## 9.3 Finite Sample Error Under The Linear Factor Model

Okay, so this is a nice trick, but analysts should also want to quantify the conditions under which SCM is effective. In other words, we should wish to derive the circumstances where we would expect SCM style methods to perform well. Abadie et al. (2010) motivates SCM by a linear factor model. We are interested in characterizing the post-treatment estimation error for the treated unit under the linear factor model, assuming pre-treatment fit is perfect. When I say estimation error/bias here, I refer to the absolute difference between the estimated treatment effect and the true treatment effect. We do not care at all about the direction of the bias, we just care about the absolute bias. So by definition, for any post-treatment period $t$, the bias is characterized as

$$\epsilon := \hat{\tau}_{1t} - \tau_{1t} = y_{1t}(0) - \sum_{j \in \mathcal{N}_0} w_j y_{jt}(0).$$

Substituting the linear factor model

$$y_{jt}(0) = \delta_t + \theta_t Z_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

we obtain

$$\epsilon = (\delta_t + \theta_t Z_1 + \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t}) - \sum_{j \in \mathcal{N}_0} w_j (\delta_t + \theta_t Z_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}).$$

Expanding the weighted sum:

$$\epsilon = (\delta_t + \theta_t Z_1 + \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t}) - \left( \delta_t \sum_j w_j + \theta_t \sum_j w_j Z_j + \sum_j w_j \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \sum_j w_j \varepsilon_{jt} \right).$$

By SCM weight constraints, $\sum_j w_j = 1$, so the intercept cancels:

$$\delta_t - \delta_t \sum_j w_j = 0.$$

If pre-treatment covariates are balanced, $\sum_j w_j Z_j = Z_1$, so the covariate term cancels:

$$\theta_t Z_1 - \theta_t \sum_j w_j Z_j = 0.$$

This leaves

$$\epsilon = \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t} - \sum_j w_j \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t - \sum_j w_j \varepsilon_{jt}.$$

Define the factor-loading mismatch vector

$$\boldsymbol{\nu} := \boldsymbol{\mu}_1 - \sum_j w_j \boldsymbol{\mu}_j.$$

Then the post-treatment error becomes

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

Exact pre-treatment fit implies

$$0 = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}, \quad \forall t \in \mathcal{T}_1.$$

Rearranging:

$$\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} = \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Stacking over all pre-treatment periods defines

$$\Lambda \boldsymbol{\nu} = b, \quad b := \begin{pmatrix} \sum_j w_j (\varepsilon_{jt_1} - \varepsilon_{1t_1}) \\ \vdots \\ \sum_j w_j (\varepsilon_{jt_{T_0}} - \varepsilon_{1t_{T_0}}) \end{pmatrix},$$

where $\Lambda \in \mathbb{R}^{T_0 \times r}$ has rows $\boldsymbol{\lambda}_t^\top$. If $\Lambda$ has full column rank $r$, the least squares solution is

$$\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b.$$

Expanding:

$$\Lambda^\top \Lambda = \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top, \quad \Lambda^\top b = \sum_{t \in \mathcal{T}_t} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Hence

$$\boldsymbol{\nu} = \Big( \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Finally, the post-treatment estimation error can be written as

$$\epsilon = \boldsymbol{\lambda}_t^\top \Big( \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}) + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

This decomposition shows two sources of post-treatment error: (i) pre-treatment idiosyncratic differences projected through the factor space, and (ii) direct post-treatment idiosyncratic mismatch. Given this finite sample bound, we can now get to the main identifying assumption of the SCM:

**Lemma B.1 (Li & Shankar (2024))**

Consider the linear factor model:

$$y_{jt}(0) = \delta_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}, \quad j \in \mathcal{N}_0 \cup \{1\},\ t \in \mathcal{T}_1,$$

where $\delta_j$ is the intercept, $\boldsymbol{\mu}_j \in \mathbb{R}^r$ is the vector of factor loadings for unit $j$, and $\boldsymbol{\lambda}_t \in \mathbb{R}^r$ is a vector of common factors at time $t$. SCM's identification assumption holds if all control units share the same intercept and factor loadings:

$$\delta_j = \delta, \quad \boldsymbol{\mu}_j = \boldsymbol{\mu}, \quad \forall j \in \mathcal{N}_0,$$

and the treated unit's intercept and factor loadings can be expressed as a convex combination of the controls:

$$\delta_1 = \sum_{j \in \mathcal{N}_0} w_j \delta_j, \quad \boldsymbol{\mu}_1 = \sum_{j \in \mathcal{N}_0} w_j \boldsymbol{\mu}_j, \quad \mathcal{W} \in \Big\{ \mathbf{w} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \Big\}.$$

However, it is important to note that the underlying assumption that the treated unit's intercept $\delta_1$ and factor loadings $\boldsymbol{\mu}_1$ can be represented as a convex combination of the controls' parameters is not directly testable, because the latent factors $\boldsymbol{\lambda}_t$ and unit-specific loadings $\boldsymbol{\mu}_j$ are unobserved. This is the finite sample bound; asymptotically, we can show that the error is bounded by a function in Theorem 9.1.

**Theorem 9.1** (Asymptotic Bias Bound)**.** *As shown in Abadie et al. (2010), as the number of pre-treatment periods $|\mathcal{T}_1| \to \infty$ and the synthetic control achieves increasingly precise pre-treatment fit, the bound on the post-treatment bias shrinks.*

$$\Big|\mathbb{E}[\epsilon]\Big| \lesssim C(g)^{1/g} \frac{\bar{\lambda}^2 F N^{1/g}}{\xi_{A_P}} \max\Big( \frac{\bar{m}_g^{1/g}}{T_0^{1-1/g}}, \frac{\bar{\sigma}^{1/2}}{T_0^{1/2}} \Big).$$

Proof

When we construct a synthetic control, the quantity $\epsilon = \hat{\tau}_{1t} - \tau_{1t}$ measures the difference between the estimated and true post-treatment effect. The asymptotic bias bound provides a way to understand how large this expected error can be as the number of pre-treatment periods grows. It shows that as the pre-treatment window $T_0$ increases and the synthetic control fits the treated unit more precisely, the expected bias shrinks.

The bound depends on several intuitive factors. The term $C(g)^{1/g}$ comes from controlling the contribution of large, rare shocks in the pre-treatment idiosyncratic errors; it scales the bound according to how "heavy-tailed" these shocks are but does not shrink with more data. The factor $\bar{\lambda}^2$ measures the size of the common factors; larger factors amplify any mismatch between the treated unit and the synthetic control. The number of latent factors, $F$, represents the dimensions along which the treated unit can differ from the controls, so more factors increase potential bias. The term $N^{1/g}$ reflects the influence of having more donor units: even though the synthetic control combines them optimally, more donors introduce additional sources of variation. Finally, the smallest eigenvalue of the factor Gram matrix, $\xi_{A_P}$, captures the geometry of the pre-treatment factors. A well-conditioned factor space (larger $\xi_{A_P}$) reduces the potential amplification of any mismatch, while a poorly conditioned factor space increases it.

The last piece of the bound, the maximum between $\bar{m}_g^{1/g}/T_0^{1-1/g}$ and $\bar{\sigma}^{1/2}/T_0^{1/2}$, reflects the stochastic component of the bias coming from the idiosyncratic shocks. The first term captures the effect of occasional large shocks, while the second captures typical fluctuations measured by the variance. As the number of pre-treatment periods grows, both terms shrink, with the variance term typically dominating, leading to the familiar $T_0^{-1/2}$ rate of decay. The expected post-treatment bias decreases as more pre-treatment data are available and as the synthetic control fits better, but it also clarifies how the number of factors, size of the factors, number of donors, and factor space geometry influence the potential size of the bias.

**Problem 9.8. Factor-Loading Mismatch**

Starting from the linear factor model

$$y_{jt}(0) = \delta_t + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

show that the post-treatment error for the treated unit can be written as

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

**Problem 9.9. Corner Solution Behavior**

Suppose the donor pool consists of two units with pre-treatment outcomes

$$\mathbf{y}_1 = [2, 2]^\top, \quad \mathbf{y}_2 = [10, 10]^\top$$

and the treated unit has pre-treatment outcomes

$$\mathbf{y}_{\text{treated}} = [12, 12]^\top.$$

- Let the synthetic control weight vector be $\mathbf{w} = [w_1, w_2]^\top$.
- Analyze what happens to the synthetic control vector $\hat{\mathbf{y}}_{\text{SC}} = w_1\mathbf{y}_1 + w_2\mathbf{y}_2$ as $\lim_{w_2 \to 1^-} \mathbf{w}$.

**Problem 9.10. Pre-Treatment Fit and Bias**

Using the expression for $\Delta\boldsymbol{\mu}$, explain why increasing the number of pre-treatment periods $|\mathcal{T}_1|$ improves identification of the treated unit's latent factors.

**Problem 9.11. Convex SCM Implementation**

Using `cvxpy`, write a small script to compute SCM weights for a treated unit with 3 donors in 5 pre-treatment periods. Verify that weights are nonnegative and sum to 1.

**Problem 9.12. Comparing OLS and SCM**

For the Basque Country data, compute OLS weights and convex SCM weights. Compare the pre-treatment RMSE of both approaches. Which method produces more interpretable weights?

**Problem 9.13. Convex Hull Visualization**

Using Python or R, create a 2D plot showing the donor vectors and the convex hull. Plot the treated unit and the synthetic control on the same graph.

**Problem 9.14. Pre-Treatment MSE and Factor Matching**

Compute the pre-treatment mean squared error for SCM with varying subsets of donors. How does the MSE change as you remove the donors closest to the treated unit?

**Problem 9.15. Discussion / Critical Thinking**

Suppose the treated unit is extreme relative to all donors. What are the implications for SCM estimation and interpretation? How might you address this issue in practice?

**Problem 9.16. Negative Weights**

In standard SCM, weights are constrained to be nonnegative and sum to one. Discuss the implications of allowing negative weights.

- How might negative weights affect the interpretability of the synthetic control?
- When might allowing negative weights be useful?

**Problem 9.17. Adding an Intercept**

Suppose we modify the SCM optimization to include an unrestricted intercept term in addition to the convex combination of donors.

- How does this change the optimization problem?
- How might this affect pre-treatment fit and post-treatment extrapolation?
- Discuss potential benefits and drawbacks of including an intercept.

**Problem 9.18. High-Dimensional Factor Models**

Consider a scenario where the number of latent factors $r$ exceeds the number of pre-treatment periods $|\mathcal{T}_1|$.

- What happens to the factor-loading mismatch solution $\Delta\boldsymbol{\mu} = (\Lambda^\top\Lambda)^{-1}\Lambda^\top b$ in this case?
- How does this affect identification of the SCM weights?

**Problem 9.19. Heterogeneous Factor Loadings**

Suppose that in the linear factor model, donor units have heterogeneous factor loadings that cannot perfectly replicate the treated unit.

- How does this heterogeneity influence the post-treatment bias?
- How might one diagnose or mitigate this issue in applied settings?

**Problem 9.20. Convex Hull Limitations**

The SCM ensures that the synthetic control lies within the convex hull of the donors.

- Discuss situations in which this restriction might prevent a good counterfactual fit.
- How does the geometry of the donor pool constrain the SCM solution?
- Are there alternatives or modifications to SCM that can relax the convex hull constraint while retaining interpretability?

## 9.4 Conclusion

SCM provides a principled way to construct counterfactuals that respect the geometry of the donor pool, avoiding the implausible extrapolations that can occur under unconstrained OLS. By restricting weights to lie within the convex hull of the donors, SCM (ideally) produces interpretable and stable synthetic controls, even in small-$N$ settings. While the key identification assumption—that the treated unit's latent factors can be represented as a convex combination of the controls—is not directly testable, Abadie et al. (2010) shows that as the number of pre-treatment periods increases and pre-treatment fit improves, the bound on post-treatment bias shrinks. In practice, a lower pre-treatment mean squared error indicates that the synthetic control is closely tracking the treated unit's unobserved factors, making it more likely that the SCM weights are effectively matching the relevant unit-specific characteristics. This geometric and factor-based perspective provides both theoretical justification and practical guidance for applying SCM reliably.

## 9.5 Asymptotic Bias Bound

*Proof.* The finite sample bound gives us a nice approximation for any given set of factors/loadings in the finite case. However, mathematics is not concerned with secular realities. Econometricians oftentimes wish to make statements about how estimators behave as a sample size grows without bound. For this proof, we are going to algebraically manipulate the finite sample bound into something asymptotic in nature. This way, we can make statements about how the bound of the bias changes given some set of inputs.

Above I mentioned how the finite-sample bound has two sources: (i) pre-treatment idiosyncratic differences projected through the factor space, and (ii) direct post-treatment idiosyncratic mismatch. Thus, this proof proceeds in two acts. In the first act, we focus on the bias coming from pre-treatment idiosyncratic differences projected through the factor space. The tools we use to do this are the triangle inequality, Jensen's inequality, and Cauchy–Schwarz. In the second act, we deal with the bias from direct post-treatment idiosyncratic mismatches. Here, we use Hölder's inequality, the vector-valued Rosenthal inequality, and Minkowski's inequality.

### 9.5.1 Setup

Recall that in the finite sample bias is expressed as:

$$\epsilon := |\hat{\tau}_{1t} - \tau_{1t}|.$$

The interactive fixed effects model is:

$$y_{jt} = \boldsymbol{\lambda}_t^\top \boldsymbol{\mu}_j + \varepsilon_{jt}, \quad j = 1, \ldots, N+1,$$

where $\boldsymbol{\lambda}_t$ are common factors, $\boldsymbol{\mu}_j$ are factor loadings, and $\varepsilon_{jt}$ are idiosyncratic shocks. The post-intervention error for the treated unit is

$$\epsilon = \boldsymbol{\lambda}_t^\top \Big( \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}) + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}$$

or more simply

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt},$$

where

$$\boldsymbol{\nu} := \boldsymbol{\mu}_1 - \sum_j w_j \boldsymbol{\mu}_j$$

is the factor-loading difference between the treated unit and the synthetic control.

### 9.5.2 Act 1

To start off, we begin by taking expectations and the absolute value of both sides of the equal sign:

$$|\mathbb{E}[\epsilon]| = \Big| \mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}] \Big|.$$

As we mentioned, we do this because we do not care about the direction of the bias, so the absolute value allows us to make more general statements about the bias in this manner. By the linearity of expectation, we can separate the sum inside the expectation into two parts:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}] + \mathbb{E}[\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}]|.$$

Because the RHS of the equals sign is a sum that evalautes to a scalar, the first inequality from our toolkit that we will apply is the triangle inequality. Recall that this inequality states that the magnitude of a sum cannot exceed the sum of magnitudes — the "length" of two steps taken together can never be longer than taking them one after another. In our context, this means that the total expected bias (which combines factor-driven and idiosyncratic components) cannot exceed the sum of the individual contributions of each source of bias. Formally:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + (\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt})]| \leq |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]| + |\mathbb{E}[\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}]|.$$

We assume that the idiosyncratic shocks are mean-zero and have finite variance, i.e. $\mathbb{E}[\varepsilon_{jt}] = 0$ and $\text{Var}(\varepsilon_{jt}) < \infty$. Hence, the expected post-treatment idiosyncratic mismatch vanishes, leaving only the component driven by the common factors:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]|.$$

Note that even if the random noise cancels out on average, in any finite sample it might not — and we want to know how big that randomness can make our bias look (that is the point of Act 2).

Anyways,this stage, we have the absolute value of an expectation:

$$|\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]|.$$

Note that the absolute value function $f(x) = |x|$ is convex. Why? Consider two numbers $x = -2$ and $y = 4$, and let $\theta = 0.3$. Then the weighted average is

$$\theta x + (1-\theta)y = 0.3(-2) + 0.7(4) = -0.6 + 2.8 = 2.2.$$

Taking the absolute value gives

$$|\theta x + (1-\theta)y| = |2.2| = 2.2.$$

On the other hand, the weighted sum of absolute values is

$$\theta|x| + (1-\theta)|y| = 0.3(2) + 0.7(4) = 0.6 + 2.8 = 3.4.$$

Indeed, we see that

$$|\theta x + (1-\theta)y| = 2.2 \leq 3.4 = \theta|x| + (1-\theta)|y|,$$

Extending the triangle inequality to weighted sums, the "length" of a weighted average of two numbers cannot exceed the weighted sum of their individual lengths. Because the absolute value function is convex, we can bound this term with Jensen's inequality. Applying Jensen gives

$$|\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]| \leq \mathbb{E}[|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}|].$$

Intuitively, this says that the magnitude of the average bias is no larger than the average of the magnitudes of the biases across different realizations.

Now note that $\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}$ on the LHS is a dot product between two vectors: the common-factor vector at time $t$ and the factor-loading difference between the treated unit and the estimated synthetic control. The way we bound dot products between vectors is with Cauchy-Schwarz. When we apply this to the LHS, we get this result:

$$|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}| \leq \|\boldsymbol{\lambda}_t\|_2 \, \|\boldsymbol{\nu}\|_2.$$

However, this bound may be further simplified by introducing a uniform bound over all pre-treatment periods. To do this, we take the max norm of all of the common factors in the pre-treatment period

$$\bar{\lambda} := \|\boldsymbol{\lambda}\|_\infty = \max_{\in \mathcal{T}_1} \|\boldsymbol{\lambda}_t\|_2.$$

This is the largest value of any common factor across the pre-treatment periods. Since each $\|\boldsymbol{\lambda}_t\|_2 \leq \bar{\lambda}$, we can factor this constant out of the expectation:

$$\mathbb{E}[|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}|] \leq \mathbb{E}[\bar{\lambda} \, \|\boldsymbol{\nu}\|_2] = \bar{\lambda} \, \mathbb{E}[\|\boldsymbol{\nu}\|_2].$$

Thus, combining with the previous steps, we obtain

$$|\mathbb{E}[\epsilon]| \leq \bar{\lambda} \, \mathbb{E}\|\boldsymbol{\nu}\|_2.$$

At this point, the only remaining random or unknown quantity is the factor-loading difference $\boldsymbol{\nu}$. So far, we've manipulated the norms of these, but we have not considered the factor difference itself. As it happens, we can bound this term too. Recall the definition of $\boldsymbol{\nu}$, or the difference in factor loadings. In the finite sample derivation, we defined it implicitly via the least squares system

$$\Lambda \boldsymbol{\nu} = b, \quad b_t = \sum_j w_j(\varepsilon_{jt} - \varepsilon_{1t}), \quad \Lambda = [\boldsymbol{\lambda}_1^\top; \ldots; \boldsymbol{\lambda}_{T_0}^\top],$$

solving for it analytically. The solution is:

$$\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b.$$

To place a bound on the maximum factor loading mismatch, we can use the operator norm. The point of this here is to essentially answer "Given the pre-treatment mismatches ($b$) and the geometry of the factor space ($(\Lambda^\top \Lambda)^{-1}\Lambda^\top$), what is the largest possible factor-loading difference that could result?" By the submultiplicativity of the operator norm:

$$\|\boldsymbol{\nu}\|_2 = \|(\Lambda^\top \Lambda)^{-1} \Lambda^\top b\|_2 \leq \|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} \, \|\Lambda^\top b\|_2.$$

Think of the factor-loading difference as the outcome of shooting a bullet at a bullseye in a dark room. The matrix transformation, which combines the geometry of the factor space and the projection of pre-treatment mismatches, acts like the layout of the room, stretching and rotating the trajectory of the bullet in different directions. The operator norm then

identifies the worst-case scenario: the maximum distance the bullet could land from the bullseye, given your current vantage point. In other words, even without knowing the exact direction of the pre-treatment mismatches, the operator norm provides a bound on how large the factor-loading difference can be, capturing the greatest possible amplification caused by the factor space.

Normalize the Gram matrix $M = \frac{1}{T_0}\Lambda^\top\Lambda$, giving

$$\|(\Lambda^\top\Lambda)^{-1}\|_{\text{op}} = \frac{1}{T_0}\|M^{-1}\|_{\text{op}} \leq \frac{1}{T_0\,\xi_{A_P}}, \quad \xi_{A_P} = \lambda_{\min}(M).$$

In the bullseye analogy, $T_0$ represents the number of vantage points from which you can take a shot at the bullseye. Each vantage point provides information about the geometry of the room, and averaging across these points (normalizing by $T_0$) reduces the maximum potential deviation of a shot. The smallest eigenvalue $\xi_{A_P}$ identifies the weakest direction in the room, the direction in which the bullet can be stretched the most. Together, $T_0$ and $\xi_{A_P}$ give a conservative bound on how far any shot could land from the bullseye, capturing the worst-case amplification of pre-treatment mismatches.

### 9.5.3 Act 2: Bounding the Stochastic Component

We now address the stochastic component of the estimation error, which arises because the idiosyncratic shocks $\varepsilon_{jt}$ of the control units cannot perfectly match those of the treated unit $\varepsilon_{1t}$, even with optimal weights $w_j$. The goal is to bound the expected factor-loading mismatch $\mathbb{E}\|\boldsymbol{\nu}\|_2$, which captures how random pre-treatment shocks propagate through the factor structure to affect the bias. The approach uses Hölder's inequality to control per-period mismatches, the vector-valued Rosenthal inequality to quantify their accumulation over time, and Minkowski's inequality to simplify the final bound.

The per-period mismatch is defined as $b_t = \sum_j w_j(\varepsilon_{jt} - \varepsilon_{1t})$, where $\varepsilon_{jt}$ represents the idiosyncratic shock for control unit $j$ at time $t$, $\varepsilon_{1t}$ is the shock for the treated unit, and $w_j$ are the weights forming the synthetic control. This $b_t$ measures how much the weighted average of control shocks deviates from the treated unit's shock at time $t$. Imagine trying to match a song's melody with a choir that's slightly out of tune: $b_t$ is the error in that approximation for one note. For clarity, define $\bar{m}_g = \sup_t \mathbb{E}|b_t|^g$ for some $g \geq 2$, which captures the worst-case size of large shocks (higher moments), and $\bar{\sigma} = \sup_t \mathbb{E}[b_t^2]$, which measures typical variability (variance). Assume the shocks $\varepsilon_{jt}$ are independent across time $t$ and units $j$, with mean zero ($\mathbb{E}[\varepsilon_{jt}] = 0$) and finite moments up to order $g$. The factors $\boldsymbol{\lambda}_t$ are treated as fixed (non-random), a standard simplification in this context.

To control $b_t$, apply Hölder's inequality, which separates the weights and shocks:

$$|b_t| \leq \left(\sum_j w_j^q\right)^{1/q} \left(\sum_j |\varepsilon_{jt} - \varepsilon_{1t}|^p\right)^{1/p}, \quad \frac{1}{p} + \frac{1}{q} = 1.$$

This is like balancing two forces: the weights $w_j$ (how much each control unit contributes) and the shock differences $\varepsilon_{jt} - \varepsilon_{1t}$ (how far each unit's noise deviates from the treated unit). Hölder's inequality ensures neither dominates excessively, bounding the mismatch by the interplay of weight distribution and shock variability.

Next, define the vector $X_t = b_t \boldsymbol{\lambda}_t$, where $\boldsymbol{\lambda}_t$ scales the mismatch by the factor magnitudes. The total stochastic deviation over the pre-treatment window is $S = \sum_{t=1}^{T_0} X_t$, capturing the cumulative effect of these mismatches. To bound $\mathbb{E}\|S\|_2$, use the vector-valued Rosenthal inequality for independent mean-zero vectors $X_t \in \mathbb{R}^F$ with $g \geq 2$:

$$\mathbb{E}\|S\|_2^g \lesssim C(g) \left[ \sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^g + \left( \sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^2 \right)^{g/2} \right],$$

where $C(g)$ is a constant depending only on $g$, and $\lesssim$ hides absolute constants irrelevant for the asymptotic rate. This inequality is like estimating the total error in a series of coin tosses over $T_0$ periods: the first term captures rare large errors (higher moments), while the second reflects typical fluctuations (variance). Since $\|X_t\|_2 = |b_t| \|\boldsymbol{\lambda}_t\|_2 \leq |b_t| \bar{\lambda}$, where $\bar{\lambda} = \max_{t \in \mathcal{T}_1} \|\boldsymbol{\lambda}_t\|_2$, account for $F$ factors (since $X_t$ has $F$ components). Thus, $\mathbb{E}\|X_t\|_2^g \lesssim F \bar{\lambda}^g \mathbb{E}|b_t|^g$ and $\mathbb{E}\|X_t\|_2^2 \lesssim F \bar{\lambda}^2 \mathbb{E}[b_t^2]$.

Using Hölder's inequality and the normalization $\sum_j w_j^q \leq 1$, obtain $\mathbb{E}|b_t|^g \lesssim \bar{m}_g$ and $\mathbb{E}[b_t^2] \leq \bar{\sigma}$. Including $N$ control units (which enters multiplicatively in $b_t$), this becomes $\mathbb{E}\|X_t\|_2^g \lesssim F N^{1/g} \bar{\lambda}^g \bar{m}_g$ and $\mathbb{E}\|X_t\|_2^2 \lesssim F N^{1/g} \bar{\lambda}^2 \bar{\sigma}$. Summing over $t = 1, \ldots, T_0$ gives $\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^g \lesssim T_0 F N^{1/g} \bar{\lambda}^g \bar{m}_g$ and $\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^2 \lesssim T_0 F N^{1/g} \bar{\lambda}^2 \bar{\sigma}$. Substituting into Rosenthal's inequality yields:

$$\mathbb{E}\|S\|_2^g \lesssim C(g) \left[ T_0 F N^{1/g} \bar{\lambda}^g \bar{m}_g + \left( T_0 F N^{1/g} \bar{\lambda}^2 \bar{\sigma} \right)^{g/2} \right].$$

Taking the $g$-th root and applying Minkowski's inequality (which picks the dominant contribution, like choosing the louder of two competing noises) simplifies to:

$$\mathbb{E}\|S\|_2 \lesssim C(g)^{1/g} F N^{1/g} \max \left( \left( T_0 \bar{\lambda}^g \bar{m}_g \right)^{1/g}, \left( T_0 \bar{\lambda}^2 \bar{\sigma} \right)^{1/2} \right).$$

Simplifying each term, $\left( T_0 \bar{\lambda}^g \bar{m}_g \right)^{1/g} = T_0^{1/g} \bar{\lambda} \bar{m}_g^{1/g}$ and $\left( T_0 \bar{\lambda}^2 \bar{\sigma} \right)^{1/2} = \sqrt{T_0} \bar{\lambda} \bar{\sigma}^{1/2}$, so:

$$\mathbb{E}\|S\|_2 \lesssim C(g)^{1/g} F N^{1/g} \bar{\lambda} \max \left( T_0^{1/g} \bar{m}_g^{1/g}, \sqrt{T_0} \bar{\sigma}^{1/2} \right).$$

This bound shows that the stochastic mismatch grows with the number of factors $F$ and control units $N^{1/g}$, but is moderated by the pre-treatment window $T_0$. To connect to the factor-loading mismatch, recall $\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b$, so $\mathbb{E}\|\boldsymbol{\nu}\|_2 \leq \|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} \mathbb{E}\|S\|_2$. Since $\|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} = 1/(T_0 \xi_{A_P})$, where $\xi_{A_P} = \lambda_{\min}(M)$ and $M = \frac{1}{T_0} \Lambda^\top \Lambda$, combine to get:

$$\mathbb{E}\|\boldsymbol{\nu}\|_2 \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}}{T_0 \xi_{A_P}} \max\left(T_0^{1/g}\bar{m}_g^{1/g}, \sqrt{T_0}\bar{\sigma}^{1/2}\right).$$

Simplify the powers of $T_0$: $T_0^{1/g}/T_0 = T_0^{-(1-1/g)}$ and $\sqrt{T_0}/T_0 = T_0^{-1/2}$, yielding:

$$\mathbb{E}\|\boldsymbol{\nu}\|_2 \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}}{\xi_{A_P}} \max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right).$$

Since the post-treatment bias satisfies $|\mathbb{E}[\epsilon]| \leq \bar{\lambda}\mathbb{E}\|\boldsymbol{\nu}\|_2$, the final bound is:

$$|\mathbb{E}[\epsilon]| \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}^2}{\xi_{A_P}} \max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right).$$

This bound reveals the bias's behavior as $T_0$ grows. The term $\max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right)$ determines the decay rate: the variance term $\bar{\sigma}^{1/2} T_0^{-1/2}$ typically dominates, giving a decay rate of $O(T_0^{-1/2})$, meaning the bias shrinks as more pre-treatment data is available. For example, if $T_0 = 100$ and the variance term dominates, $T_0^{-1/2} = 0.1$, significantly reducing the bias. However, more factors $F$ or control units $N$ can increase the bound, while a well-conditioned factor space (larger $\xi_{A_P}$) reduces it. Hölder's inequality controlled per-period errors, Rosenthal's inequality quantified their accumulation, and Minkowski's inequality isolated the dominant term, providing a complete picture of how stochastic noise propagates through the factor structure and decays with $T_0$. □

:::

# Chapter 10

# Validation of Synthetic Control Methods

In this chapter, we focus on validating and conducting inference for SCM. As we have seen in the last chapter, SCM provides a transparent and flexible framework for estimating counterfactual outcomes. However, assessing the credibility of these estimates requires careful inferential tools. In principle, SCM is typically justified in small-$N$ settings, where the number of units is limited relative to methods such as difference-in-differences (DID). Because standard inferential techniques rely on asymptotic approximations and the Central Limit Theorem, they may lack power or validity when the number of units is small, necessitating alternative strategies tailored to the SCM framework.

A natural class of methods are *placebo tests*, which provide falsification checks by applying the SCM procedure in settings where no treatment effect should exist. In-space placebos examine whether the treated unit's post-treatment gap is extreme relative to gaps observed when donor units are treated instead. In-time placebos artificially shift the treatment to pre-intervention periods to detect spurious effects arising from model misspecification or temporal trends. Standard leave-one-out analyses sequentially remove individual donors to identify dominant contributors, though they do not capture joint dependencies among donors.

Building on these ideas, the *Leave-Two-Out Synthetic Control Method* (LTO-SCM) evaluates robustness by systematically removing all pairs of donors. By generating a distribution of treatment effects under different donor pair omissions, LTO-SCM captures the sensitivity of the estimated effect to the donor composition. Empirical quantiles of this distribution provide p-values and confidence intervals, allowing finite-sample inference while highlighting model fragility when certain donor combinations strongly influence the estimate.

Other approaches focus on resampling to quantify uncertainty. *Subsampling* re-estimates SCM weights on subsets of pre-treatment periods, producing empirical distributions of post-treatment deviations for confidence intervals and t-statistics. *Bootstrap methods* instead resample donor units to capture variability arising from the composition of the donor pool. Finally, the *debiased K-fold t-test* mitigates overfitting bias by cross-fitting SCM weights on out-of-sample pre-treatment blocks, producing corrected estimates, standard errors, and

t-statistics. Collectively, these methods offer a structured framework for understanding the robustness, uncertainty, and inferential credibility of synthetic control estimates, particularly in small-sample settings where traditional techniques fail.

## 10.1 Placebos

To address challenges of inference with a small $N$, a natural starting point is placebo tests, which serve as falsification checks within the SCM framework.

### 10.1.1 In-Space Placebo Test

SCM is rooted in experimental design philosophy, so it makes sense that the in-space placebo method would follow naturally. The method is based on a simple premise: if the treated unit $i = 1$ truly experiences a treatment effect, then its post-treatment gap should appear extreme relative to "placebo" effects obtained by pretending that donor units were treated instead.

Formally, for each unit $i \in \mathcal{N}_0$, the donor set is adjusted to $\mathcal{N}_0 \setminus \{i\}$, and placebo weights $\mathbf{w}^{\mathrm{SCM},i}$ are obtained by minimizing

$$\mathbf{w}^{\mathrm{SCM},i} = \underset{\mathbf{w} \in \mathcal{W}_{\mathrm{conv}} \cap \mathbb{R}^{\mathcal{N}_0 \setminus \{i\}}}{\operatorname{argmin}} \left\| \mathbf{y}_i - \mathbf{Y}_{\mathcal{N} \setminus \{i\}} \mathbf{w} \right\|_2^2, \tag{10.1}$$

$$\mathcal{W}_{\mathrm{conv}} \cap \mathbb{R}^{\mathcal{N}_0 \setminus \{i\}} = \left\{ \mathbf{w} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0 \setminus \{i\}|} : \sum_{j \in \mathcal{N}_0 \setminus \{i\}} w_j = 1 \right\}. \tag{10.2}$$

The resulting placebo counterfactual is then

$$\hat{y}_{it}(0) = \sum_{j \in \mathcal{N}_0 \setminus \{i\}} w_j^{\mathrm{SCM},i} \, y_{jt},$$

yielding placebo effects

$$\hat{\tau}_{it} = y_{it} - \hat{y}_{it}(0)$$

across all periods. These effects are collected into a set $\{\hat{\tau}_{it}\}_{i \in \mathcal{N}_0}$, or averaged as

$$\bar{\hat{\tau}}_i = \frac{1}{|\mathcal{T}_0|} \sum_{t \in \mathcal{T}_0} \hat{\tau}_{it},$$

with placebos filtered to those matching the treated unit's pre-treatment RMSE for comparability.

### 10.1.2 In Time

In-time placebos serve as a critical robustness check within SCM framework by testing the model's ability to correctly identify the absence of treatment effects when the intervention is artificially shifted to pre-treatment periods. The core idea is to backdate the treatment timing to an earlier point $T_0' < T_0$, where $T_0$ denotes the original pre-treatment cutoff, and assess whether the SCM generates significant placebo effects in what should remain untreated periods. This helps validate that the observed post-treatment gap is not an artifact of the model fitting process.

To formalize this, consider the pre-treatment period $\mathcal{T}_1 = 1, 2, \ldots, T_0$ and the post-treatment period $\mathcal{T}_2 = T_0 + 1, \ldots, T$, where $T$ is the total number of time periods. For the in-time placebo, select a placebo pre-treatment cutoff $T_0' < T_0$, such as the midpoint of $\mathcal{T}_1$ (e.g., $T_0' = \lfloor T_0/2 \rfloor$), to split $\mathcal{T}_1$ into a placebo pre-period $\mathcal{T}_1' = 1, 2, \ldots, T_0'$ and a placebo post-period $\mathcal{T}_0' = T_0' + 1, \ldots, T_0$. The full donor set $\mathcal{N}_0$ remains unchanged, as the test focuses on temporal rather than unit variation. The SCM weights $\mathbf{w}'^*$ are then estimated by minimizing the squared Euclidean norm of the difference between the treated unit's outcomes $\mathbf{y}_1'$ over $\mathcal{T}_1'$ and the donor matrix $\mathbf{Y}_0'$ over the same period. Mathematically, this is expressed as

$$\mathbf{w}'^* = \underset{\mathbf{w} \in \mathcal{W}_{\text{conv}}}{\operatorname{argmin}} \left\| \mathbf{y}_1' - \mathbf{Y}_0' \mathbf{w} \right\|_2^2,$$

where $\mathbf{y}_1'$ is the vector of treated unit outcomes $y_{1t}$ for $t \in \mathcal{T}_1'$ and $\mathbf{Y}_0'$ is the corresponding submatrix of donor outcomes $\mathbf{Y}_{0t}$ for $t \in \mathcal{T}_1'$. The placebo counterfactual for the treated unit in the placebo post-period is then constructed as $\hat{y}_{1t}'(0) = \sum_{j \in \mathcal{N}_0} w_j'^* y_{jt}$ for $t \in \mathcal{T}_0'$, and the placebo effect is computed as $\hat{\tau}_{1t}' = y_{1t} - \hat{y}_{1t}'(0)$. To summarize the effect across the placebo post-period, calculate the average placebo effect $\bar{\hat{\tau}}_1' = \frac{1}{|\mathcal{T}_0'|} \sum_{t \in \mathcal{T}_0'} \hat{\tau}_{1t}'$, where $|\mathcal{T}_0'|$ is the number of periods in $\mathcal{T}_0'$.

The justification for this approach rests on its ability to detect model fragility. If the SCM fits well only due to over-adaptation to pre-treatment trends, backdating the intervention should produce non-zero $\hat{\tau}_{1t}'$ values in $\mathcal{T}_0'$, where no true effect exists. For instance, a significant average effect $\bar{\hat{\tau}}_1'$ might indicate that the donor pool fails to capture the treated unit's latent dynamics. To enhance sensitivity, analysts can repeat the procedure for multiple $T_0'$ values. Additionally, comparing the root mean squared prediction error (RMSE) ratio—defined as the post-placebo RMSE over the pre-placebo RMSE—across these shifts can quantify how robust the model is to changes in treatment timing. Non-zero effects or high RMSE ratios in $\mathcal{T}_0'$ suggest that the original treatment effect may be unreliable, prompting further investigation into model specification or donor selection.

In the Basque data example, the intervention is backdated to 1973, two years before the original 1975 cutoff, by setting $T_0' = T_0 - 2$. The plot of placebo gaps versus the treated unit gap reveals that the backdated placebo effects remain close to the original, with the gap differing by less than 5% across the period. This stability suggests that the SCM fit is not overly sensitive to the exact timing of the intervention, supporting its credibility, though analysts should explore additional $T_0'$ values to confirm this robustness.

## 10.2 Confidence Intervals

Building on placebo-based validation, confidence intervals offer a way to quantify the uncertainty of SCM estimates. Inference for synthetic control estimates can be conducted using iterative methods such as altered placebo methods, subsampling, bootstrapping. Each method targets a different source of uncertainty and defines its own confidence intervals and test statistics.

### 10.2.1 Leave Two Out

While the in-space and in-time placebo tests provide valuable falsification checks, both rely on the assumption that the donor pool is exchangeable and that the model fit can be directly compared across units or time. However, these methods have important limitations. In-space placebos only test whether the treated unit's post-treatment gap appears large relative to donors, but not how sensitive the estimated effect is to the composition of the donor pool. In-time placebos test robustness to treatment timing, but as shown in simulation work by Lei & Sudijono (2025), such tests can yield spurious significance when selection bias is present or the donor pool poorly approximates the treated unit's latent factors.

SCM addresses these issues by constructing a counterfactual for the treated unit using a weighted combination of control units, optimized to match pre-treatment outcomes and covariates. Traditional inference methods, such as asymptotic approaches or standard placebo tests, often falter in small-sample settings common in SCM applications, for example where $N = 17$ as in the original Basque example. The Leave-Two-Out placebo test, proposed by Lei & Sudijono (2025), refines this framework to enhance inference validity and power, particularly when the number of control units is small.

The computation of p-values to assess the significance of the treatment effect proceeds under the null hypothesis $H_0 : Y_{I,t}(1) = Y_{I,t}(0)$ for all $t \geq T_0 + 1$, where $I$ is the treated unit and $T_0$ is the pre-treatment period. First, the approximate placebo p-value, denoted $p_{app-placebo}$, is calculated by fitting a synthetic control method for each control unit $i$ not equal to $I$ using all other units and computing the root mean squared prediction error ratio $R_i$, which is the post-treatment RMSPE divided by the pre-treatment RMSPE. The value of $R_I$ is similarly computed for the treated unit. Then $p_{app-placebo}$ is found as the proportion of control units where $R_i$ is greater than or equal to $R_I$, expressed as $p_{app-placebo} = \frac{1}{N-1} \sum_{i \neq I} 1R_i \geq R_I$. This method excludes the treated unit from the denominator, adjusting the standard placebo p-value downward.

Next, the exact placebo p-value, denoted $p_{exact-placebo}$, follows a similar process but includes the treated unit in the comparison set. For every unit $i$ from 1 to $N$, fit the synthetic control method using all other units and obtain $R_i$. The value of $p_{exact-placebo}$ is then the proportion where $R_i$ is greater than or equal to $R_I$, given by $p_{exact-placebo} = \frac{1}{N} \sum_{i=1}^{N} 1R_i \geq R_I$, where $N$ is the total number of units. This provides a valid p-value under uniform assignment but suffers from coarse granularity, with possible values restricted to the set $1/N, 2/N, \ldots, 1$.

The naive LTO p-value, denoted $p_{naive-LTO}$, is computed by considering every pair of control units $(k, l)$ where $k$ and $l$ are distinct and not equal to $I$. For each pair, fit synthetic control

methods for the treated unit $I$ and the two placebo units $k$ and $l$ using the remaining $N-3$ control units. Calculate $R_I$, $R_k$, and $R_l$ as the RMSPE ratios for each. The value of $p_{naive-LTO}$ is the proportion of pairs where $R_I$ is less than or equal to the maximum of $R_k$ and $R_l$, written as $p_{naive-LTO} = \frac{1}{C(N-1,2)} \sum_{(k,l)} 1R_I \leq \max(R_k, R_l)$, where $C(N-1,2)$ is the number of ways to choose 2 items from $N-1$. This method increases granularity with $O(N^2)$ comparisons, improving resolution over standard placebos.

Finally, the powered LTO p-value, denoted $p_{powered-LTO}(\alpha)$, adjusts $p_{naive-LTO}$ to enhance power while maintaining Type-I error control. It subtracts a correction term $c(N,\alpha)$, which is computed via binary search on the function $f(N,a)$ defined in the paper, from $p_{naive-LTO}$, with a small positive value $\epsilon$ added to avoid negative p-values, resulting in $p_{powered-LTO} = p_{naive-LTO} - c(N,\alpha) + \epsilon$. This optimization improves power for large effect sizes, with theoretical guarantees holding for $N\alpha \leq 1$ or $N\alpha < 3.9$ when $N > 10$.

Confidence intervals are computed within this framework by collecting the average treatment effect estimates from the placebo units in the LTO procedure. For each pair $(k,l)$, after fitting the synthetic control method for placebo units $k$ and $l$, compute the treatment effect $\tau_k$ and $\tau_l$ as the difference between observed post-treatment outcomes and their counterfactuals, then take the mean $\tau_{avg,k}$ and $\tau_{avg,l}$. Gather all such $\tau_{avg,k}$ and $\tau_{avg,l}$ values that correspond to valid fits, where validity is determined by the pre-fit RMSE being less than or equal to a threshold times the original pre-fit RMSE. If there are any valid $\tau_{avg}$ values, sort them and find the lower bound of the confidence interval as the value at the $\alpha/2$ quantile and the upper bound as the value at the $1-\alpha/2$ quantile, resulting in a $(1-\alpha)\times 100$ confidence interval. If no valid $\tau_{avg}$ values are available, the interval is set to (nan, nan).

The advantage of the LTO method over the standard placebo tests lies in several key areas. The standard placebo test generates only $N$ reference statistics, limiting p-values to a coarse grid such as $1/N, 2/N, \ldots, 1$. This can prevent rejection at common significance levels like $\alpha = 0.05$ when $1/N$ exceeds 0.05, a frequent issue in small samples. The LTO method, by considering all pairs of controls, produces $O(N^2)$ comparisons. Additionally, Lei & Sudijono (2025) find that the LTO method achieves better unconditional Type-I error control. The exact placebo test has a conditional Type-I error of $\lfloor N\alpha \rfloor / N$, which can be zero when $\alpha < 1/N$, while the approximate placebo inflates this to $(\lfloor N\alpha \rfloor + 1)/N$, risking over-rejection. The LTO method often realizes a lower unconditional Type-I error than its theoretical upper bound, for example frequently zero in simulations, providing validity across a wide range of $N$ and $\alpha$ as established in Theorem 2.1 of the paper.

Furthermore, the LTO method increases power. Empirical results from the California Proposition 99 dataset show LTO outperforming the approximate placebo in power for large effect sizes and matching or exceeding the exact placebo power across all sizes. The powered LTO variant further narrows the power gap for intermediate effects by leveraging the richer reference distribution from its numerous comparisons. Unlike asymptotic methods requiring large $N$ or $T$, LTO maintains finite-sample validity under uniform assignment and handles $\alpha < 1/N$ settings where standard placebos fail, making it particularly effective for typical SCM applications with few control units. Below, I implement these methods for the Basque Country (note that the authors use relatively in their specifications and I do not, so my results differ from theirs):

Here are the results from the p-values by Lei & Sudijono (2025), very similar to theirs with some exceptions. For example, my naive-LTO p value is 0.695238 whereas theirs is 0.67. My powered p value is 0.682738, whereas theirs is 0.66. Their exact and applied p values respectively are 0.41 and 0.35, whereas mine are 0.5 and 0.42

### 10.2.2 Block Permutation Inference

One approach to inference in synthetic control designs is based on *block permutation* Arkhangelsky et al. (2021). This method addresses *temporal dependence* in the pre-treatment residuals. The key idea is to treat the pre-treatment residuals as a stationary time series and generate placebo effects by permuting *blocks* of consecutive residuals rather than individual periods. This preserves the autocorrelation structure within each block while breaking its link with the treatment period.

Let $\hat{\varepsilon}_t = y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\mathrm{SCM}}$ denote the pre-treatment residuals for $t \le T_0$. Define the observed average post-treatment effect as

$$S_{\mathrm{obs}} = \frac{1}{T_2} \sum_{t=T_0+1}^{T} \left| y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\mathrm{SCM}} \right|.$$

We estimate the typical *block length* $B$ by identifying the lag at which the autocorrelation of $\hat{\varepsilon}_t$ falls below a chosen threshold $\rho$, for instance $\rho = 0.1$. Denote by $\mathcal{B}_1, \ldots, \mathcal{B}_K$ the $K = \lfloor T_0/B \rfloor$ contiguous blocks of pre-treatment residuals. To generate a permutation replicate, we draw (with replacement) a sequence of blocks $\{\mathcal{B}_{k_1}, \ldots, \mathcal{B}_{k_M}\}$ until we fill a vector of length $T_2$, concatenate them, and compute the corresponding placebo statistic

$$S^{*(s)} = \frac{1}{T_2} \sum_{t=1}^{T_2} \left| \tilde{\varepsilon}_t^{(s)} \right|,$$

where $\tilde{\varepsilon}_t^{(s)}$ are the permuted residuals. After repeating this for $S$ random permutations, the *global p-value* is

$$p_{\mathrm{global}} = \frac{1}{S} \sum_{s=1}^{S} \mathbf{1}\{S^{*(s)} \ge S_{\mathrm{obs}}\}.$$

We can also compute *per-period p-values* by comparing each post-treatment effect to the empirical distribution of the absolute pre-treatment residuals:

$$p_t = \frac{1}{T_0} \sum_{s=1}^{T_0} \mathbf{1}\{|\hat{\varepsilon}_s| \ge |y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\mathrm{SCM}}|\}, \quad t > T_0.$$

Finally, split-conformal confidence intervals for each period are given by

$$\text{CI}_t = \Big[(y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\text{SCM}}) - q_{1-\alpha},\ (y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\text{SCM}}) + q_{1-\alpha}\Big],$$

where $q_{1-\alpha}$ is the $(1-\alpha)$ quantile of the absolute pre-treatment residuals.

Block permutation inference therefore provides a robust, nonparametric way to generate uncertainty measures while respecting temporal dependence in the data. It is most effective when the pre-treatment sample is moderately long (say, $T_0 > 15$ periods) and exhibits serial correlation, and can be viewed as a natural complement to the bootstrap and subsampling approaches.

### 10.2.3 Subsampling-Based Inference

Inference for synthetic control estimators can proceed by exploiting the variability across pre-treatment periods. Because SCM weights are estimated by minimizing pre-treatment discrepancies, the distribution of these weights reflects the sampling variation of the pre-period outcomes. Subsampling-based inference, developed by Li (2020), formalizes this idea by repeatedly estimating the SCM on randomly drawn subsets of the pre-treatment window and using the resulting dispersion in estimated weights to approximate the sampling distribution of the treatment effect estimator.

Let $T_1$ and $T_2$ denote the number of pre- and post-treatment periods, respectively. Denote by $\mathbf{w}^{\text{SCM}}$ the weight vector estimated on the full pre-treatment sample, and let the estimated post-treatment effect be

$$\hat{\delta} = \frac{1}{T_2} \sum_{t>T_1} \left(y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\text{SCM}}\right).$$

To approximate the sampling variation of $\hat{\delta}$, we draw $J$ random subsets of pre-treatment periods, each of size $m < T_1$. Let $\mathcal{I}_m^{(j)} \subset \{1, \ldots, T_1\}$ denote the $j$-th subsample. For each draw, we re-estimate the synthetic control weights

$$\mathbf{w}^{(j)} = \arg\min_{\mathbf{w} \in \mathcal{W}_{\text{conv}}} \left\| \mathbf{y}_{1,\mathcal{I}_m^{(j)}} - \mathbf{Y}_{0,\mathcal{I}_m^{(j)}}\mathbf{w} \right\|_2^2,$$

and compute the associated post-treatment counterfactual

$$\mathbf{y}_{0,\text{post}}^{(j)} = \mathbf{Y}_{0,\text{post}}\mathbf{w}^{(j)}.$$

The deviations between the subsample and full-sample estimates are summarized by

$$D^{(j)} = -\sqrt{\frac{T_2}{T_1}} \left(\bar{\mathbf{Y}}_{0,\text{post}}^{\top} \sqrt{m} \left(\mathbf{w}^{(j)} - \mathbf{w}^{\text{SCM}}\right)\right) + \frac{1}{\sqrt{T_2}} \sum_{t>T_1} \varepsilon_t^{(j)},$$

where $\varepsilon_t^{(j)} \sim \mathcal{N}(0, \hat{\sigma}^2)$ are simulated residuals with variance estimated from post-treatment discrepancies. The empirical distribution of $\{D^{(j)}\}_{j=1}^{J}$ serves as an approximation to the sampling distribution of $\hat{\delta}$.

Let $D_{(\alpha/2)}$ and $D_{(1-\alpha/2)}$ denote the $\alpha/2$ and $(1-\alpha/2)$ empirical quantiles of the $J$ draws. The $(1-\alpha)$ confidence interval for the average treatment effect is then

$$\left[\hat{\delta} - \frac{1}{\sqrt{T_2}} D_{(1-\alpha/2)},\ \hat{\delta} - \frac{1}{\sqrt{T_2}} D_{(\alpha/2)}\right].$$

An extension of this approach produces **time-varying uncertainty bands** for the estimated treatment path. Instead of summarizing deviations $D^{(j)}$ into a single average treatment effect, we compute the subsample-based deviations at each post-treatment period $t > T_1$:

$$\tau_t^{(j)} = y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{(j)}, \quad j = 1, \ldots, J.$$

The empirical $\alpha/2$ and $(1-\alpha/2)$ quantiles of $\{\tau_t^{(j)}\}_{j=1}^{J}$ define pointwise lower and upper confidence bands for the treatment effect at each time $t$. This procedure naturally captures how uncertainty evolves over time, reflecting periods when the synthetic control fit is more or less variable across subsamples. The full timewise treatment effect path

$$\hat{\tau}_t = y_{1t} - \mathbf{Y}_{0t}\mathbf{w}^{\text{SCM}}$$

is then reported alongside these bands.

Subsampling primarily captures uncertainty due to the *time dimension*, and is most effective when the pre-treatment period is long, even if the donor pool is relatively small.

### 10.2.4 Bootstrap-Based Confidence Intervals

Bootstrap intervals developed by Arkhangelsky & Hirshberg (2023) resample the *donor units* with replacement. For each iteration $s$, a new donor matrix $\mathbf{Y}_0^{(s)}$ is constructed. The weights are estimated using the full pre-treatment period:

$$\mathbf{w}^{(s)} = \underset{\mathbf{w} \in \mathcal{W}_{\text{conv}}}{\operatorname{argmin}} \left\| \mathbf{y}_1 - \mathbf{Y}_0^{(s)}\mathbf{w} \right\|_2^2,$$

and the post-treatment effect is computed as

$$\hat{\tau}^{(s)} = \frac{1}{T_2} \sum_{t \in \mathcal{T}_2} \left( y_{1t} - \mathbf{Y}_{0t}^{(s)}\mathbf{w}^{(s)} \right).$$

The *bootstrap standard deviation* is

$$\hat{\sigma}_{\text{boot}} = \sqrt{\frac{1}{S-1}\sum_{s=1}^{S}\left(\hat{\tau}^{(s)} - \bar{\hat{\tau}}\right)^2}, \quad \bar{\hat{\tau}} = \frac{1}{S}\sum_{s=1}^{S}\hat{\tau}^{(s)}.$$

The $(1-\alpha)$ bootstrap confidence interval is

$$\text{CI}_{1-\alpha}^{\text{boot}} = \hat{\tau} \pm z_{1-\alpha/2}\,\hat{\sigma}_{\text{boot}},$$

where $\hat{\tau}$ is the treatment effect estimated on the full dataset and $z_{1-\alpha/2}$ is the standard normal quantile. The *bootstrap test statistic* for a null hypothesis $\tau_0$ is

$$t_{\text{boot}} = \frac{\hat{\tau} - \tau_0}{\hat{\sigma}_{\text{boot}}}.$$

Bootstrap inference is particularly stable when the donor pool is large, since resampling provides meaningful variation without generating extreme or unstable synthetic controls.

### 10.2.5 Debiased K-Fold t-Test for SCM

An alternative approach to inference is the *debiased K-fold t-test* by Chernozhukov et al. (2025). This method corrects for the overfitting bias inherent in SCM by using cross-fitting on the pre-treatment periods. Here the pre-treatment periods are split into $K$ consecutive blocks of time periods $H_1, \ldots, H_K$. For each block $k$, we first estimate SCM weights $\mathbf{w}^{(k)}$ on the complement $H_{-k}$ (all pre-periods not in $H_k$):

$$\mathbf{w}^{(k)} = \underset{\mathbf{w}\in\mathcal{W}_{\text{conv}}}{\text{argmin}} \left\|\mathbf{y}_{H_{-k}} - \mathbf{Y}_{0,H_{-k}}\mathbf{w}\right\|_2^2$$

The next step is to compute the *block-specific debiased treatment effect*:

$$\tau_k = \frac{1}{T_1}\sum_{t=T_0}^{T-1}\left(y_t - \mathbf{Y}_{0t}\mathbf{w}^{(k)}\right) - \frac{1}{|H_k|}\sum_{t\in H_k}\left(y_t - \mathbf{Y}_{0t}\mathbf{w}^{(k)}\right)$$

Next, we aggregate across blocks to get the overall estimate:

$$\hat{\tau} = \frac{1}{K}\sum_{k=1}^{K}\tau_k$$

Now, we estimate the standard deviation:

$$\hat{\sigma}_{\hat{\tau}} = \sqrt{1 + \frac{Kr}{T_1}}\sqrt{\frac{1}{K-1}\sum_{k=1}^{K}(\tau_k - \hat{\tau})^2}, \quad r = \min\left(\left\lfloor\frac{T_0}{K}\right\rfloor, T_1\right)$$

The corresponding *debiased t-statistic* for testing $H_0 : \tau = 0$ is

$$t_{\text{debias}} = \frac{\sqrt{K}\,\hat{\tau}}{\hat{\sigma}_{\hat{\tau}}}.$$

The $(1-\alpha)$ confidence interval is

$$\text{CI}_{1-\alpha}^{\text{debias}} = \hat{\tau} \pm t_{1-\alpha/2,\,K-1}\,\frac{\hat{\sigma}_{\hat{\tau}}}{\sqrt{K}},$$

where $t_{1-\alpha/2,\,K-1}$ is the $(1-\alpha/2)$ quantile of the $t$-distribution with $K-1$ degrees of freedom. This approach is particularly effective when the pre-treatment period is short relative to the number of donors, as it reduces overfitting bias by leveraging out-of-sample weight estimation. Here is what we get using the Basque data:

For the Basque data, the debiased t-test yields a significant negative ATE of -0.74 with a 95% CI of (-1.11, -0.37), suggesting a robust treatment effect.

## 10.3 Summary

This chapter has explored a diverse array of validation and inference techniques for SCM, designed to assess the credibility and robustness of counterfactual estimates, particularly in small-$N$ settings where traditional asymptotic methods like those relying on the Central Limit Theorem may fall short.These methods collectively illustrate the breadth of tools available to quantify uncertainty and test model robustness, from capturing temporal trends to addressing donor pool variability. The point of the chapter is not to insist that analysts must use all these techniques, or indeed any specific one, but rather to emphasize the importance of conducting sensitivity analyses tailored to the base model, whatever form that takes or is sensible within the context of a given study. Whether through placebo shifts, resampling strategies, or debiased tests, the effort to explore how estimates hold under different assumptions enhances the reliability of SCM findings, especially in the small-sample settings where it is most often applied.

## 10.4 Problem Set: Validation of Synthetic Control Methods

Below are 10 problem set questions designed to reinforce the key concepts from the chapter on "Validation of Synthetic Control Methods." These questions encourage practical application with the Basque data or similar datasets, promote critical thinking about sensitivity analysis and inference in SCM, and align with the chapter's emphasis on thorough validation without advocating for a single method.

1. **Conceptual Understanding of Placebo Tests**
   Explain the purpose of in-space and in-time placebo tests in the context of Synthetic

Control Methods (SCM). Describe how each test helps identify potential issues with model specification or donor pool selection, using the Basque data example from the chapter as a reference. What limitation do these tests share when applied in small-$N$ settings?

2. **Mathematical Derivation of In-Space Placebo**
Given the SCM weight optimization problem for an in-space placebo: $\mathbf{w}^{SCM,i} = \text{argmin}_{\mathbf{w}\in\mathcal{W}_{conv}\cap\mathbb{R}^{\mathcal{N}_0\setminus\{i\}}} \left\|\mathbf{y}_i - \mathbf{Y}_{\mathcal{N}\setminus\{i\}}\mathbf{w}\right\|_2^2$, where $\mathcal{W}_{conv} = \{\mathbf{w} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0\setminus\{i\}|} : \sum_{j\in\mathcal{N}_0\setminus\{i\}} w_j = 1\}$, derive the placebo effect $\hat{\tau}_{it}$ for a donor unit $i$. How would you interpret a non-zero $\hat{\tau}_{it}$ in the post-treatment period?

3. **In-Time Placebo Sensitivity Analysis**
Using the Basque data from the chapter (available at the provided URL), implement an in-time placebo test by shifting the treatment cutoff $T_0'$ to $T_0 - 3$ instead of $T_0 - 2$. Compute the average placebo effect $\bar{\hat{\tau}}_1'$ over the placebo post-period $\mathcal{T}_0'$ and compare it to the original treatment gap. What does this suggest about the robustness of the SCM fit?

4. **LTO-SCM Justification**
Why mights the standard placebo test of the in-space placebos be underpowered?

5. **Interpreting LTO-SCM Results**
Under what conditions would you use an in-time placebo test? What might some shortcomings of it be>

6. **Block Permutation Inference Implementation**
Why might we prefer to use block permutation instead of using all of the pretreatment period?

7. **Subsampling Confidence Interval Analysis**
Apply the subsampling inference method to the Basque data with a subsample size $m = T_0/2$ and 500 iterations ($J = 500$). Compute the 95% confidence interval for the average treatment effect and compare it to the chapter's result. Discuss how the choice of $m$ influences the interval width.

8. **Bootstrap vs. Subsampling Comparison**
Contrast the bootstrap and subsampling methods for SCM inference using the Basque data. Implement the bootstrap method with 1000 iterations and estimate the 95% confidence interval. How do the sources of variability (donor units vs. time periods) affect the resulting intervals, and which method might be more appropriate given the dataset's characteristics?

9. **Debiased K-Fold t-Test Sensitivity**
Re-run the debiased K-fold t-test on the Basque data with $K = 5$ folds instead of 3, using the provided code. Compare the resulting t-statistic, standard error, and 95% confidence interval to the chapter's output (e.g., t-statistic -8.63, CI (-1.11, -0.37)). What does a change in $K$ suggest about the trade-off between bias correction and precision?

10. **Comprehensive Sensitivity Analysis**
    Design a sensitivity analysis plan for an SCM study using any dataset you wish, incorporating at least three of the methods discussed (e.g., in-time placebo, LTO-SCM, and subsampling). Specify the parameters for each method (e.g., $T_0'$, number of iterations) and outline how you would interpret the results to assess model robustness. Justify your choices based on the study context.

# Part III

# Applications

# Chapter 11

# Control Group Selection for Synthetic Controls

> "*Cause I can't rush decisions, give me a second. All of your pressuring, making me question things, 'Do I want ya?' 'Do I need ya?'*"
> — *Sabrina Claudio*

## 11.1 Introduction

Synthetic controls are typically most valuable in settings with a few treated units and a large number of potential controls. Throughout this book, a recurring theme has been *how* we assign weights to those controls. We have seen that unconstrained OLS tends to overfit and extrapolate, that penalized estimators using the $\ell_1$ and $\ell_2$ norms can regularize these weights, and that even the convex hull constraint in the canonical SCM can be interpreted as a geometric form of regularization. However, we have yet to confront an even deeper question: *which units should be included in the donor pool in the first place?* The quality of any synthetic control depends not only on *how* we combine donors, but also on *whether the donor set itself* is well chosen to represent the untreated counterfactual. In many empirical settings $N_0 > T_0$, which means the SCM optimization may fail to have a unique solution. When this happens, the SCM optimization problem may no longer yield a unique or stable solution. Sometimes, the solution is even undefined. Thus, to use SCM as a weapon in our toolkit, we need to have formalized strategies for how to choose a smaller donor pool from a universe of potential donors to pick from. The remainder of this chapter explores what donor selection strategies look like in practice—how analysts can algorithmically or empirically refine donor pools, and how thoughtful selection can improve both interpretability and inferential credibility in synthetic control analyses.

## 11.2 The Art of Control Group Selection

### 11.2.1 Qualitative Problems

Much of the decisions around donor pool selection are ad-hoc: we typically compare wealthy nations to other wealthy nations (and vice versa), urban areas to other ones, and so on. In other words, given a set of control units to pick from, we oftentimes make decisions about who should be a donor based on domain expertise. However, even with this, control group selection is more difficult than it first appears. I have argued, sometimes quite bitterly, with marketers who believe that domain expertise alone suffices for selecting a control group. It does not. To see why, consider a simple example: suppose we work at Whole Foods and introduce a pricing strategy in one store (say, South Market Street, or SoMa) but not in others. There are roughly 95 Whole Foods locations in California alone. Even with rich metadata and strong intuition about local demographics, competition, and store characteristics, the epistemic question of *which other stores belong in the donor pool* remains unsettled. Well-informed experts can (and do) disagree: "Who should be SoMa's donors? Huntington Beach, West Hollywood, San Jose, Malibu? Should we include Berkeley or Beverly Hills?", and why do we hold *this* preference to *that* one? Notice that every one of these candidate stores (probably) comes with a very plausible narrative. That's exactly why domain expertise, while crucial, is insufficient.

### 11.2.2 Quantitative Problems

Beyond this qualitative problem are two quantitative problems. The first has to do with similarity between the control group and the target series. Recalling the linear factor model:

$$y_{jt}(0) = \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

where $\boldsymbol{\mu}_j$ represents latent factor loadings capturing unit-specific characteristics and the finite sample bound:

$$\epsilon = \boldsymbol{\lambda}_t^\top \Big( \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}) + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt},$$

we know that post-treatment error arises from two sources: (i) pre-treatment idiosyncratic differences projected through the factor space, and (ii) direct post-treatment idiosyncratic mismatch. By restricting the donor pool to similar units (that is, those more with similar latent factor loadings to the treated unit), we reduce the factor-loading mismatch $\boldsymbol{\nu} = \boldsymbol{\mu}_1 - \sum_j w_j \boldsymbol{\mu}_j$, thereby lowering the finite-sample bound on post-treatment error.

Asymptotically, trimming the donor pool to similar units has additional potential benefits. First, by shrinking the numerator term in the asymptotic bias bound, we reduce the overall diversity of factor loadings among the controls. This reduces the "noise" in the factor-loading space, which can otherwise inflate the expected post-treatment bias. Second, the contribution of random idiosyncratic shocks from individual control units is less likely to dominate the

bias. In other words, the synthetic control constructed from a smaller, more similar set of donors is both less variable and more likely to align closely with the true latent characteristics of the treated unit. Formally, the asymptotic bias is bounded by

$$|\mathbb{E}[\epsilon]| \lesssim \bar{\lambda}\, \mathbb{E}\|\boldsymbol{\nu}\|_2,$$

and reducing the factor-loading mismatch $\|\boldsymbol{\nu}\|_2$ by selecting similar donors directly lowers this bound. By carefully selecting the donor pool in this way, we ensure that SCM comparisons are driven by meaningful similarities in latent characteristics rather than merely including all units, enhancing both the interpretability and credibility of the analysis.

### 11.2.3 Challenges with Measuring Similarity

While the logic above motivates restricting the donor pool to units with similar latent factor loadings, two significant challenges immediately arise.

The first central problem is that the factor loadings $\mu_j$ are unobserved. In practice, we rarely have any intuition about how many factors actually matter, how strongly each factor influences the outcome, or the direction and nature of their effects. Without observing these latent structures directly, our idea of "similarity" becomes highly speculative. Even sophisticated dimensionality reduction or factor estimation techniques can only give approximations, and these approximations themselves may introduce bias or instability into the donor selection process.

Now, suppose we try to define similarity empirically. How should we measure it? The mean squared error between one donor's pre-treatment trajectory and another's? Correlation in outcomes? Distance in a factor-estimated space? Each choice has implications: some measures may favor units with low variance, others may emphasize temporal alignment, and yet others may overweight extreme events. More fundamentally, what does it mean for two units' latent factor loadings to be "similar" in an empirically meaningful sense? This question is not just theoretical: it affects which units we include in the donor pool and, by extension, the post-treatment bias of our synthetic control.

In other words, while restricting donors to "similar" units is very appealing, the notion of similarity is inherently fuzzy. Any operational definition introduces additional assumptions, and the sensitivity of results to these assumptions is rarely fully explored.bAny choice of metric implicitly defines an equivalence class of 'similar' units, and different metrics can lead to different donor pools. This challenge lies at the heart of the control group selection problem: we can formalize and bound the theoretical benefits of similarity, but applying these ideas in practice requires judgment, careful testing, and transparency.

Furthermore, even if we had intuition about this, there is another issue we face, namely the cardinality of the donor set. This is in principle a combinatorial problem. Each potential control can either be included or excluded from the donor pool. If we have $N_0$ potential controls, the total number of non-empty subsets (candidate donor sets) is $2^{N_0} - 1$, because each unit represents an independent binary choice:

include or exclude. Suppose we have 4 potential control units: $C_1, C_2, C_3, C_4$. The set of all non-empty subsets can be written explicitly as: $\{C_1\}, \{C_2\}, \{C_3\}, \{C_4\}$ for the individual sets, $\{C_1, C_2\}, \{C_1, C_3\}, \{C_1, C_4\}, \{C_2, C_3\}, \{C_2, C_4\}, \{C_3, C_4\}$ for pairs, $\{C_1, C_2, C_3\}, \{C_1, C_2, C_4\}, \{C_1, C_3, C_4\}, \{C_2, C_3, C_4\}$ for triplets, and of course $\{C_1, C_2, C_3, C_4\}$ as the full donor pool. More compactly, we can write this as

$$\mathcal{P}(\{C_1, C_2, C_3, C_4\}) \setminus \{\emptyset\} = \{S \subseteq \{C_1, C_2, C_3, C_4\} \mid S \neq \emptyset\}.$$

There are $2^4 - 1 = 15$ non-empty subsets. Now suppose we have 9 control units in an area. This gives us 255 candidate donor sets. This is computationally feasible, especially when we use OLS. However, in the Whole Foods example, where there are roughly 30 potential control stores, the number of non-empty subsets jumps to

$$2^{30} - 1 \approx 1.07 \text{ billion}.$$

Anything after this, say 50 American states, is totally infeasible, $2^{50} - 1 \approx 1.12 \times 10^{15}$ subsets. This would require a computational time longer than the lifespan of the universe to compute. Assuming we want the *globally optimal* subset of controls, we are prohibited from finding the solution computationally. Now, you may be wondering "Well Jared, why don't we just use the $\ell_1$ norm to select the donor pool?" Indeed we have discussed the benefits of the $\ell_1$ norm before. However, standard regularization does not solve the problem because the weighting step is itself highly sensitive to which units were made available in the first place. I wish to emphasize once more, the problem we are discussing precedes estimation of weights. Thus, given that we prefer our models to finish running by dinner time (and by the end of our lives certainly), analysts can make use of algorithms to aid them in both the size of the donor pool as well as which ones to include.

## 11.3 Forward Selection for Donor Pool Construction

Forward selection provides a greedy yet highly effective solution to the combinatorial donor-pool problem introduced in Section Chapter 11. It builds the donor set one unit at a time, at each step adding the single donor that—when combined with the units already selected and optimally weighted—most improves pre-treatment fit. We present two versions of the algorithm: Forward DID (FDID), which uses uniform weights, and Forward Synthetic Control (FSCM), which uses convex (non-negative, sum-to-one) weights. Before we apply these to real data, I wish to spell out the algorithmic fine print of both.

## 11.4 Forward DID

Forward DID (FDID) solves the donor-pool selection problem by greedily maximizing pre-treatment fit under uniform weighting and a single intercept. The loss function is

$$\ell_{\text{FDID}}(\widehat{U}) = \min_{\beta \in \mathbb{R}} \left\| \mathbf{y}_1 - \beta \mathbf{1}_{T_0} - \frac{1}{|\widehat{U}|} \sum_{j \in \widehat{U}} \mathbf{y}_j \right\|_2^2 .$$

This is equivalent to maximizing in-sample $R^2$, since minimizing the sum of squared residuals maximizes fit. The ideal donor pool would be

$$\widehat{U}^\star = \operatorname*{argmax}_{\widehat{U} \subseteq \mathcal{N}_0} R^2(\widehat{U}) \quad \Longleftrightarrow \quad \operatorname*{argmin}_{\widehat{U} \subseteq \mathcal{N}_0} \ell_{\text{FDID}}(\widehat{U}).$$

But this is a combinatorial search over $2^{N_0} - 1$ non-empty subsets. Forward DID replaces this exact (infeasible) solution with a classic greedy algorithm. We start with $\widehat{U}_0 = \emptyset$. At the first iteration, we estimate $N_0$ DID models, one for each possible single-control specification. We then choose the donor with the largest pre-treatment $R^2$. This yields $\widehat{U}_1$. At iteration $k$, we add the single control that yields the best improvement in pre-treatment fit:

$$j_k^* = \operatorname*{argmax}_{j \notin \widehat{U}_{k-1}} R^2\Big(\widehat{U}_{k-1} \cup \{j\}\Big),$$

and update the candidate subset:

$$\widehat{U}_k = \widehat{U}_{k-1} \cup \{j_k^*\}.$$

The final FDID donor pool is selected from the greedy path by minimizing the FDID loss:

$$\widehat{U}^{\text{FDID}} = \operatorname*{argmin}_{\widehat{U} \in \{\widehat{U}_1, \ldots, \widehat{U}_{N_0}\}} \ell_{\text{FDID}}(\widehat{U}) = \widehat{U}_{\hat{k}},$$

where

$$\hat{k} = \operatorname*{argmin}_{k = 1, \ldots, N_0} \ell_{\text{FDID}}(\widehat{U}_k).$$

Crucially, this reduces computational complexity from exponential to polynomial. The greedy path requires estimating only

$$1 + 2 + \cdots + N_0 = \frac{N_0(N_0 + 1)}{2}$$

models, instead of $2^{N_0} - 1$. For example, with $N_0 = 50$, the exact search would require approximately $1.1 \times 10^{15}$ models, whereas FDID needs only 1275.

### 11.4.1 Solving Forward DID

The cool thing about solving FDID comes from recalling this proof which demonstrates to us that DID is a kind of centering estimator with a closed form solution, not just for the couterfactual but for the residuals too. Recall that for DID the residuals may be expressed as a centered version of the treated group and donor pool:

$$\mathbf{r} = \mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0} \qquad \Rightarrow \qquad \ell_{\text{DID}}(\mathcal{N}_0) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}\|_2^2$$

where

$$\mathbf{y}_c = \mathbf{y}_1 - \bar{y}_1 \, \mathbf{1}_{T_0}, \qquad \bar{\mathbf{y}}_{c,\mathcal{N}_0} = \bar{\mathbf{y}}_{\mathcal{N}_0} - \bar{m}_{\mathcal{N}_0} \, \mathbf{1}_{T_0}.$$

The key difference for FDID however is that we iteratively build the donor pool by seeing which average of controls maximizes the in-sample R-squared statistic. One solution to this problem is to simply estimate one DID model per each iteration of the FDID design. However, this is expensive: what if there is a more efficient way to do this? As it turns out, we do not recompute the average from scratch each time! The donor pool average, and therefore the residuals, can be maintained with a running mean.

**Definition 11.1** (Running Mean)**.** A *running mean* updates the average incrementally as new observations arrive. For scalars:

$$\bar{x}_{k+1} = \frac{k \cdot \bar{x}_k + x_{k+1}}{k+1}$$

The same formula works for vectors. If $\bar{\mathbf{y}}_k$ is the current average of $k$ pre-treatment trajectories, adding a new donor $j$ gives

$$\bar{\mathbf{y}}_{k+1} = \frac{k \cdot \bar{\mathbf{y}}_k + \mathbf{y}_j}{k+1}$$

Of course, we can express the running mean using a weighted sum representation. Start with the first number $x_1 = 4$. Its mean is trivially

$$\bar{x}_1 = x_1 = 1 \cdot x_1.$$

Now add a second number $x_2 = 8$. Using the running mean formula

$$\bar{x}_2 = \frac{1 \cdot \bar{x}_1 + x_2}{2} = \frac{1 \cdot (1 \cdot x_1) + 1 \cdot x_2}{2} = \frac{1}{2}x_1 + \frac{1}{2}x_2.$$

Next, add a third number $x_3 = 12$. Again applying the running mean formula:

$$\bar{x}_3 = \frac{2 \cdot \bar{x}_2 + x_3}{3} = \frac{2 \cdot \left(\frac{1}{2}x_1 + \frac{1}{2}x_2\right) + x_3}{3} = \frac{1}{3}x_1 + \frac{1}{3}x_2 + \frac{1}{3}x_3.$$

In general, each running mean can be seen as a weighted sum of all the numbers included so far, where the weights sum to 1:

$$\bar{x}_k = \sum_{i=1}^{k} w_i x_i, \quad \text{with } \sum_{i=1}^{k} w_i = 1.$$

This shows that we did not need to sum all three numbers from scratch; we just updated the previous mean with the new value. We can even do this with vectors. Say we have these vectors

$$\mathbf{y}_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \ \mathbf{y}_2 = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}.$$

The arithmetic mean of these is

$$\bar{\mathbf{y}}_2 = \begin{bmatrix} 1.5 \\ 3.5 \\ 5.5 \end{bmatrix}$$

Now let's add $\mathbf{y}_3 = \begin{bmatrix} 0 \\ 2 \\ 4 \end{bmatrix}$:

$$\bar{\mathbf{y}}_3 = \frac{2 \cdot \bar{\mathbf{y}}_2 + \mathbf{y}_3}{3} = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}$$

We can exploit this to make the forward selection procedure very fast. At iteration $k$ we have selected set $\widehat{U}_k$ with size $m_k = |\widehat{U}_k|$ and running average

$$\bar{\mathbf{y}}_k = \frac{1}{m_k} \sum_{j \in \widehat{U}_k} \mathbf{y}_j \in \mathbb{R}^{T_0}.$$

When we consider adding candidate donor $j \notin \widehat{U}_k$, the updated average is:

$$\bar{\mathbf{y}}_{k \cup j} = \frac{m_k \cdot \bar{\mathbf{y}}_k + \mathbf{y}_j}{m_k + 1}.$$

Its time-centered version is

$$\bar{\mathbf{y}}_{c,k\cup j} = \bar{\mathbf{y}}_{k\cup j} - \underbrace{\left(\frac{1}{T_0}\mathbf{1}_{T_0}^\top \bar{\mathbf{y}}_{k\cup j}\right)}_{\bar{m}_{k\cup j}} \mathbf{1}_{T_0}.$$

From the proof, it logically follows that the pre-treatment loss for an additional donor added is simply

$$\boxed{\ell_{\text{FDID}}(\widehat{U}_k \cup \{j\}) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,k\cup j}\|_2^2}$$

or, expanded,

$$\ell_{\text{FDID}}(\widehat{U}_k \cup \{j\}) = \|\mathbf{y}_c\|_2^2 + \|\bar{\mathbf{y}}_{c,k\cup j}\|_2^2 - 2\,\mathbf{y}_c^\top \bar{\mathbf{y}}_{c,k\cup j}.$$

We just maintain one $T_0$-vector ($\bar{\mathbf{y}}_k$) and one scalar ($m_k$), update them with a rank-1 formula when we add a donor, and compute a distance to the fixed $\mathbf{y}_c$.

### 11.4.2 Batch Solving FDID

Now we will vectorize this update across all remaining candidates simultaneously using a single matrix operation, turning what would naïvely be an $O(N_0^2 T_0)$ procedure into something effectively $O(N_0 T_0)$. Let $C_k$ denote the set of remaining candidate donors at iteration $k$. Let $\mathbf{Y}_{\text{cand}} \in \mathbb{R}^{T_0 \times |C_k|}$ be the matrix whose $j$-th column is the pre-treatment trajectory $\mathbf{y}_j$ for candidate donor $j \in C_k$. For notational clarity define $\bar{\mathbf{y}}_{k\cup j}$ to be the updated running mean obtained by adding candidate $j$ to the current selected set $\widehat{U}_k$, and let $\bar{\mathbf{Y}}_{k\cup\cdot}$ be the matrix whose $j$-th column is $\bar{\mathbf{y}}_{k\cup j}$ (one column per candidate $j \in C_k$).

The matrix of all updated averages (one column per candidate) is

$$\boxed{\bar{\mathbf{Y}}_{k\cup\cdot} = \frac{m_k\,\bar{\mathbf{y}}_k\,\mathbf{1}_{|C_k|}^\top + \mathbf{Y}_{\text{cand}}}{m_k + 1} \in \mathbb{R}^{T_0 \times |C_k|}}$$

This is a single rank-1 update broadcast across all candidates. Time-demean every column in one operation:

$$\bar{\mathbf{Y}}_{c,k\cup\cdot} = \bar{\mathbf{Y}}_{k\cup\cdot} - \mathbf{1}_{T_0}\left(\frac{1}{T_0}\mathbf{1}_{T_0}^\top \bar{\mathbf{Y}}_{k\cup\cdot}\right) \in \mathbb{R}^{T_0 \times |C_k|}.$$

Form the residual matrix relative to the centered treated trajectory $\mathbf{y}_c$:

$$\mathbf{R} = \mathbf{y}_c\,\mathbf{1}_{|C_k|}^\top - \bar{\mathbf{Y}}_{c,k\cup\cdot} \in \mathbb{R}^{T_0 \times |C_k|}.$$

The pre-treatment loss for adding candidate $j$ is the squared norm of the $j$-th column of $\mathbf{R}$:

$$\boxed{\ell_{\text{FDID}}(\widehat{U}_k \cup \{j\}) = \|\mathbf{R}_{:,j}\|_2^2.}$$

Equivalently, the $R^2$ value for candidate $j$ at iteration $k$ is

$$\boxed{R_k^2(j) = 1 - \frac{\|\mathbf{R}_{:,j}\|_2^2}{\|\mathbf{y}_c\|_2^2}.}$$

Select the best donor by choosing the candidate that maximizes $R_k^2(j)$ (equivalently minimizes the loss):

$$j_k^* = \underset{j \in C_k}{\text{argmax}}\ R_k^2(j) \ = \ \underset{j \in C_k}{\text{argmin}}\ \|\mathbf{R}_j\|_2^2.$$

Append the chosen donor to the selected set, update the running mean and counter,

$$\bar{\mathbf{y}}_{k+1} \leftarrow \bar{\mathbf{y}}_{k \cup j_k^*}, \qquad m_{k+1} \leftarrow m_k + 1,$$

remove $j_k^*$ from $C_k$, and repeat the batch computation on the new candidate set.

## 11.5 Forward Synthetic Control (FSCM)

Note that unlike FDID, FSCM needs to re-solve a constrained QP at every step—there is no closed-form update analogous to the running mean. Unfortunately, FSCM has no closed form solution to exploit, making FSCM slower than FDID but still polynomial and tractable. In FSCM we keep the standard synthetic control weight space:

$$\mathcal{W}_{\text{FSCM}}(\widehat{U}) = \left\{ \mathbf{w} \in \mathbb{R}_{\geq 0}^{N_0} : \sum_{j \in \widehat{U}} w_j = 1,\ w_j = 0 \text{ for } j \notin \widehat{U} \right\}.$$

The inner (pre-treatment) loss for a candidate subset $\widehat{U}$ is the usual SCM objective:

$$\boxed{\ell_{\text{FSCM}}(\widehat{U}) \ := \min_{\mathbf{w} \in \mathcal{W}_{\text{FSCM}}(\widehat{U})} \|\mathbf{y}_1 - \mathbf{Y}_{\widehat{U}}\mathbf{w}\|_2^2},$$

where $\mathbf{Y}_{\widehat{U}} \in \mathbb{R}^{T_0 \times |\widehat{U}|}$ stacks the pre-treatment outcome vectors of units in $\widehat{U}$. This is a small convex quadratic program — fast for moderate $|\widehat{U}|$, but must be re-solved at every forward step. The greedy forward selection algorithm is identical in structure to FDID. We start with $\widehat{U}_0 = \emptyset$, or an empty set of control units. Now, at iteration $k$, for each remaining candidate $j \notin \widehat{U}_{k-1}$, solve the inner SCM on $\widehat{U}_{k-1} \cup \{j\}$ and compute $\ell_{\text{FSCM}}(\widehat{U}_{k-1} \cup \{j\})$. Of these, we permanently add the $j$ that yields the smallest inner loss. We repeat until all control units have been added.

## 11.6 Barcelona Case Study

In 2015, Barcelona faced a classic urban policy dilemma: how to balance booming tourism with the well-being of residents and the local economy. The city implemented a one-year moratorium on new hotel construction, affecting eight of eleven planned luxury hotel projects. Despite the ban, Barcelona remained an attractive destination for international investors — it already had more 5-star hotels than London or Paris, and demand for premium accommodation continued to rise. Policymakers wanted to understand whether restricting supply would actually reduce hotel prices, or whether market pressures would offset the intended effect.

Bel et al. (2021) study the effect of this policy using synthetic controls. My data are slightly different from theirs: I use weekly observations, consisting of 236 pre-treatment periods and 111 post-treatment periods, with 83 control units. Among these, 16 cities are located along the Mediterranean coast, which we have labeled in the dataset. However, note that Booking.com anonymized the city names, meaning we cannot rely on subjective judgments to select the donor pool.

From an empirical standpoint, this poses a challenging question: how can we estimate the causal impact of the moratorium on hotel prices? Which cities should we consider as appropriate comparisons, and how should we weight them? Synthetic control methods provide a principled framework for answering these questions, allowing analysts to construct credible counterfactuals even when subjective information is limited.

One natural consideration is whether we should restrict the donor set to cities along the Mediterranean coast. These cities may share similar climate, culture, and tourism potential with Barcelona, making them more likely to exhibit comparable dynamics in hotel prices absent the policy intervention. Formally, this aligns with the intuition behind the linear factor model:

## 11.7 Synthesizing the Asymptotic and Finite-Sample Perspectives

The Barcelona case highlights the tension between designing a credible donor pool ex ante and evaluating the statistical reliability of a synthetic control ex post. Both the asymptotic and finite-sample perspectives help clarify what is at stake when choosing among possible donor sets—especially given the anonymized Booking.com data and the ambiguous status of the 16 Mediterranean cities.

From the perspective of the linear factor model, restricting the donor pool to Mediterranean cities is intuitively appealing. These cities plausibly share latent factors with Barcelona: climate-driven seasonality, regional holiday shocks, cultural amenities, and similar patterns of baseline tourist inflows. In an idealized asymptotic setting (with $T_0 \to \infty$), such similarities matter because SCM's consistency hinges on matching the treated unit's factor loadings. Under the asymptotic bound, any residual imbalance in latent factors decays as the length of the pre-treatment period grows. With sufficiently long pre-treatment data, even rough structural similarity can be enough for the synthetic control to converge to the true counterfactual path.

But the Barcelona application is a finite-sample problem. With 236 pre-treatment weeks, the asymptotic intuition is informative but not dispositive. The finite-sample bound makes this explicit: post-treatment bias depends jointly on (i) how well the donor pool spans the treated unit's latent factors, and (ii) how closely the synthetic control fits Barcelona in the pre-treatment period. Restricting the donor pool to Mediterranean cities cuts both ways. A more homogeneous set may reduce latent-factor mismatch, lowering bias—if these cities genuinely share Barcelona's structure. Yet shrinking the donor pool from 83 to 16 units reduces the dimensionality of the convex hull, potentially making it harder to achieve a tight pre-treatment fit. If the restricted pool cannot reproduce Barcelona's weekly price dynamics, finite-sample imbalance increases and estimation quality worsens. This is precisely the tradeoff the finite-sample bound formalizes. The question is not whether the Mediterranean subset is "better" in an informal sense, but whether its geometry and variation allow a synthetic control that achieves sufficiently low pre-period error. The asymptotic and finite-sample perspectives therefore motivate a unified empirical strategy:

Asymptotics justify why Mediterranean donors might be attractive: they are likely to share the treated unit's latent structure. Finite-sample logic emphasizes that plausibility is not enough—the donor set must actually fit the data well.

Together, these perspectives illustrate a core insight of synthetic control methods: credible identification requires alignment between the population story (which units plausibly share the underlying factors) and the sample story (which convex combination best reproduces the treated unit's observed history). When these align, SCM can isolate the causal effect of Barcelona's moratorium even in an anonymized, high-frequency dataset with a deceptively large donor pool.

At the same time, it is important to acknowledge the limits of this reasoning. Whether viewed asymptotically or in finite samples, the Mediterranean restriction is a very sensible design choice—yet still fundamentally arbitrary. Because the Booking.com data anonymize city names, we cannot verify whether any particular city is truly comparable; we only observe their time-series behavior. Treating the Mediterranean subset as the correct donor pool is ultimately an informed, defensible guess, not an empirically testable fact. Synthetic control methods are built to handle precisely this kind of uncertainty: they allow economic intuition to guide donor selection while letting the data-driven weighting mechanism prevent us from locking in an arbitrary restriction that might exclude cities carrying meaningful predictive information.

### 11.7.1 Weights

First let's consider the weights produced by FSCM and FDID.

The FSCM results show that Mediterranean donors dominate the synthetic control, contributing about 88.6% of the total effective weight, while non-Mediterranean donors contribute the remaining 11.4%. At the individual donor level, only a small subset carries meaningful weight: Donor 82 (32.8%), Donor 81 (29.6%), Donor 6 (13.5%), and Donor 40 (12.8%) are the most influential. Moderate contributors include Donor 77 (7.5%), Donor 61 (2.7%), and Donor 4 (1.2%), while Donors 56, 62, and 60 receive zero weight. The key takeaway

is that intuition aligns partially—but not perfectly—with the data. Mediterranean cities heavily influence the synthetic control, but FSCM does not rely exclusively on them; several non-Mediterranean donors also play a non-trivial role. If one followed pure intuition and discarded non-Mediterranean donors entirely, about 11% of useful predictive weight would be lost, resulting in a worse pre-treatment fit and potentially biased estimates. For the five donors selected by FDID (Donor 30, Donor 40, Donor 80, Donor 81, Donor 82), 4 (80%) are Mediterranean. This aligns with intuition, but notice that 20% are non-Mediterranean, indicating that even donors we might have initially ignored can play an important role. Intuition guides us, but FDID and FSCM both ensure we don't overlook useful information that strengthens the counterfactual.

### 11.7.2 Estiamnds and Fit

Now let's consdier the actual ATT and fit. When comparing different donor pools (full vs Mediterranean-only) and methods (FDID, DID, Convex SCM, Forward SCM), a few patterns emerge. Across the board, ATT estimates are broadly similar between the full donor pool and the Mediterranean-only subset, though the Mediterranean-only pool tends to produce slightly smaller ATT values for FDID. For example, FDID gives 12.99 with the full pool versus 11.64 with Mediterranean donors. This suggests that while Mediterranean donors carry most of the signal, non-Mediterranean donors still contribute meaningfully.

In terms of pre-treatment fit, measured by RMSE and $R^2$, FDID consistently improves over simple DID, yielding higher $R^2$ and lower RMSE. For instance, FDID with the full donor pool has an $R^2$ of 0.853, compared with 0.327 for DID. Restricting to Mediterranean donors slightly improves FDID's $R^2$ (0.859) while reducing RMSE marginally, showing that FDID efficiently captures the main patterns even in a smaller, targeted donor pool. Convex SCM and Forward SCM achieve the best pre-treatment fit overall, with $R^2$ around 0.898–0.894 and RMSE slightly lower than FDID. These methods are particularly robust with larger donor pools, but FDID provides a competitive alternative with fewer selected donors, balancing interpretability and accuracy. Overall, the results illustrate that while domain knowledge (Mediterranean vs full donor pool) matters, FDID and FSCM's algorithmic selection ensures that suboptimal donors are not discarded, capturing essential variation efficiently.

## 11.8 Bayesian Methods for Donor Screening

Of course, other methods may be used to select a donor pool as well. In fact, we can build from the in-space placebo idea we discussed previously.

The central idea of Bayesian donor screening is to evaluate the suitability of each donor by treating it as a pseudo-treated unit. For a given donor $j \in \mathcal{N}_0$, we define the pseudo-treated vector $\mathbf{y}_j$ and the remaining donors $\mathcal{N}_0 \setminus \{j\}$, forming the matrix $\mathbf{Y}_0^{(-j)}$. On the pre-treatment period, we fit a Bayesian synthetic control model:

$$\mathbf{y}_{j,\mathcal{T}_1} \sim \mathcal{N}\Big(\alpha_j + \mathbf{Y}_{0,\mathcal{T}_1}^{(-j)}\mathbf{w}_j, \sigma_j^2 I\Big),$$

where the weight vector $\mathbf{w}_j$ has a Dirichlet prior, the intercept $\alpha_j$ has a diffuse Gaussian prior, and the noise term $\sigma_j$ has a half-normal prior.

It is worth pausing to explain what these terms mean. The prior represents our initial beliefs about the parameters before seeing any data. For instance, the Dirichlet prior encodes that weights are non-negative and sum to one, without favoring any donor at the outset. In the Bayesian synthetic control context, the posterior is the updated belief about the model parameters after seeing the pre-treatment data. For each donor, we start with prior beliefs about the weights assigned to other donors, the intercept, and the noise. These priors reflect what seems plausible before observing any data: for example, that all donor weights are non-negative and sum to one, that the intercept could be anywhere, and that the noise is positive but not extremely large.

Once we observe the pre-treatment outcomes for a donor, we can combine this information with our priors to form the posterior. The posterior answers the question "given what we saw previously in the data, which combinations of weights, intercept, and noise are most plausible?" In other words, it is a probability distribution over all possible parameter values, reflecting both the prior beliefs and the observed data.

Computing the posterior is generally not possible analytically, so we use Markov Chain Monte Carlo (MCMC) methods. MCMC generates a sequence of parameter values that approximates the full posterior distribution. Each draw is a plausible combination of weights, intercept, and noise, consistent with both prior beliefs and the observed data. For high-dimensional models like the Bayesian SCM, the No-U-Turn Sampler (NUTS) is particularly effective. NUTS automatically adapts its path through the parameter space, avoiding inefficient "turning back" steps and efficiently exploring complex posteriors. Before using the samples to summarize the posterior, MCMC runs through a warm-up phase. During this period, early draws are discarded as the sampler adjusts to the shape of the posterior, ensuring that the remaining samples reliably reflect the true posterior distribution. Each sample is one plausible combination of weights, intercept, and noise, consistent with both the prior and the data. By looking at many such samples, we can summarize the posterior: for example, we can compute the average predicted outcome or construct a credible interval. A credible interval is a range derived from the posterior that contains a specified proportion of probable values for a quantity of interest, such as the predicted outcome for a donor. Unlike a classical confidence interval, a credible interval can be interpreted as "given our model and data, there is a 95% probability that the true value lies within this range."

Using posterior draws from this model, we generate predictive outcomes for the first $k$ post-treatment periods $\mathcal{T}_{2,k} \subseteq \mathcal{T}_2$:

$$\tilde{y}_{jt} \sim \mathcal{N}\Big(\alpha_j + (\mathbf{Y}_0^{(-j)})_t \mathbf{w}_j, \sigma_j^2\Big), \quad t \in \mathcal{T}_{2,k}.$$

We construct $(1-\epsilon)$ credible intervals $\mathcal{C}_{jt} = [\tilde{y}_{jt}^{\text{lo}}, \tilde{y}_{jt}^{\text{hi}}]$ and define an indicator function

$$\mathbf{1}_{jt} = \begin{cases} 1 & \text{if } y_{jt} \in \mathcal{C}_{jt} \\ 0 & \text{otherwise.} \end{cases}$$

This checks whether the actual donor outcome falls within the model's predictive interval. To emphasize more recent post-treatment periods, we introduce exponentially decaying weights $w_t$ and define a weighted predictive score:

$$f(\mathcal{T}_{2,k}, j) = \sum_{t \in \mathcal{T}_{2,k}} w_t \mathbf{1}_{jt}, \quad w_t = \frac{e^{-\lambda(t-T_0-1)}}{\sum_{s \in \mathcal{T}_{2,k}} e^{-\lambda(s-T_0-1)}}.$$

This defines a function $f$ mapping a donor and a set of post-treatment periods to a number between 0 and 1, capturing how well the donor's outcomes are predicted by its peers. Donors are retained if their score meets a threshold $\tau$, forming the subset

$$\mathcal{N}_{\text{kept}} = \{j \in \mathcal{N}_0 : f(\mathcal{T}_{2,k}, j) \geq \tau\}.$$

With this screened set of donors, we fit the final Bayesian synthetic control for the treated unit. Over the pre-treatment periods:

$$\mathbf{y}_{1,\mathcal{T}_1} \sim \mathcal{N}\Big(\alpha + \mathbf{Y}_{0,\mathcal{T}_1}^{(\mathcal{N}_{\text{kept}})}\mathbf{w}, \sigma^2 I\Big),$$

and the posterior predictive distribution over the post-treatment periods is

$$\mathbf{y}_{1,\mathcal{T}_2} \sim \mathcal{N}\Big(\alpha + \mathbf{Y}_{0,\mathcal{T}_2}^{(\mathcal{N}_{\text{kept}})}\mathbf{w}, \sigma^2 I\Big),$$

where $\mathbf{Y}_0^{(\mathcal{N}_{\text{kept}})}$ denotes the donor matrix restricted to the columns in $\mathcal{N}_{\text{kept}}$.

This procedure formalizes the intuition that a valid donor should be predictable from its peers, and that only those donors whose post-treatment behavior consistently falls within posterior predictive intervals contribute to constructing the synthetic control. By weighting more recent periods more heavily, the method prioritizes donors that accurately capture near-term trends, which are most informative for estimating the treatment effect.

Abadie, A., Diamond, A., & Hainmueller, J. (2010). Synthetic control methods for comparative case studies: Estimating the effect of california's tobacco control program. *Journal of the American Statistical Association*, *105*(490), 493–505. https://doi.org/10.1198/jasa.2009.ap08746

Abadie, A., Diamond, A., & Hainmueller, J. (2015). Comparative politics and the synthetic control method. *American Journal of Political Science*, *59*(2), 495–510. https://doi.org/https://doi.org/10.1111/ajps.12116

Agarwal, A., Shah, D., Shen, D., & Song, D. (2021). On robustness of principal component regression. *Journal of the American Statistical Association*, *116*(536), 1731–1745. https://doi.org/10.1080/01621459.2021.1928513

Amjad, M., Shah, D., & Shen, D. (2018). Robust synthetic control. *Journal of Machine Learning Research*, *19*(22), 1–51. http://jmlr.org/papers/v19/17-777.html

Arkhangelsky, D., Athey, S., Hirshberg, D. A., Imbens, G. W., & Wager, S. (2021). Synthetic difference-in-differences. *American Economic Review*, *111*(12), 4088–4118. https://doi.org/10.1257/aer.20190159

Arkhangelsky, D., & Hirshberg, D. (2023). *Large-sample properties of the synthetic control method under selection on unobservables*. https://arxiv.org/abs/2311.13575

Baker, A., Callaway, B., Cunningham, S., Goodman-Bacon, A., & Sant'Anna, P. H. C. (2025). *Difference-in-differences designs: A practitioner's guide*. https://arxiv.org/abs/2503.13323

Bel, G., Joseph, S., & Mazaira-Font, F. A. (2021). The effects of moratoriums on hotel building: An anti-tourism measure, or rather protection for local incumbents? *Applied Economics Letters*, *29*(14), 1319–1324. https://doi.org/10.1080/13504851.2021.1927958

Brabander, E. de, Juodis, A., & Szini, G. M. (2025). On the use of synthetic difference-in-differences approach with (-out) covariates: The case study of brexit referendum. *Econometric Reviews*, *44*(10), 1617–1646. https://doi.org/10.1080/07474938.2025.2530649

Chatterjee, S. (2015). Matrix estimation by Universal Singular Value Thresholding. *The Annals of Statistics*, *43*(1), 177–214. https://doi.org/10.1214/14-AOS1272

Chernozhukov, V., Wuthrich, K., & Zhu, Y. (2025). *Debiasing and t-tests for synthetic control inference on average causal effects*. https://arxiv.org/abs/1812.10820

Chernozhukov, V., Wüthrich, K., & Zhu, Y. (2021). An exact and robust conformal inference method for counterfactual and synthetic controls. *Journal of the American Statistical Association*, *116*(536), 1849–1864. https://doi.org/10.1080/01621459.2021.1920957

Cunningham, S. (2021). *Causal inference: The mixtape*. Yale University Press. https://mixtape.scunning.com/

Donoho, D., Gavish, M., & Romanov, E. (2023). ScreeNOT: Exact MSE-optimal singular value thresholding in correlated noise. *The Annals of Statistics*, *51*(1), 122–148. https://doi.org/10.1214/22-AOS2232

Facure Alves, M. (2022). *Causal inference for the brave and true*. https://matheusfacure.github.io/python-causality-handbook

Filipovic, D., & Schneider, P. (2025). *Fundamental properties of linear factor models*. https://arxiv.org/abs/2409.02521

Gavish, M., & Donoho, D. L. (2014). The optimal hard threshold for singular values is $4/\sqrt{3}$. *IEEE Transactions on Information Theory*, *60*(8), 5040–5053. https://doi.org/10.1109/TIT.2014.2323359

Holland, P. W. (1986). Statistics and causal inference. *Journal of the American Statistical Association*, *81*(396), 945–960. https://doi.org/10.1080/01621459.1986.10478354

Huntington-Klein, N. (2025). *The effect: An introduction to research design and causality* (2nd ed.). Chapman; Hall/CRC. https://theeffectbook.net/

Lei, L., & Sudijono, T. (2025). *Inference for synthetic controls via refined placebo tests*. https://arxiv.org/abs/2401.07152

Li, K. T. (2020). Statistical inference for average treatment effects estimated by synthetic control methods. *Journal of the American Statistical Association*, *115*(532), 2068–2083.

https://doi.org/10.1080/01621459.2019.1686986

Li, K. T., & Shankar, V. (2024). A two-step synthetic control approach for estimating causal effects of marketing events. *Management Science*, *70*(6), 3734–3747. https://doi.org/10.1287/mnsc.2023.4878

Liao, C., Shi, Z., & Zheng, Y. (2025). *A relaxation approach to synthetic control*. https://arxiv.org/abs/2508.01793

Nguyen, M. (2025). Synthetic control. In *A guide on data analysis*. https://bookdown.org/mike/data_analysis/sec-synthetic-control.html.

Rubin, D. B. (2008). For objective causal inference, design trumps analysis. *The Annals of Applied Statistics*, *2*(3), 808–840. https://doi.org/10.1214/08-AOAS187

Shi, Z. (n.d.). *Lecture notes on econometrics*. https://zhentaoshi.github.io/Econ5121A/.

Stöffelbauer, A. (2023). *Causal inference using synthetic controls*. https://medium.com/data-science-at-microsoft/causal-inference-using-synthetic-controls-d96a890c83a7.

Tocqueville, A. de, Mansfield, H. C., & Winthrop, D. (2012). *Democracy in america*. University of Chicago Press. https://books.google.com/books?id=obyzxRmY7nEC